

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



DEPARTMENT OF COMPUTER SCIENCE

REPORT OF PROJECT 2

Project: Data Fitting and Ordinary Least Squares Method

Lecturer:	ThS. Võ Nam Thực Đoàn	
	ThS. Lê Nhật Nam	
Leader:	Khang Phúc Nguyen	24120068
Members:	Nghia Trong Hoang	24120103
	Nhat Hoang Mai	24120109
	Nhat Hoang Nguyen	24120110
	Long Nhat Phung Vo	24120088

Ho Chi Minh City, 4/2026

Table of Contents

1	Phần 1: Lý Thuyết và Minh Họa	1
1.1	Bài Toán Data Fitting và Phương Pháp OLS	1
1.1.1	Phát Biểu Bài Toán Data Fitting Tổng Quát	1
1.1.1.a	Đặt vấn đề	1
1.1.1.b	Mô hình hồi quy tuyến tính	1
1.1.2	Các Giả Thiết Gauss–Markov	2
1.1.3	Hàm Mất Mát RSS và Chứng Minh Nghiệm OLS	3
1.1.3.a	Hàm mất mát RSS	4
1.1.3.b	Chứng minh nghiệm OLS (Normal Equations)	5
1.1.4	Ma Trận Chiều (Hat Matrix)	7
1.1.4.a	Định nghĩa và hình học	7
1.1.4.b	Các tính chất của Hat Matrix	8
1.1.5	Ước Lượng Phương Sai Nhiều σ^2	9
1.1.6	Định Lý Gauss–Markov (BLUE)	9
1.1.7	Đánh Giá Mô Hình	11
1.1.7.a	Phân rã phương sai và hệ số xác định	11
1.1.7.b	Kiểm định giả thuyết	11
1.1.7.c	Chẩn đoán phần dư	13
1.1.8	Các Vấn Đề Nâng Cao trong Data Fitting	13
1.1.8.a	Đa cộng tuyến và Tác động của nó	14
1.1.8.b	Hồi quy được chuẩn hóa (Regularized Regression)	15
1.2	Cài Đặt Python	17
1.2.1	Hàm <code>ols_fit(X, y)</code>	17
1.2.1.a	Mô tả	17
1.2.1.b	Pseudo-code	17
1.2.1.c	Hàm <code>hat_matrix(X)</code>	17
1.2.2	Chuẩn hóa Ridge Regression	18
1.2.2.a	Generalized Cross-Validation cho Ridge	19
1.2.3	Đánh giá Mô hình	21
1.2.4	Hồi Quy Lasso và Thuật Toán Coordinate Descent	21



1.2.4.a	Bài Toán Tối Ưu và Lý Do Không Tồn Tại Nghiệm Dạng Đóng	21
1.2.4.b	Soft-Thresholding: Nghiệm Giải Tích của bài toán con . .	22
1.2.4.c	Thuật Toán Coordinate Descent và Tính Hội Tụ	23
1.2.4.d	Quỹ Đạo Chuẩn Hóa	25
1.2.5	Suy diễn Thống kê cho Hệ số	27
1.2.6	Phát hiện Đa cộng tuyến qua VIF	27
1.2.7	Kiểm chứng Implementation From Scratch	28
1.3	Định Lý Gauss–Markov và Kiểm Chứng Monte Carlo	29
1.3.1	Các Giả Thiết Gauss–Markov	29
1.3.1.a	Bối cảnh	29
1.3.1.b	Chẩn đoán trực quan giả thiết GM1	30
1.3.2	Định Lý Gauss–Markov (BLUE)	31
1.3.2.a	Phát biểu	31
1.3.2.b	Chứng minh	31
1.3.3	Minh Họa Định Lý Gauss–Markov bằng Mô Phỏng Monte Carlo	33
1.3.3.a	Ý tưởng	33
1.3.3.b	Công thức ước lượng Monte Carlo	33
1.3.3.c	Thiết lập thực nghiệm	35
1.3.3.d	Cài đặt Python	35
1.3.3.e	Kết quả mô phỏng	35
1.3.3.f	Kiểm chứng với NumPy và scikit-learn	38
2	Phần 2: Ứng Dụng Thực Tế	39
2.1	Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế	39
2.1.1	Mô Tả Dataset	39
2.1.1.a	Tanzania Tourism Expenditure Dataset	39
2.1.1.b	Missing Values	40
2.1.2	Phân Tích Dữ Liệu Khám Phá (EDA)	40
2.1.2.a	Phân Bố Biến Mục Tiêu	40
2.1.2.b	Phân Bố Các Biến Số	41
2.1.2.c	Ma Trận Tương Quan	42
2.1.2.d	Phân Tích Outliers Qua Boxplot	44



2.1.2.e	Kiểm Tra Đa Cộng Tuyến (VIF)	45
2.1.2.f	Tương Quan Các Biến với Mục Tiêu	47
2.1.2.g	Phát Hiện Data Shift (Kolmogorov–Smirnov Test)	48
2.1.3	Preprocessing Dữ Liệu	49
2.1.3.a	Xử Lý Giá Trị Thiếu	49
2.1.3.b	Mã Hóa Biến Phân Loại	50
2.1.3.c	Chuẩn Hóa Đặc Trưng	51
2.1.3.d	Dữ Liệu Sau Xử Lý	51
2.1.4	Xây Dựng và Đánh Giá Mô Hình	52
2.1.4.a	Mô Hình 1: Ordinary Least Squares (OLS)	52
2.1.4.b	Mô Hình 2: Ridge Regression	53
2.1.4.c	Mô Hình 3: Lasso Regression	53
2.1.4.d	Mô Hình 4: ElasticNet	54
2.1.4.e	Mô Hình 5: Polynomial Regression kết hợp ElasticNet	55
2.1.4.f	Mô Hình 6: Kernel Ridge Regression (KRR)	56
2.1.4.g	Đánh Giá và So Sánh Các Mô Hình	56
2.1.5	Cross-Validation và Lựa Chọn Siêu Tham Số	58
2.1.5.a	K-Fold Cross-Validation	58
2.1.5.b	Lựa Chọn Siêu Tham Số λ cho Ridge	59
2.1.6	Phân Tích Tầm Quan Trọng Đặc Trưng	60
2.1.6.a	Top 10 Đặc Trưng Quan Trọng Nhất	60
2.1.7	Kết Quả Mô Hình Hồi Quy	61
2.1.7.a	So Sánh Hệ Số Giữa Các Phương Pháp	61
2.1.7.b	Hiệu Năng Cross-Validation	62
2.1.7.c	Chẩn Đoán Phần Dư	62
2.1.8	Triển Khai Python	64
2.1.8.a	Pipeline Tổng Thể	65
2.1.9	Phân Tích SHAP — Giải Thích Mô Hình	65
2.1.9.a	Nền Tảng Lý Thuyết của SHAP	66
2.1.9.b	SHAP Summary Plot — Phân Bố Tác Động	66
2.1.9.c	SHAP Feature Importance — Tầm Quan Trọng Tổng Thể	67
2.1.9.d	SHAP Waterfall — Giải Thích Từng Prediction	68
2.1.10	Kết Luận và Đề Xuất	69



2.1.10.a	Kết Luận Chính	69
2.1.10.b	Đề Xuất Cải Thiện	70
3	Kết Luận Chung	72
3.1	Tóm Tắt Kết Quả	72
3.2	Bài Học Rút Ra	72
3.3	Hướng Mở Rộng	73
3.4	Phân Công Nhóm	73

1 Phần 1: Lý Thuyết và Minh Họa

1.1 Bài Toán Data Fitting và Phương Pháp OLS

1.1.1 Phát Biểu Bài Toán Data Fitting Tổng Quát

Bài toán Data Fitting đã có nền tảng toán học và lịch sử từ lâu đời, nó luôn đi chung với sự phát triển của phương pháp **Bình Phương Tối Thiểu (OLS)**. Bài toán được hình thành được đặt ra không chỉ để thỏa mãn niềm đam mê toán học của các nhà nghiên cứu mà nó còn có mục đích rất thực tiễn trong các ngành liên quan về thiên văn học và trắc địa vào cuối thế kỷ 18 và đầu thế kỷ 19.

1.1.1.a Đặt vấn đề

Vào những năm 1790, để có thể thiết lập ra hệ mét ¹ thì ông đã phải đo đạc chính xác chiều dài **cung kinh tuyến** từ cực Bắc đến Xích đạo đi qua Paris [1]. Họ đã không trực tiếp mà họ đo một đoạn cung kinh tuyến, cụ thể là đoạn từ *Dunkirk* đến *Barcelona*, rồi dùng mô hình toán học để suy ra độ dài **một phần tư kinh tuyến Trái Đất**.

Do đó ta có thể thấy rằng, bài toán **Data Fitting** đã xuất hiện từ rất lâu và nó đã trở thành một trong những bài toán kinh điển trong Toán học và là nền tảng cho các mô hình học máy sau này. Cụ thể hơn, ta có một tập dữ liệu quan sát được và muốn tìm một hàm số mô tả mối quan hệ giữa đầu vào và đầu ra. Bài toán này được gọi chung là **data fitting** (khớp dữ liệu).

Định nghĩa 1.1: Bài toán Data Fitting tổng quát

Cho tập dữ liệu $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ với $\mathbf{x}_i \in \mathbb{R}^p$, $y_i \in \mathbb{R}$. Bài toán data fitting là tìm hàm $f: \mathbb{R}^p \rightarrow \mathbb{R}$ trong một Class hàm cho trước $\mathcal{F}$ sao cho f xấp xỉ tốt nhất ánh xạ từ $\mathbf{x}_i$ đến y_i theo một tiêu chí tối ưu:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \mathcal{L}(f; \mathcal{D}),$$

trong đó $\mathcal{L}$ là hàm mất mát (*loss function*) đo mức độ sai lệch giữa Prediction của mô hình và giá trị thực.

1.1.1.b Mô hình hồi quy tuyến tính

Mô hình hồi quy tuyến tính là nền tảng cốt lõi trong toán học thống kê, nó có lịch sử phát triển gắn với bài toán Data Fitting và phương pháp bình phương tối thiểu. Mục tiêu của mô

¹ Hệ mét là hệ thống đo lường thập phân được thống nhất rộng rãi trên quốc tế. Dù có nhiều biến thể của hệ thống số liệu nổi lên vào cuối thế kỷ XIX và đầu thế kỷ XX, thuật ngữ này thường được dùng như một từ đồng nghĩa là "SI" hoặc "Hệ thống đơn vị Quốc Tế" Applied Mathematics in Statistics

hình là có thể ước lượng các tham số chưa biết thông qua các dữ liệu thực nghiệm đã quan sát được [2]. Trong mô hình này, các quan sát được biểu diễn dưới dạng một **tổ hợp tuyến tính** của các biến đã biết và các **tham số ẩn**.

Giả sử ta có một chuỗi các quan sát y_i phụ thuộc vào một mô hình tuyến tính $\alpha + \beta x_i$ ($i = 1, \dots, n$). Bài toán đặt ra là làm sao ta có thể ước lượng các giá trị $\hat{\alpha}$ và $\hat{\beta}$ cho các tham số α và β để có thể phù hợp với bộ dữ liệu quan sát được một cách tốt nhất và ít chi phí nhất [2]. Tổng quát bài toán, thì phần lớn ta sẽ có thể viết mô hình dưới dạng $Y = M\beta + \theta$ và cụ thể hơn $\mathcal{F}$ là tập các hàm tuyến tính theo tham số [3]. Khi đó, giả thiết:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \varepsilon_i = \mathbf{x}_i^T \boldsymbol{\beta} + \varepsilon_i,$$

với $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^T \in \mathbb{R}^{p+1}$ là vector tham số cần ước lượng và ε_i là nhiễu ngẫu nhiên (sai số mô hình).

Đặt ma trận thiết kế (*design matrix*) $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$ trong đó hàng thứ i là $(1, x_{i1}, \dots, x_{ip})$ (cột đầu tiên toàn số 1 ứng với hệ số tự do β_0). Khi đó toàn bộ hệ có thể viết gọn:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \quad \mathbf{y} \in \mathbb{R}^n, \mathbf{X} \in \mathbb{R}^{n \times (p+1)}, \boldsymbol{\varepsilon} \in \mathbb{R}^n. \quad (1)$$

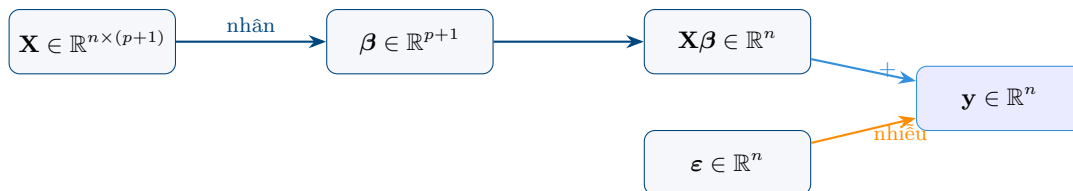


Figure 1: Cấu trúc mô hình hồi quy tuyến tính dạng ma trận: $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$. Dữ liệu quan sát $\mathbf{y}$ được phân tích thành thành phần *tín hiệu* $\mathbf{X}\boldsymbol{\beta}$ (xanh) và thành phần *nhiều* $\boldsymbol{\varepsilon}$ (cam).

1.1.2 Các Giả Thiết Gauss–Markov

Chứng minh gốc của Gauss (1809) dựa trên giả thiết sai số tuân theo phân phối chuẩn và dùng nguyên lý hợp lý cực đại (maximum likelihood). Về sau Gauss (1821, 1823) chứng minh lại tính tối ưu của OLS mà không cần giả thiết phân phối; nhà toán học Nga Andrei Markov (1900) hệ thống hóa và trình bày kết quả này tường minh trong giáo trình của ông. Đây là lý do định lý mang tên cả hai — **Gauss–Markov**. Để ước lượng OLS có các tính chất thống kê tốt, mô hình cần thỏa mãn các giả thiết sau:

Để ước lượng OLS có các tính chất thống kê tốt, mô hình tuyến tính (1) cần thỏa mãn năm

giả thiết Gauss–Markov sau [4, 5]. Các giả thiết này xác định khuôn khổ mà trong đó OLS được đảm bảo là ước lượng tốt nhất.

Định nghĩa 1.2: Các giả thiết Gauss–Markov (GM1–GM5)

1. **Tuyến tính:** $\mathbf{y} = \mathbf{X}\beta + \varepsilon$. Mô hình tuyến tính theo tham số β ; cho phép áp dụng đại số tuyến tính để giải bài toán tối ưu.
2. **Không hoàn hảo đa cộng tuyến:** $\text{rank}(\mathbf{X}) = p + 1$, tức các cột của $\mathbf{X}$ độc lập tuyến tính và $\mathbf{X}^T \mathbf{X}$ khả nghịch. Nếu vi phạm, nghiệm OLS không xác định duy nhất.
3. **Ngoại Tạo (zero conditional mean):** $\mathbb{E}[\varepsilon | \mathbf{X}] = \mathbf{0}$, tức $\mathbb{E}[\varepsilon_i | \mathbf{x}_i] = 0$ với mọi i . Đảm bảo không có biến quan trọng bị bỏ sót có tương quan với $\mathbf{X}$.
4. **Đồng phương sai (homoscedasticity):** $\text{Var}(\varepsilon | \mathbf{X}) = \sigma^2 \mathbf{I}_n$, tức $\text{Var}(\varepsilon_i) = \sigma^2$ và $\text{Cov}(\varepsilon_i, \varepsilon_j) = 0$ với $i \neq j$. Nếu vi phạm (heteroscedasticity), sai số chuẩn bị lệch.
5. **Phần dư chuẩn (normality):** $\varepsilon | \mathbf{X} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n)$. Điều kiện bổ sung — cần cho kiểm định t và F trong mẫu nhỏ; với mẫu lớn có thể nới lỏng nhờ định lý giới hạn trung tâm.

Giả thiết 1~4 đủ để đảm bảo OLS là **BLUE** (Best Linear Unbiased Estimator) theo Định lý Gauss–Markov (xem mục 1.1.6 bên dưới). Giả thiết 5 chỉ cần thiết khi thực hiện kiểm định giả thuyết và xây dựng khoảng tin cậy.

1.1.3 Hàm Mất Mát RSS và Chứng Minh Nghiệm OLS

Hàm mất mát Residual Sum of Squares được gọi là tổng bình phương các sai số, là mục tiêu cốt lõi khi các nhà toán học làm việc với bài toán Data Fitting, họ sẽ cố gắng **tối thiểu hóa** hàm mất mát này để có thể tạo ra mô hình tốt nhất cho tập dữ liệu mà họ có, điều này rất phổ biến trong phương pháp bình phương tối thiểu và hồi quy.

1.1.3.a Hàm mất mát RSS

Vào khoảng năm 1805, nhà toán học Andrien-Marie Legendre đã xây dựng một hệ phương trình tuyến tính cho các quan sát mà ông có được, trong đó mỗi phương trình của hệ có một sai số E . Hàm mất mát RSS được ông ký hiệu là S đại diện cho tổng bình phương của tất cả các sai số này với $S = E^2 + E'^2 + E''^2 + \dots$ [2]

Định nghĩa 1.3: Residual Sum of Squares (RSS)

Hàm mất mát của phương pháp Ordinary Least Squares (OLS) là tổng bình phương phần dư (*Residual Sum of Squares*):

$$\text{RSS}(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2 = \sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta)^2.$$

OLS tìm $\hat{\beta}$ tối thiểu hóa $\text{RSS}(\beta)$.

RSS đo tổng bình phương khoảng cách từ các điểm quan sát y_i đến các giá trị Prediction $\hat{y}_i = \mathbf{x}_i^T \beta$ của mô hình. OLS tìm hyperplane tốt nhất theo nghĩa cực tiểu hóa tổng khoảng cách vuông góc theo chiều y .

Với sự ra đời của RSS thì bài toán OLS đã trở nên rõ ràng hơn, nó đã cung cấp một công cụ mạnh mẽ để ước lượng các tham số của mô hình hồi quy tuyến tính, từ đó giúp các nhà nghiên cứu có thể hiểu rõ hơn về mối quan hệ giữa các biến và đưa ra Prediction chính xác hơn dựa trên dữ liệu quan sát được. (1) *Tạo ra sự cân bằng và triệt tiêu sai số cực đoan.* Điều này đã được Legendre lập luận và chứng minh rằng hàm bình phương mang lại một sự cân bằng lớn giữa các sai số. Nó đưa ra điểm phạt rất lớn cho các sai số lớn từ đó mà ngăn được các phần tử ngoại lai giúp ta tìm ra trạng thái gần thực tế nhất. (2) *Dễ dàng tính được bằng giải tích.* Khác với hàm giá trị tuyệt đối khác gặp khó khăn lớn trong việc tính toán thì RSS lại mang lại khả năng tính toán vô cùng thuận lợi. (3) *Nên tảng xác suất vững chắc.* RSS đã được Gauss chứng minh hoàn hảo thông qua thống kê, ông đã chứng minh rằng nếu các phần dư tuân theo quy luật phân phối chuẩn thì việc cực tiểu hóa hàm mất mát RSS hoàn toàn tương đương với **maximum likelihood** [2].

1.1.3.b Chứng minh nghiệm OLS (Normal Equations)

Xuyên suốt lịch sử phát triển, quá trình chứng minh phương pháp Bình Phương tối thiểu là một cuộc hành trình dài từ đầu thế kỷ 19, nó đã đi từ những lập luận đại số thuần túy cho tới nền tảng xác suất và cuối cùng chỉ thật sự được chứng minh tối ưu thông qua hình học trực giao và đại số tuyến tính ma trận hiện đại.

Vào những năm đầu tiên thuở sơ khai của phương pháp, nhà toán học người pháp Legendre đã đề xuất phương pháp OLS kèm theo một cách chứng minh bằng đại số thuần túy, không có yếu tố xác suất đi cùng. Để tìm được giá trị nhỏ nhất của S , ông lấy đạo hàm riêng của S theo từng ẩn số $(x, y, z, \dots)$ và cho chúng bằng 0. Nhờ đó quá trình này đã tạo ra một hệ phương trình tuyến tính mới có nghiệm duy nhất. Tuy nhiên, cách chứng minh này vẫn còn tồn đọng hạn chế là ông chỉ biện minh phương pháp này dựa trên lý luận thực nghiệm rằng bình phương sẽ phạt các sai số lớn, tạo ra 'sự cân bằng' giữa các quan sát [2] mà lại không đưa ra được độ chính xác hay sai số của các hệ số tìm được.

Sau đó vào năm 1809, Carl Friedrich Gauss đã đưa ra cách chứng minh của mình được xuất bản bên trong cuốn *Theoria Motus Corporum Coelestium*. Ông đã đưa OLS lên một nền tảng xác suất vững chắc hơn. Gauss sử dụng nguyên lý **Hợp lý cực đại** để chứng minh bài toán này. Ông giả sử các sai số đo đạc là các biến ngẫu nhiên độc lập, và từ đó mà đã tìm ra một quy luật phân phối sai số sao cho mà giá trị trung bình cộng chính là giá trị có xác suất xảy ra cao nhất. Điều này đã dẫn tới việc ông tìm ra hàm **Mật độ xác suất phân phối chuẩn**. Khi mà các sai số tuân theo phân phối chuẩn, việc cực đại hóa hàm hợp lý hoàn toàn tương đương về mặt toán học với việc **cực tiểu hóa tổng bình phương các sai số**.

Và sau Gauss đã có rất nhiều người tham gia vào chứng minh và sửa các hạn chế mà người đi trước mắc phải. Chẳng hạn như sau đó ông Laplace đã đề xuất cách chứng minh để khắc phục hạn chế của Gauss khi ông đang ép buộc sai số phải tuân theo phân phối chuẩn. Laplace đã áp dụng một phiên bản sơ khai của **Định lý giới hạn trung gian** chứng minh rằng khi số lượng quan sát là rất lớn, thì sự phân phối của tổng hoặc trung bình các sai số sẽ luôn hội tụ về phân phối chuẩn, bất kể phân phối gốc của từng sai số đơn lẻ là gì. Và từ đó mà ông đã chứng minh rằng OLS của Legendre và Gauss là phương pháp mang lại độ chính xác cao nhất cho các tập dữ liệu lớn. Sau này Gauss đã đề xuất cách chứng minh khi không biết quy luật phân phối của sai số ban đầu.

Cuối cùng theo thời gian thì hiện nay ta chứng minh OLS bằng hình học trực giao và đại số ma trận, đây chính là cách được xem là tối ưu, ngắn gọn và đẹp đẽ nhất. Khi nó kết hợp **Đại**

số tuyến tính và Phép chiếu trực giao, là cách tiếp cận bao trùm mọi vấn đề của OLS mà không dùng đến đạo hàm phức tạp.

Định lý 1.1: Nghiệm OLS — Normal Equations

Nếu $\mathbf{X}^T \mathbf{X}$ khả nghịch (tương đương $\text{rank}(\mathbf{X}) = p + 1$), bài toán cực tiểu hóa $\text{RSS}(\beta)$ có nghiệm duy nhất:

$$\hat{\beta}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

Chứng minh. Khai triển RSS dưới dạng tích vô hướng để áp dụng giải tích ma trận [6]:

$$\begin{aligned} \text{RSS}(\beta) &= (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) \\ &= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\beta - \beta^T \mathbf{X}^T \mathbf{y} + \beta^T \mathbf{X}^T \mathbf{X}\beta. \end{aligned}$$

Vì $\mathbf{y}^T \mathbf{X}\beta$ là một số thực (ma trận 1×1), nó bằng chuyển vị của chính nó: $\mathbf{y}^T \mathbf{X}\beta = (\mathbf{y}^T \mathbf{X}\beta)^T = \beta^T \mathbf{X}^T \mathbf{y}$. Do đó:

$$\text{RSS}(\beta) = \mathbf{y}^T \mathbf{y} - 2\beta^T \mathbf{X}^T \mathbf{y} + \beta^T \mathbf{X}^T \mathbf{X}\beta.$$

Bước 1: Lấy gradient theo β .

Áp dụng các công thức đạo hàm ma trận chuẩn: $\nabla_{\beta}(\mathbf{a}^T \beta) = \mathbf{a}$ và $\nabla_{\beta}(\beta^T \mathbf{M}\beta) = 2\mathbf{M}\beta$ (với $\mathbf{M}$ đối xứng — ở đây $\mathbf{M} = \mathbf{X}^T \mathbf{X}$ đối xứng):

$$\nabla_{\beta} \text{RSS} = \mathbf{0} - 2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X}\beta = -2\mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta).$$

Bước 2: Điều kiện cần tại cực trị.

Cho gradient bằng không:

$$\nabla_{\beta} \text{RSS} = \mathbf{0} \implies \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta) = \mathbf{0} \iff \mathbf{X}^T \mathbf{X}\beta = \mathbf{X}^T \mathbf{y}.$$

Đây là hệ phương trình **Normal Equations**.

Bước 3: Nghiệm duy nhất.

Vì $\mathbf{X}^T \mathbf{X}$ khả nghịch theo giả thiết:

$$\hat{\beta}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

Bước 4: Kiểm tra đây là cực tiểu (không phải cực đại hay điểm yên ngựa).

Ma trận Hessian của RSS là $\mathbf{H}_{\text{RSS}} = 2\mathbf{X}^T\mathbf{X}$. Vì $\text{rank}(\mathbf{X}) = p + 1$, với mọi $\mathbf{v} \neq \mathbf{0}$:

$$\mathbf{v}^T(\mathbf{X}^T\mathbf{X})\mathbf{v} = (\mathbf{X}\mathbf{v})^T(\mathbf{X}\mathbf{v}) = \|\mathbf{X}\mathbf{v}\|_2^2 > 0,$$

nên Hessian xác định dương. Vậy điểm tối hạn tìm được là **cực tiểu toàn cục** (và là duy nhất do hàm RSS lồi chặt). $\square$

Hệ quả 1.1: Tính chất của phần dư OLS

Phần dư OLS $\hat{\mathbf{e}} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}$ trực giao với không gian cột của $\mathbf{X}$:

$$\mathbf{X}^T\hat{\mathbf{e}} = \mathbf{0}.$$

Chứng minh. Từ Normal Equations: $\mathbf{X}^T\mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}^T\mathbf{y}$, suy ra $\mathbf{X}^T(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = \mathbf{0}$, tức là $\mathbf{X}^T\hat{\mathbf{e}} = \mathbf{0}$. $\square$

1.1.4 Ma Trận Chiếu (Hat Matrix)

1.1.4.a Định nghĩa và hình học

Định nghĩa 1.4: Hat Matrix (Ma trận chiếu)

Ma trận chiếu (hay hat matrix) là:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T \in \mathbb{R}^{n \times n}.$$

Tên gọi “hat” xuất phát từ việc $\mathbf{H}$ biến $\mathbf{y}$ thành $\hat{\mathbf{y}}$ (“đội mũ” lên y): $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{H}\mathbf{y}$.

Về mặt hình học, $\mathbf{H}$ thực hiện phép chiếu trực giao của $\mathbf{y}$ lên không gian cột của $\mathbf{X}$, ký hiệu $\mathcal{C}(\mathbf{X})$ [7]. Ma trận bù $\mathbf{I} - \mathbf{H}$ chiếu $\mathbf{y}$ lên không gian trực giao $\mathcal{C}(\mathbf{X})^\perp$, cho ra phần dư $\hat{\mathbf{e}}$.

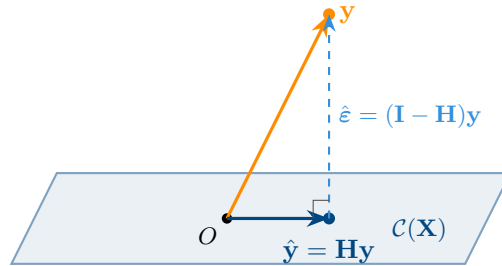


Figure 2: Hình học của Hat Matrix: $\mathbf{H}$ chiếu trực giao $\mathbf{y}$ lên không gian cột $\mathcal{C}(\mathbf{X})$, cho ra giá trị khớp $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$. Phần dư $\hat{\mathbf{e}} = (\mathbf{I} - \mathbf{H})\mathbf{y}$ vuông góc với $\mathcal{C}(\mathbf{X})$.

1.1.4.b Các tính chất của Hat Matrix

Mệnh đề 1.1: Tính chất của H

[8]

- (i) **Idempotent:** $\mathbf{H}^2 = \mathbf{H}$.
- (ii) **Đối xứng:** $\mathbf{H}^T = \mathbf{H}$.
- (iii) **Giá trị riêng:** Chỉ nhận giá trị 0 hoặc 1.
- (iv) **Hạng:** $\text{rank}(\mathbf{H}) = p + 1$.
- (v) **Giá trị khớp và phần dư:** $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$; $\hat{\mathbf{e}} = (\mathbf{I} - \mathbf{H})\mathbf{y}$.

Chứng minh. (i) **Idempotent:**

$$\begin{aligned}\mathbf{H}^2 &= [\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T][\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T] \\ &= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1} \underbrace{(\mathbf{X}^T\mathbf{X})(\mathbf{X}^T\mathbf{X})^{-1}}_{=\mathbf{I}_{p+1}} \mathbf{X}^T \\ &= \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T = \mathbf{H}.\end{aligned}$$

(ii) **Đối xứng:**

$$\mathbf{H}^T = [\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T]^T = \mathbf{X}[(\mathbf{X}^T\mathbf{X})^{-1}]^T\mathbf{X}^T.$$

Vì $\mathbf{X}^T\mathbf{X}$ đối xứng, nghịch đảo của nó cũng đối xứng: $[(\mathbf{X}^T\mathbf{X})^{-1}]^T = (\mathbf{X}^T\mathbf{X})^{-1}$. Do đó $\mathbf{H}^T = \mathbf{H}$.

(iii) **Giá trị riêng:** Với mọi giá trị riêng λ và vector riêng $\mathbf{v}$: $\mathbf{H}\mathbf{v} = \lambda\mathbf{v}$. Nhân hai vế bởi $\mathbf{H}$: $\mathbf{H}^2\mathbf{v} = \lambda\mathbf{H}\mathbf{v}$, tức $\mathbf{H}\mathbf{v} = \lambda^2\mathbf{v}$. Suy ra $\lambda\mathbf{v} = \lambda^2\mathbf{v}$, nên $\lambda(\lambda - 1) = 0$, tức $\lambda \in \{0, 1\}$.

(iv) **Hạng:** $\text{rank}(\mathbf{H}) = \text{tr}(\mathbf{H})$ (vì giá trị riêng chỉ là 0 hoặc 1). Ta có: $\text{tr}(\mathbf{H}) = \text{tr}(\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T) =$

$$\text{tr}((\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X}) = \text{tr}(\mathbf{I}_{p+1}) = p + 1.$$

(v) **Giá trị khớp và phần dư:** Theo định nghĩa, $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{H}\mathbf{y}$. Phần dư: $\hat{\boldsymbol{\varepsilon}} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\mathbf{y}$. $\square$

1.1.5 Ước Lượng Phương Sai Nhiều σ^2

Định lý 1.2: Ước lượng không Bias của σ^2

Dưới giả thiết Gauss–Markov (đặc biệt $\mathbb{E}[\boldsymbol{\varepsilon}|\mathbf{X}] = \mathbf{0}$ và $\text{Var}(\boldsymbol{\varepsilon}|\mathbf{X}) = \sigma^2 \mathbf{I}_n$), ước lượng:

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n - p - 1} = \frac{\|\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|_2^2}{n - p - 1}$$

là ước lượng không Bias của σ^2 , tức $\mathbb{E}[\hat{\sigma}^2|\mathbf{X}] = \sigma^2$.

Chứng minh. Viết $\text{RSS} = \hat{\boldsymbol{\varepsilon}}^T \hat{\boldsymbol{\varepsilon}}$ với $\hat{\boldsymbol{\varepsilon}} = (\mathbf{I} - \mathbf{H})\mathbf{y} = (\mathbf{I} - \mathbf{H})\boldsymbol{\varepsilon}$ (vì $(\mathbf{I} - \mathbf{H})\mathbf{X}\boldsymbol{\beta} = \mathbf{0}$).

Lấy kỳ vọng:

$$\begin{aligned}\mathbb{E}[\text{RSS}|\mathbf{X}] &= \mathbb{E}[\boldsymbol{\varepsilon}^T (\mathbf{I} - \mathbf{H}) \boldsymbol{\varepsilon} | \mathbf{X}] = \text{tr}((\mathbf{I} - \mathbf{H}) \text{Var}(\boldsymbol{\varepsilon}|\mathbf{X})) \\ &= \text{tr}((\mathbf{I} - \mathbf{H}) \sigma^2 \mathbf{I}_n) = \sigma^2 \text{tr}(\mathbf{I} - \mathbf{H}) \\ &= \sigma^2 (n - \text{tr}(\mathbf{H})) = \sigma^2 (n - p - 1).\end{aligned}$$

Bước thứ nhất dùng công thức $\mathbb{E}[\mathbf{v}^T \mathbf{A} \mathbf{v}] = \text{tr}(\mathbf{A} \boldsymbol{\Sigma})$ với $\boldsymbol{\Sigma} = \text{Var}(\mathbf{v})$. Suy ra $\mathbb{E}[\hat{\sigma}^2|\mathbf{X}] = \sigma^2$. $\square$

Mẫu số $n - p - 1$ là số bậc tự do còn lại sau khi ước lượng $p + 1$ tham số từ n quan sát. Nếu dùng n ở mẫu, ước lượng sẽ bị Bias về phía nhỏ hơn (biased downward).

1.1.6 Định Lý Gauss–Markov (BLUE)

Định lý 1.3: Gauss–Markov (BLUE)

[4] Dưới các giả thiết **GM1–GM4**, ước lượng OLS $\hat{\boldsymbol{\beta}}_{\text{OLS}}$ là ước lượng tuyến tính không Bias tốt nhất (*Best Linear Unbiased Estimator* — **BLUE**):

(i) **Không Bias:** $\mathbb{E}[\hat{\boldsymbol{\beta}}_{\text{OLS}} | \mathbf{X}] = \boldsymbol{\beta}$.

- (ii) **Ma trận phương sai:** $\text{Var}(\hat{\beta}_{\text{OLS}} | \mathbf{X}) = \sigma^2(\mathbf{X}^T \mathbf{X})^{-1}$.
- (iii) **Tốt nhất (phương sai nhỏ nhất):** Với mọi ước lượng tuyến tính không Bias $\tilde{\beta}$ khác, $\text{Var}(\tilde{\beta}_j) \geq \text{Var}(\hat{\beta}_j^{\text{OLS}})$ với mọi j .

Chứng minh. (i) **Tính không Bias.**

Từ Normal Equations: $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. Thay $\mathbf{y} = \mathbf{X}\beta + \varepsilon$:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{X}\beta + \varepsilon) = \beta + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \varepsilon.$$

Lấy kỳ vọng có điều kiện và áp dụng GM3:

$$\mathbb{E}[\hat{\beta} | \mathbf{X}] = \beta + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \underbrace{\mathbb{E}[\varepsilon | \mathbf{X}]}_{= \mathbf{0} \text{ (GM3)}} = \beta.$$

(ii) **Ma trận phương sai.**

Đặt $\mathbf{C} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$, khi đó $\hat{\beta} - \beta = \mathbf{C}\varepsilon$. Áp dụng GM4:

$$\text{Var}(\hat{\beta} | \mathbf{X}) = \mathbf{C} \underbrace{\text{Var}(\varepsilon | \mathbf{X})}_{= \sigma^2 \mathbf{I}_n \text{ (GM4)}} \mathbf{C}^T = \sigma^2 \mathbf{C} \mathbf{C}^T = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}.$$

(iii) **Tính tốt nhất.**

Với mọi ước lượng tuyến tính không Bias $\tilde{\beta} = \mathbf{A}\mathbf{y}$ (với $\mathbf{A}$ là ma trận hằng số), đặt $\mathbf{D} = \mathbf{A} - \mathbf{C}$. Tính không Bias đòi hỏi $\mathbf{D}\mathbf{X} = \mathbf{0}$, suy ra $\mathbf{C}\mathbf{D}^T = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{D}^T = \mathbf{0}^T = \mathbf{0}$. Do đó:

$$\begin{aligned} \text{Var}(\tilde{\beta} | \mathbf{X}) &= \sigma^2 \mathbf{A} \mathbf{A}^T = \sigma^2 (\mathbf{C} + \mathbf{D})(\mathbf{C} + \mathbf{D})^T \\ &= \sigma^2 (\mathbf{C} \mathbf{C}^T + \mathbf{C} \mathbf{D}^T + \mathbf{D} \mathbf{C}^T + \mathbf{D} \mathbf{D}^T) \\ &= \underbrace{\sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}}_{\text{Var}(\hat{\beta})} + \underbrace{\sigma^2 \mathbf{D} \mathbf{D}^T}_{\succeq \mathbf{0}}. \end{aligned}$$

Vì $\mathbf{D} \mathbf{D}^T$ nửa xác định dương, $\text{Var}(\tilde{\beta}) - \text{Var}(\hat{\beta}) \succeq \mathbf{0}$, tức $\text{Var}(\tilde{\beta}_j) \geq \text{Var}(\hat{\beta}_j^{\text{OLS}})$ với mọi j . $\square$

1.1.7 Đánh Giá Mô Hình

1.1.7.a Phân rã phương sai và hệ số xác định

Việc đánh giá mô hình bắt đầu từ việc phân tích nguồn gốc biến động của biến mục tiêu y .

Định nghĩa 1.5: Phân rã tổng bình phương

Khi mô hình có hệ số chặn β_0 , tổng bình phương toàn phần (TSS) phân rã thành:

$$\underbrace{\sum_{i=1}^n (y_i - \bar{y})^2}_{\text{TSS}} = \underbrace{\sum_{i=1}^n (\hat{y}_i - \bar{y})^2}_{\text{ESS}} + \underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{RSS}},$$

trong đó $\bar{y} = \frac{1}{n} \sum_i y_i$. **TSS** là biến động toàn phần chưa có mô hình; **ESS** là phần mô hình giải thích được; **RSS** là phần chưa giải thích.

Định nghĩa 1.6: Hệ số xác định R^2 và $\bar{R}^2$

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} = \frac{\text{ESS}}{\text{TSS}} \in [0, 1].$$

Vì R^2 luôn tăng (hoặc giữ nguyên) khi thêm bất kỳ biến nào, ta dùng $\bar{R}^2$ **hiệu chỉnh** để phạt việc thêm biến không có ý nghĩa:

$$\bar{R}^2 = 1 - \frac{\text{RSS}/(n-p-1)}{\text{TSS}/(n-1)} = 1 - \frac{n-1}{n-p-1}(1-R^2).$$

$\bar{R}^2$ chỉ tăng khi biến mới cải thiện mô hình đủ bù đắp bậc tự do bị mất; $\bar{R}^2$ có thể âm khi mô hình kém hơn mô hình rỗng.

1.1.7.b Kiểm định giả thuyết

Dưới giả thiết GM5, $\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(\mathbf{X}^T \mathbf{X})^{-1})$.

Định lý 1.4: Kiểm định t cho từng hệ số

Để kiểm định $H_0 : \beta_j = 0$ so với $H_1 : \beta_j \neq 0$, sử dụng thống kê:

$$t_j = \frac{\hat{\beta}_j}{\hat{\sigma} \sqrt{[(\mathbf{X}^T \mathbf{X})^{-1}]_{jj}}} \sim t_{n-p-1} \text{ dưới } H_0.$$

Sai số chuẩn của $\hat{\beta}_j$ là $\text{se}(\hat{\beta}_j) = \hat{\sigma} \sqrt{[(\mathbf{X}^T \mathbf{X})^{-1}]_{jj}}$. Khoảng tin cậy $(1 - \alpha)$ cho β_j :

$$\hat{\beta}_j \pm t_{\alpha/2, n-p-1} \cdot \text{se}(\hat{\beta}_j).$$

Định lý 1.5: Kiểm định F cho toàn mô hình

Để kiểm định $H_0 : \beta_1 = \dots = \beta_p = 0$ (không biến nào có tác động), sử dụng:

$$F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n-p-1)} = \frac{R^2/p}{(1-R^2)/(n-p-1)} \sim F_{p, n-p-1} \text{ dưới } H_0.$$

Bác bỏ H_0 khi $p\text{-value} = P(F_{p, n-p-1} \geq F_{\text{obs}}) < \alpha$.

Kiểm định t đánh giá từng biến *riêng lẻ* có điều kiện trên các biến khác; kiểm định F đánh giá *toàn bộ* mô hình cùng lúc. Khi các biến có tương quan cao, t -test của từng biến có thể yếu nhưng F -test vẫn có thể mạnh — hai kiểm định bổ sung cho nhau và cần được xem xét cùng nhau.

Table 1: Tóm tắt các chỉ số đánh giá mô hình hồi quy

Chỉ số	Công dụng chính	Ngưỡng kỳ vọng
R^2	Tỷ lệ phương sai được giải thích.	Càng gần 1 càng tốt
$\bar{R}^2$	Lựa chọn số biến tối ưu (phạt overfitting).	Tối đa hóa
t_j -test	Ý nghĩa thống kê từng hệ số β_j .	$ t_j > t_{\alpha/2, n-p-1}$
F -test	Ý nghĩa tổng thể của mô hình.	$p\text{-value} < \alpha$
$\hat{\sigma}$	Độ chính xác tuyệt đối của Prediction.	Càng nhỏ càng tốt

1.1.7.c Chẩn đoán phần dư

Bốn đồ thị chẩn đoán trong Hình 3 được Tạo trực tiếp bởi hàm `residual_plots`. Residuals vs Fitted và Scale-Location dùng để phát hiện dạng hàm sai hoặc phương sai thay đổi; Q-Q plot đánh giá giả thiết chuẩn cần cho suy luận mẫu nhỏ; đồ thị leverage giúp phát hiện quan sát có ảnh hưởng lớn. Trên dữ liệu mô phỏng, phần dư phân bố không có xu hướng rõ rệt quanh 0 và Q-Q plot bám tương đối sát đường chuẩn.

Bốn đồ thị chẩn đoán phần dư của mô hình OLS

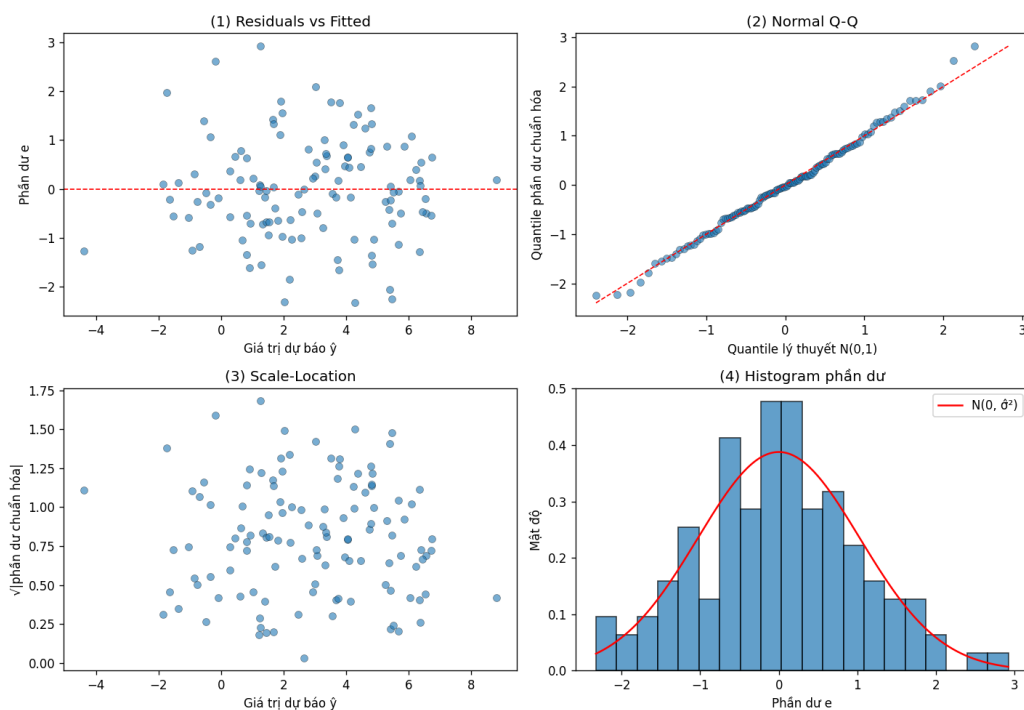


Figure 3: Bốn đồ thị chẩn đoán phần dư được tạo từ implementation Part 1.

1.1.8 Các Vấn Đề Nâng Cao trong Data Fitting

Phương pháp OLS, mặc dù là một công cụ mạnh mẽ với các tính chất thống kê tuyệt vời dưới các giả thiết Gauss–Markov, lại gặp phải những hạn chế đáng kể khi áp dụng vào các bài toán thực tế. Các vấn đề này nảy Tạo từ những hiện tượng như sự tương quan mạnh giữa các biến (đa cộng tuyến), sự quá khớp dữ liệu, hay khi số lượng biến vượt quá số lượng quan sát. Để vượt qua những thách thức này, các phương pháp hiện đại đã kế thừa và phát triển từ OLS qua kỹ thuật chính quy hóa (regularization), điều này đánh dấu một bước tiến quan trọng trong lịch

sử của phương pháp ước lượng tham số.

1.1.8.a Đa cộng tuyến và Tác động của nó

Trong thực tế, khi xử lý dữ liệu từ nhiều nguồn khác nhau, ta thường phát hiện rằng các biến độc lập không thực sự độc lập tuyến tính với nhau. Hiện tượng này, được gọi là **đa cộng tuyến**, xảy ra khi hai hay nhiều biến độc lập có mối liên hệ tuyến tính chặt chẽ, nghĩa là một biến có thể được Prediction với độ chính xác cao qua các biến khác. Đây là một trong những vấn đề cốt lõi mà các nhà thống kê phải đối mặt [9], vì nó vi phạm trực tiếp giả thiết GM2 về tính độc lập tuyến tính của các cột ma trận thiết kế.

Định nghĩa 1.7: Đa cộng tuyến (Multicollinearity)

Đa cộng tuyến xảy ra khi ma trận thiết kế $\mathbf{X}$ có các cột gần như phụ thuộc tuyến tính, tức là $\text{rank}(\mathbf{X}) < p + 1$ (hoặc tiến gần đến giới hạn này). Khi đó, không tồn tại nghịch đảo của $\mathbf{X}^T \mathbf{X}$, hoặc nếu tồn tại, nó sẽ có một số phần tử rất lớn, dẫn đến ước lượng OLS trở nên không ổn định về mặt số học.

Hậu quả của đa cộng tuyến đối với mô hình OLS là vô cùng nghiêm trọng, vì nó tác động đến mọi khía cạnh của ước lượng và suy diễn thống kê. Thứ nhất, khi các cột của $\mathbf{X}$ gần như phụ thuộc tuyến tính, định thức của ma trận $(\mathbf{X}^T \mathbf{X})$ tiến gần về 0, làm cho phép nghịch đảo trở nên cực kỳ thiếu ổn định. Thứ hai, các hệ số hồi quy ước lượng $\hat{\beta}$ sẽ có giá trị tuyệt đối rất lớn với phương sai cực cao, một hiện tượng gọi là “thổi phồng hệ số”. Điều này có nghĩa là những thay đổi rất nhỏ trong dữ liệu huấn luyện có thể dẫn đến những thay đổi hoàn toàn trong dấu và độ lớn của các hệ số, làm cho mô hình trở nên vô cùng nhạy cảm với nhiễu. Thứ ba, đa cộng tuyến phá hủy khả năng diễn giải của mô hình, vì ta không còn có thể phân tách tác động độc lập của từng biến đối với biến mục tiêu, do đó các kiểm định giả thuyết (như t-test) trở nên không đáng tin cậy.

Để phát hiện đa cộng tuyến, phương pháp phổ biến nhất là sử dụng **Hệ số phóng đại phương sai** (Variance Inflation Factor, VIF). Cho mỗi biến X_j , ta tính:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}, \quad (2)$$

trong đó R_j^2 là hệ số xác định khi hồi quy X_j theo tất cả các biến độc lập còn lại. Giá trị $VIF_j = 1$ chỉ rằng biến X_j không tương quan với các biến khác, trong khi $VIF_j > 10$ (tương ứng $R_j^2 > 0.9$) cảnh báo mức độ đa cộng tuyến nghiêm trọng cần xử lý ngay.

1.1.8.b Hồi quy được chuẩn hóa (Regularized Regression)

Để giải quyết bài toán đa cộng tuyến cũng như các vấn đề về quá khớp dữ liệu, các nhà thống kê đã phát triển một Class các phương pháp gọi chung là **chuẩn hóa** (regularization). Ý tưởng cốt lõi của chính quy hóa, mà các phương pháp hiện đại như Ridge, Lasso, Elastic Net đều tuân theo, là chấp nhận hy Tọa một chút độ chính xác đối với tập dữ liệu huấn luyện (tính không Bias của ước lượng) để đổi lấy sự ổn định lớn và khả năng tổng quát hóa tốt hơn. Điều này phản ánh một sự cân bằng cơ bản trong học máy giữa độ Bias (bias) và phương sai (variance), được gọi là *bias-variance tradeoff*.

Lịch sử phát triển của chính quy hóa bắt đầu từ những năm 1970, khi các nhà toán học và thống kê học nhận ra rằng OLS, mặc dù không Bias, lại có phương sai rất lớn khi đa cộng tuyến xảy ra. Phương pháp **Ridge Regression**, được đề xuất bởi Hoerl và Kennard [10], là một trong những phương pháp chính quy hóa sớm nhất và quan trọng nhất. Ý tưởng cơ bản là thêm vào hàm mất mát OLS một “khoản phạt” tỷ lệ thuận với chuẩn L_2 bình phương của vector hệ số, từ đó buộc các hệ số phải nhỏ hơn. Khoảng hai mươi năm sau, vào năm 1996, phương pháp **Lasso Regression**, được Tibshirani phát triển [11], đã mở ra một hướng mới bằng cách sử dụng chuẩn L_1 thay vì L_2 . Đặc biệt quan trọng là Lasso có khả năng thực hiện tự động lựa chọn biến (feature selection) bằng cách đặt một số hệ số chính xác bằng không.

Định nghĩa 1.8: Hồi quy Ridge (Ridge Regression)

Hồi quy Ridge giải bài toán tối ưu:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left\{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}, \quad (3)$$

trong đó $\lambda \geq 0$ là tham số chính quy hóa và hệ số chặn β_0 không bị phạt. Nghiệm dạng đóng là:

$$\hat{\beta}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{P})^{-1} \mathbf{X}^T \mathbf{y}, \quad (4)$$

trong đó $\mathbf{P} = \text{diag}(0, 1, \dots, 1)$ để không phạt hệ số chặn β_0 .

Sự thêm vào thành phần $\lambda \mathbf{P}$ vào ma trận $(\mathbf{X}^T \mathbf{X})$ là chìa khóa để giải quyết vấn đề đa cộng tuyến. Bằng cách này, ma trận $(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{P})$ luôn khả nghịch, ngay cả khi ma trận gốc $\mathbf{X}^T \mathbf{X}$ là suy biến hoặc gần suy biến. Hơn nữa, Ridge Regression có thể xử lý trường hợp $p > n$ (số biến vượt quá số quan sát), một tình huống mà OLS thông thường không thể làm được. Tuy nhiên, nhược điểm của Ridge là nó không loại bỏ biến nào hoàn toàn, vì các hệ số chỉ được co lại về 0 mà không bao giờ thực sự bằng 0 (trừ khi $\lambda \rightarrow \infty$).

Định nghĩa 1.9: Hồi quy Lasso (L_1 Regularization)

Hồi quy Lasso (Least Absolute Shrinkage and Selection Operator) giải bài toán:

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \left\{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1 \right\}, \quad (5)$$

trong đó $\|\beta\|_1 = \sum_{j=0}^p |\beta_j|$ là chuẩn L_1 . Không giống Ridge, Lasso không có nghiệm dạng đóng và phải được giải bằng các thuật toán tối ưu lặp (ví dụ như coordinate descent).

Ưu điểm sáng suốt nhất của Lasso so với Ridge là khả năng thực hiện **lựa chọn biến tự động** (automatic feature selection). Vì bản chất hình học của chuẩn L_1 , các hệ số của Lasso có xu hướng chính xác bằng 0 khi giá trị λ đủ lớn, điều này có nghĩa là một số biến được loại bỏ hoàn toàn khỏi mô hình. Đây là một lợi thế to lớn trong những ứng dụng nơi mà ta muốn tìm ra tập con nhỏ các biến thực sự quan trọng trong số một tập hợp lớn các ứng cử viên. Tuy nhiên, khi các biến có sự tương quan cao, Lasso có thể chọn một cách tùy ý giữa chúng, lựa chọn một biến này và loại bỏ biến kia. Để khắc phục hạn chế này, **Elastic Net** đã được phát triển bởi Zou và Hastie [12] như một sự kết hợp của Ridge và Lasso, sử dụng một kết hợp lồi của chuẩn L_1 và L_2 để đạt được những lợi ích của cả hai phương pháp.

Trong thực tế, việc lựa chọn giá trị tối ưu của tham số λ là một bước quan trọng và có ảnh hưởng lớn đến hiệu suất của mô hình. Phương pháp phổ biến nhất là sử dụng **kiểm chứng chéo** (cross-validation), nơi mà dữ liệu được chia thành các tập con, mô hình được huấn luyện trên một phần và đánh giá trên phần còn lại, quy trình này lặp đi lặp lại với

các giá trị λ khác nhau. Giá trị λ được chọn là giá trị làm cực tiểu hóa lỗi kiểm chứng chéo, từ đó đảm bảo rằng mô hình có khả năng tổng quát hóa tốt nhất trên dữ liệu chưa thấy.

1.2 Cài Đặt Python

1.2.1 Hàm `ols_fit(X, y)`

1.2.1.a Mô tả

Hàm `ols_fit` tính toán tường minh theo công thức Normal Equations: (1) tính $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$; (2) tính phần dư và RSS; (3) ước lượng $\hat{\sigma}^2 = \text{RSS}/(n - p - 1)$.

Hàm nghịch đảo `_mat_inv` sử dụng phương pháp Gauss-Jordan có chọn trục (partial pivoting) trên ma trận bổ sung $[\mathbf{A} \mid \mathbf{I}]$.

1.2.1.b Pseudo-code

Algorithm 1 Ordinary Least Squares (OLS) via Normal Equations

Require: $\mathbf{X} \in \mathbb{R}^{n \times p}$, $\mathbf{y} \in \mathbb{R}^n$

Ensure: $\beta, \hat{\sigma}^2, \hat{\mathbf{y}}, \mathbf{r}$

```
1:  $\mathbf{G} \leftarrow \mathbf{X}^T \mathbf{X}$ ,  $\mathbf{q} \leftarrow \mathbf{X}^T \mathbf{y}$                                 ▷ form normal equations
2: if  $\text{rank}(\mathbf{G}) < p$  then
3:   return FAIL (singular  $\mathbf{X}^T \mathbf{X}$ , multicollinearity detected)
4: end if
5:  $\beta \leftarrow \mathbf{G}^{-1} \mathbf{q}$                                             ▷ solve normal equations
6:  $\hat{\mathbf{y}} \leftarrow \mathbf{X} \beta$                                           ▷ fitted values
7:  $\mathbf{r} \leftarrow \mathbf{y} - \hat{\mathbf{y}}$                                           ▷ residuals
8:  $\text{RSS} \leftarrow \|\mathbf{r}\|_2^2$                                        ▷ residual sum of squares
9:  $d \leftarrow n - p - 1$                                            ▷ degrees of freedom:  $n$  trừ  $p + 1$  tham số ước lượng
10:  $\hat{\sigma}^2 \leftarrow \frac{\text{RSS}}{d}$                                      ▷ error variance estimate
11: return  $(\beta, \hat{\sigma}^2, \hat{\mathbf{y}}, \mathbf{r})$ 
```

1.2.1.c Hàm `hat_matrix(X)`

Hàm này tính toán ma trận hình chiếu (hat matrix) $\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$, là công cụ chẩn đoán quan trọng trong phân tích hồi quy. Nó dùng để phát hiện outliers thông qua leverage

points và kiểm tra các tính chất lý thuyết như tính lũy đẳng (idempotent) $\mathbf{H}^2 = \mathbf{H}$ và tính đối xứng.

Algorithm 2 Hat Matrix Computation and Property Verification

Require: $\mathbf{X} \in \mathbb{R}^{n \times p}$

Ensure: $\mathbf{H} \in \mathbb{R}^{n \times n}$, symmetry error, idempotent error, rank, trace

```
1:  $\mathbf{G}_{\text{inv}} \leftarrow (\mathbf{X}^T \mathbf{X})^{-1}$ 
2:  $\mathbf{H} \leftarrow \mathbf{X} \cdot \mathbf{G}_{\text{inv}} \cdot \mathbf{X}^T$ 
3:  $\text{sym\_error} \leftarrow \|\mathbf{H} - \mathbf{H}^T\|_{\infty}$  ▷ check symmetry
4:  $\text{idem\_error} \leftarrow \|\mathbf{H}^2 - \mathbf{H}\|_F$  ▷ check idempotency
5:  $\text{rank}(\mathbf{H}) \leftarrow p + 1$  ▷ expected rank = number of features incl. hệ số tự do
6:  $\text{trace}(\mathbf{H}) \leftarrow \text{sum of diagonal elements}$  ▷ should equal  $p + 1$ 
7: return ( $\mathbf{H}$ , sym_error, idem_error, rank, trace)
```

1.2.2 Chuẩn hóa Ridge Regression

Bên cạnh OLS thông thường, phần cài đặt Python cũng cung cấp phương pháp Ridge Regression để xử lý đa cộng tuyến. Hàm `ridge_fit(X, y, lambda)` giải bài toán tối ưu:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left\{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\},$$

với công thức nghiệm đóng $\hat{\beta}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{P})^{-1} \mathbf{X}^T \mathbf{y}$, trong đó $\mathbf{P} = \text{diag}(0, 1, \dots, 1)$ để không phạt hệ số tự do.

Algorithm 3 Ridge Regression

Require: $\mathbf{X} \in \mathbb{R}^{n \times p}$, $\mathbf{y} \in \mathbb{R}^n$, $\lambda \geq 0$

Ensure: β_{ridge} , RSS, penalty

```
1: if  $\lambda < 0$  then
2:   return FAIL (lambda must be non-negative)
3: end if
4:  $\mathbf{G} \leftarrow \mathbf{X}^T \mathbf{X}$ 
5:  $\mathbf{G}_{\text{ridge}} \leftarrow \mathbf{G} + \lambda \mathbf{P}$  ▷  $\mathbf{P} = \text{diag}(0, 1, \dots, 1)$ 
6:  $\mathbf{q} \leftarrow \mathbf{X}^T \mathbf{y}$ 
7:  $\beta_{\text{ridge}} \leftarrow \mathbf{G}_{\text{ridge}}^{-1} \mathbf{q}$ 
8:  $\hat{\mathbf{y}} \leftarrow \mathbf{X} \beta_{\text{ridge}}$ 
9:  $\text{RSS} \leftarrow \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$ 
10:  $\text{penalty} \leftarrow \lambda \|\beta_{\text{ridge}}\|_2^2$  ▷ exclude hệ số tự do
11:  $L_{\text{total}} \leftarrow \text{RSS} + \text{penalty}$ 
12: return ( $\beta_{\text{ridge}}$ , RSS, penalty,  $L_{\text{total}}$ )
```

1.2.2.a Generalized Cross-Validation cho Ridge

Để chọn siêu tham số λ cho Ridge, cách tự nhiên nhất là **kiểm chứng chéo bỏ một** (Leave-One-Out CV, LOO-CV): với mỗi quan sát thứ i , huấn luyện mô hình trên $n - 1$ điểm còn lại và tính lỗi Prediction tại y_i . Tuy nhiên, LOO-CV yêu cầu n lần refit mô hình — quá tốn kém khi ta cần duyệt qua một dải λ .

Đối với hồi quy tuyến tính, có một rút gọn nổi tiếng: lỗi LOO tại điểm i bằng $\hat{\varepsilon}_i / (1 - h_{ii})$ trong đó h_{ii} là phần tử đường chéo thứ i của hat matrix và $\hat{\varepsilon}_i$ là phần dư khi dùng toàn bộ n điểm. Từ đây, **Generalized Cross-Validation** (GCV) [13] thay thế h_{ii} bằng trung bình của chúng $\bar{h} = \text{tr}(\mathbf{H}_\lambda)/n$, cho phép tính toàn bộ GCV chỉ từ một lần fit:

Định nghĩa 1.10: Generalized Cross-Validation cho Ridge

$$\text{GCV}(\lambda) = \frac{\text{RSS}(\lambda) / n}{(1 - \text{tr}(\mathbf{H}_\lambda) / n)^2},$$

trong đó $\mathbf{H}_\lambda = \mathbf{X}(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{P})^{-1} \mathbf{X}^T$ là hat matrix của Ridge ($\mathbf{P} = \text{diag}(0, 1, \dots, 1)$ để không phạt hệ số tự do).

Tính trace của hat matrix Ridge. Để tính $\text{tr}(\mathbf{H}_\lambda)$ mà không tạo ma trận $n \times n$, ta tách riêng đóng góp của hệ số tự do và của các feature. Đặt $\mathbf{Z}$ là ma trận các cột feature đã tâm hóa (bỏ cột hằng), và gọi $d_1^2, \dots, d_r^2$ là *trị riêng* của ma trận Gram $\mathbf{Z}^T \mathbf{Z}$ (đây cũng chính là bình phương của các singular values của $\mathbf{Z}$). Khi đó:

Mệnh đề 1.2: Trace của hat matrix Ridge

$$\text{tr}(\mathbf{H}_\lambda) = 1 + \sum_{i=1}^r \frac{d_i^2}{d_i^2 + \lambda}.$$

Số 1 đến từ hệ số tự do — vì hệ số tự do không bị phạt, nó có bậc tự do hiệu dụng bằng 1 không phụ thuộc λ .

Chứng minh. Vì $\mathbf{X}^T \mathbf{X} + \lambda \mathbf{P}$ có cấu trúc khối phân ly giữa hệ số tự do (không bị phạt) và các feature (bị phạt), ta có thể viết hat matrix theo phân tích trị riêng của $\mathbf{Z}^T \mathbf{Z}$. Gọi $\mathbf{Z}^T \mathbf{Z} = \mathbf{Q} \mathbf{D}^2 \mathbf{Q}^T$ là phân tích trị riêng, trong đó $\mathbf{D}^2 = \text{diag}(d_1^2, \dots, d_r^2)$. Phần hat matrix ứng với khối feature là:

$$\mathbf{H}_{\text{feature}} = \mathbf{Z}(\mathbf{Z}^T \mathbf{Z} + \lambda \mathbf{I})^{-1} \mathbf{Z}^T = \mathbf{Z} \mathbf{Q} (\mathbf{D}^2 + \lambda \mathbf{I})^{-1} \mathbf{Q}^T \mathbf{Z}^T.$$

Lấy trace và dùng tính chất cyclic:

$$\text{tr}(\mathbf{H}_{\text{feature}}) = \text{tr}(\mathbf{Q}^T \mathbf{Z}^T \mathbf{Z} \mathbf{Q} (\mathbf{D}^2 + \lambda \mathbf{I})^{-1}) = \text{tr}(\mathbf{D}^2 (\mathbf{D}^2 + \lambda \mathbf{I})^{-1}) = \sum_{i=1}^r \frac{d_i^2}{d_i^2 + \lambda}.$$

Cộng thêm 1 từ hệ số tự do ta được công thức cần chứng minh. $\square$

Tính trị riêng bằng phép quay Jacobi. Thay vì dùng `numpy.linalg.eigh`, các trị riêng d_i^2 của $\mathbf{Z}^T \mathbf{Z}$ được tính từ đầu qua **phương pháp Jacobi** [6]: lặp lại các phép quay phẳng (Jacobi rotations) để triệt tiêu dần các phần tử ngoài đường chéo của ma trận đối xứng cho đến khi chuẩn off-diagonal nhỏ hơn ngưỡng 10^{-10} . Mỗi phép quay chọn phần tử lớn nhất ngoài đường chéo (p, q) , tính góc quay $\theta = \frac{1}{2} \arctan\left(\frac{2A_{pq}}{A_{pp} - A_{qq}}\right)$, và áp ma trận quay trực giao $\mathbf{G}(p, q, \theta)$ để triệt tiêu A_{pq} và A_{qp} . Thuật toán đảm bảo chuẩn off-diagonal giảm đơn điệu sau mỗi bước và hội tụ đến dạng đường chéo chứa các trị riêng.

Khi $\lambda \rightarrow 0$, Ridge tiến về OLS: $\text{tr}(\mathbf{H}_\lambda) \rightarrow p + 1$ (số bậc tự do đầy đủ), mẫu số tiến về $(1 - (p+1)/n)^2 > 1$ và GCV thấp — mô hình có bias nhỏ nhưng phương sai lớn. Khi λ tăng, các hệ số bị co lại về 0, $\text{tr}(\mathbf{H}_\lambda) \rightarrow 1$ (chỉ còn hệ số tự do), mẫu số tiến về $(1 - 1/n)^2 \approx 1$ — nhưng lúc này RSS tăng mạnh do bias lớn. Tại λ^* cực tiểu GCV, hai hiệu ứng cân bằng nhau, đây chính là điểm tối ưu bias–variance.

Trong thí nghiệm với dữ liệu mô phỏng, GCV giảm từ $\lambda = 0$ đến cực tiểu tại $\lambda^* \approx 0.838$ rồi tăng trở lại khi bias vượt quá phương sai, như minh họa ở Hình 4. Vị trí cực tiểu λ^* được đánh dấu bằng đường thẳng đứng màu đỏ; đây là giá trị được `ridge_trace` chọn tự động mà không cần refit mô hình theo từng fold.

Quỹ đạo hệ số và Generalized Cross-Validation

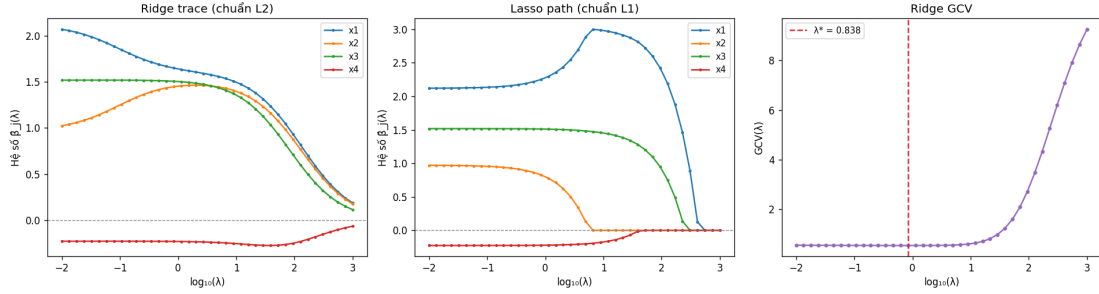


Figure 4: Ridge trace (trái) và Lasso path (giữa) biểu diễn hệ số theo λ ; đường GCV (phải) giảm đến $\lambda^* \approx 0.838$ (đường đỏ) rồi tăng, thể hiện điểm cân bằng bias-variance tối ưu.

1.2.3 Đánh giá Mô hình

Để đánh giá chất lượng mô hình, hàm `model_metrics(y, y_hat, p)` tính toán các chỉ số quan trọng như R^2 , $\bar{R}^2$, RSS, TSS, thống kê F , và RMSE.

Algorithm 4 Model Evaluation Metrics

Require: $\mathbf{y} \in \mathbb{R}^n$, $\hat{\mathbf{y}} \in \mathbb{R}^n$, p (number of features)

Ensure: R^2 , $\bar{R}^2$, RSS, TSS, F -statistic, RMSE

- | | |
|---|----------------------------------|
| 1: $\bar{y} \leftarrow \frac{1}{n} \sum_{i=1}^n y_i$ | ▷ sample mean |
| 2: $\text{RSS} \leftarrow \sum_{i=1}^n (y_i - \hat{y}_i)^2$ | ▷ residual sum of squares |
| 3: $\text{TSS} \leftarrow \sum_{i=1}^n (y_i - \bar{y})^2$ | ▷ total sum of squares |
| 4: $\text{ESS} \leftarrow \text{TSS} - \text{RSS}$ | ▷ explained sum of squares |
| 5: $R^2 \leftarrow 1 - \frac{\text{RSS}}{\text{TSS}}$ | ▷ coefficient of determination |
| 6: $\bar{R}^2 \leftarrow 1 - \frac{\text{RSS}/(n-p-1)}{\text{TSS}/(n-1)}$ | ▷ adjusted R^2 |
| 7: $F \leftarrow \frac{\text{ESS}/p}{\text{RSS}/(n-p-1)}$ | ▷ F -statistic for overall fit |
| 8: $\text{RMSE} \leftarrow \sqrt{\frac{\text{RSS}}{n}}$ | ▷ root mean squared error |
| 9: return (R^2 , $\bar{R}^2$, RSS, TSS, F , RMSE) | |
-

1.2.4 Hồi Quy Lasso và Thuật Toán Coordinate Descent

1.2.4.a Bài Toán Tối Ưu và Lý Do Không Tồn Tại Nghiệm Dạng Đóng

Trong khi Ridge Regression cho phép giải trực tiếp hệ tuyến tính $(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})\hat{\boldsymbol{\beta}} = \mathbf{X}^T \mathbf{y}$ nhờ hàm mục tiêu khả vi toàn cục, Lasso Regression đặt ra một thách thức tính toán hoàn toàn khác biệt do bản chất không trơn của chuẩn L_1 [11]. Cụ thể, hàm mục tiêu mà ta tối thiểu hóa là:

$$f(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}_{-0}\|_1, \quad (6)$$

trong đó $\|\beta_{-0}\|_1 = \sum_{j=1}^p |\beta_j|$ là chuẩn L_1 áp dụng trên tất cả hệ số hồi quy trừ hệ số chặn β_0 , còn $\lambda \geq 0$ là tham số chính quy hóa kiểm soát mức độ phạt. Theo quy ước chuẩn trong tài liệu [11, 14], hệ số chặn không bị phạt vì việc đưa β_0 vào số hạng phạt sẽ làm cho nghiệm phụ thuộc vào gốc tọa độ và đơn vị của biến đầu vào, điều này không mong muốn từ góc độ thống kê.

Hàm $|\beta_j|$ có đạo hàm theo nghĩa thông thường là $\text{sign}(\beta_j)$ khi $\beta_j \neq 0$, nhưng *không khả vi* tại $\beta_j = 0$ vì đạo hàm trái và phải không bằng nhau. Do đó, điều kiện tối ưu bậc nhất $\nabla f = \mathbf{0}$ không thể áp dụng cho (6), và bài toán Lasso không có nghiệm dạng đóng như Ridge hay OLS [11]. Thay vào đó, lý thuyết tối ưu lồi đòi hỏi sử dụng khái niệm **subdifferential**: subdifferential của $|\beta_j|$ là tập đơn điểm $\{\text{sign}(\beta_j)\}$ khi $\beta_j \neq 0$, và là đoạn $[-1, 1]$ khi $\beta_j = 0$, từ đó điều kiện tối ưu KKT (Karush–Kuhn–Tucker) trở thành điều kiện bao hàm thức $0 \in \partial f(\beta)$ thay vì phương trình đơn giản.

1.2.4.b Soft-Thresholding: Nghiệm Giải Tích của bài toán con

Thay vì tối thiểu hóa f theo toàn bộ β cùng lúc (bài toán p -chiều không tròn), ý tưởng cốt lõi của **Coordinate Descent** là lần lượt cố định tất cả β_k ($k \neq j$) và giải bài toán tối ưu theo từng β_j riêng lẻ. Điều đặc biệt quan trọng là bài toán 1D kết quả có *nghiệm giải tích chính xác*, có thể dẫn xuất tường minh từ điều kiện KKT.

Khi cố định tất cả β_k ($k \neq j$), ta định nghĩa **phần dư một phần** (partial residual) $\mathbf{r}^{(-j)} = \mathbf{y} - \sum_{k \neq j} \mathbf{X}_k \beta_k$, trong đó $\mathbf{X}_j$ là cột thứ j của $\mathbf{X}$. Hàm mục tiêu theo β_j khi đó rút gọn thành:

$$f_j(\beta_j) = z_j \beta_j^2 - 2\rho_j \beta_j + \text{const} + \lambda |\beta_j|, \quad j \geq 1, \quad (7)$$

trong đó $z_j = \|\mathbf{X}_j\|_2^2 = \sum_{i=1}^n X_{ij}^2$ là chuẩn bình phương cột j (một hằng số không phụ thuộc β_j), và:

$$\rho_j = \mathbf{X}_j^T \mathbf{r}^{(-j)} = \sum_{i=1}^n X_{ij} (r_i + X_{ij} \beta_j) \quad (8)$$

là hình chiếu của partial residual lên cột $\mathbf{X}_j$. Áp dụng điều kiện KKT cho bài toán (7): nghiệm

tối ưu $\hat{\beta}_j$ thỏa mãn $0 \in 2z_j\hat{\beta}_j - 2\rho_j + \lambda\partial|\hat{\beta}_j|$, từ đó giải theo từng trường hợp của subdifferential:

$$\begin{aligned}\hat{\beta}_j > 0 &\implies 2z_j\hat{\beta}_j - 2\rho_j + \lambda = 0 \implies \hat{\beta}_j = \frac{\rho_j - \lambda/2}{z_j}, \\ \hat{\beta}_j < 0 &\implies 2z_j\hat{\beta}_j - 2\rho_j - \lambda = 0 \implies \hat{\beta}_j = \frac{\rho_j + \lambda/2}{z_j}, \\ \hat{\beta}_j = 0 &\iff |\rho_j| \leq \lambda/2.\end{aligned}$$

Ba trường hợp này hợp nhất thành một công thức duy nhất [11, 15]:

$$\hat{\beta}_j = \frac{S\left(\rho_j, \frac{\lambda}{2}\right)}{z_j}, \quad (9)$$

trong đó $S(\rho, \lambda) = \text{sign}(\rho) \cdot \max(|\rho| - \lambda, 0)$ là **toán tử soft-thresholding**. Nhân tử $\lambda/2$ — chứ không phải λ — xuất hiện do đạo hàm của thành phần RSS Tạo ra hệ số 2: $\partial_{\beta_j} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 = -2\mathbf{X}_j^T \mathbf{r}^{(-j)} + 2z_j\beta_j$, và khi đặt điều kiện KKT rồi chia hai vế cho $2z_j$ ta thu được công thức (9) [15]. Để so sánh với scikit-learn, thư viện này tối thiểu $\frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \alpha \|\boldsymbol{\beta}\|_1$, tương đương với formulation (6) khi đặt $\alpha_{\text{sklearn}} = \lambda / (2n)$.

Toán tử soft-thresholding mang ý nghĩa hình học rõ ràng: nó tạo ra một *vùng chết* (dead zone) $[-\lambda, \lambda]$ quanh gốc tọa độ, trong đó bất kỳ ρ_j nào rơi vào vùng này đều cho $\hat{\beta}_j = 0$, nghĩa là biến X_j bị loại hoàn toàn khỏi mô hình. Đây chính là cơ chế lựa chọn biến tự động (*automatic feature selection*) của Lasso mà Ridge không có được: hàm β_j^2 khả vi tại gốc tọa độ và không tạo ra dead zone, nên nghiệm Ridge luôn khác không [11, 14]. Với hệ số chặn β_0 ($j = 0$), vì không có thành phần phạt, cập nhật trở về bài toán OLS 1D thông thường: $\hat{\beta}_0 = \rho_0/z_0$.

1.2.4.c Thuật Toán Coordinate Descent và Tính Hội Tụ

Thuật toán **Coordinate Descent** (CD) cho Lasso được Fu [16] đề xuất lần đầu vào năm 1998 dưới tên *shooting algorithm*. Lưu ý quan trọng về mặt toán học: hàm mục tiêu Lasso (6) là **lồi** (convex) nhưng **không** lồi chặt (not strictly convex) theo $\boldsymbol{\beta}$ vì chuẩn L_1 chỉ lồi, không lồi chặt. Tuy nhiên, bài toán con 1D — cố định mọi β_k ($k \neq j$) và tối thiểu theo β_j — lại lồi chặt vì thành phần bậc hai $z_j\beta_j^2$ với $z_j > 0$ chiếm ưu thế, đảm bảo mỗi cập nhật có nghiệm duy nhất là công thức soft-threshold (9). Tính hội tụ tổng quát cho bài toán không khả vi được Tseng [17] chứng minh trong khuôn khổ rộng hơn: bất kỳ điểm tích lũy nào của dãy lặp CD đều là điểm dừng (stationary point), và vì hàm mục tiêu Lasso **lồi** (dù không lồi chặt) nên mọi điểm dừng đồng thời là nghiệm tối ưu toàn cục.

Về hiệu năng tính toán, Friedman, Hastie và Tibshirani [15] tiến hành đánh giá thực nghiệm và kết luận rằng Coordinate Descent cạnh tranh tốt và thường vượt trội so với thuật toán LARS (Least Angle Regression and Shrinkage) của Efron et al. trong các bài toán quy mô lớn, đặc biệt khi cần tính toàn bộ regularization path. Kết quả này được Friedman et al. [18] làm rõ hơn: trên bài toán thưa với nhiều biến, tốc độ CD nhanh hơn LARS khoảng một bậc độ lớn (*one order of magnitude*) nhờ chi phí mỗi epoch chỉ là $O(np)$, thay vì phải duy trì và cập nhật một cơ sở vector tích cực như LARS. Lý do cốt lõi là kỹ thuật *cập nhật residual tăng dần*: sau mỗi lần thay đổi $\Delta\beta_j = \hat{\beta}_j - \beta_j^{\text{cũ}}$, ta cập nhật trực tiếp $\mathbf{r} \leftarrow \mathbf{r} - \mathbf{X}_j \Delta\beta_j$ thay vì tính lại $\mathbf{r} = \mathbf{y} - \mathbf{X}\boldsymbol{\beta}$ từ đầu, giữ chi phí mỗi cột ở mức $O(n)$ và tổng mỗi epoch là $O(np)$. Thêm vào đó, Friedman et al. [15] chỉ ra rằng CD đặc biệt hiệu quả trên bài toán thưa: khi $\hat{\beta}_j = 0$ sau cập nhật, tọa độ này có thể bỏ qua ở những epoch tiếp theo nếu residual chưa thay đổi đủ để $|\rho_j|$ vượt ngưỡng $\lambda/2$, giảm đáng kể số phép tính thực tế.

Tiêu chí hội tụ mà ta sử dụng là $\left\| \boldsymbol{\beta}^{(t)} - \boldsymbol{\beta}^{(t-1)} \right\|_{\infty} < \text{tol}$, tức là dừng khi thay đổi lớn nhất của bất kỳ hệ số nào qua một epoch nhỏ hơn ngưỡng 10^{-6} , nhất quán với cách kiểm tra hội tụ trong thư viện tham chiếu `glmnet` [15].

Algorithm 5 Lasso Regression via Coordinate Descent (`lasso_fit`)

Require: $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$, $\mathbf{y} \in \mathbb{R}^n$, $\lambda \geq 0$, `max_iter`, `tol`, $\beta^{(0)}$ (warm start, tùy chọn)
Ensure: $\hat{\beta}_{\text{lasso}}$, RSS, `lasso_penalty`, `total_loss`, `n_iter`, `converged`

```
1: if  $\lambda < 0$  then
2:   return FAIL ▷ lambda must be non-negative
3: end if
4:  $\beta \leftarrow \beta^{(0)}$  (hoặc  $\mathbf{0}$  nếu không có warm start)
5:  $\mathbf{r} \leftarrow \mathbf{y} - \mathbf{X}\beta$  ▷ residual ban đầu
6:  $z_j \leftarrow \|\mathbf{X}_j\|_2^2$  với mọi  $j = 0, \dots, p$  ▷ precompute: không đổi trong suốt quá trình
7: for  $t = 1, 2, \dots, \text{max\_iter}$  do
8:    $\beta^{\text{old}} \leftarrow \beta$ 
9:   for  $j = 0, 1, \dots, p$  do
10:     $\rho_j \leftarrow \sum_i X_{ij}(r_i + X_{ij}\beta_j)$  ▷ partial residual projection, xem (8)
11:    if  $z_j < 10^{-12}$  then
12:      continue ▷ cột hằng hoặc toàn 0: bỏ qua, tránh chia cho 0
13:    end if
14:    if  $j = 0$  then
15:       $\beta_j \leftarrow \rho_j / z_j$  ▷ hệ số tự do: cập nhật OLS, không penalty
16:    else
17:       $\beta_j \leftarrow S(\rho_j, \lambda/2) / z_j$  ▷ xem công thức (9)
18:    end if
19:     $\mathbf{r} \leftarrow \mathbf{r} - \mathbf{X}_j(\beta_j - \beta_j^{\text{old}})$  ▷ cập nhật residual tăng dần:  $O(n)$  mỗi cột
20:  end for
21:  if  $\|\beta - \beta^{\text{old}}\|_{\infty} < \text{tol}$  then
22:    break ▷ hội tụ
23:  end if
24: end for
25: RSS  $\leftarrow \|\mathbf{r}\|_2^2$ ; penalty  $\leftarrow \sum_{j=1}^p |\beta_j|$ ;  $L \leftarrow \text{RSS} + \lambda \cdot \text{penalty}$ 
26: return ( $\beta$ , RSS, penalty,  $L$ ,  $t$ , converged)
```

1.2.4.d Quỹ Đạo Chuẩn Hóa

Thay vì chỉ tính nghiệm tại một giá trị λ duy nhất, trong thực tế ta thường muốn khảo sát toàn bộ **quỹ đạo chuẩn hóa** (regularization path): đồ thị biểu diễn $\hat{\beta}(\lambda)$ khi λ biến thiên trên một dải giá trị từ rất lớn (mô hình thừa tối đa, gần như tất cả hệ số bằng không) đến rất nhỏ (mô hình gần OLS). Hàm `lasso_path` thực hiện điều này bằng cách gọi `lasso_fit` tuần tự trên dãy $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_K$ theo thứ tự giảm dần, đồng thời thu thập hệ số, RSS, total loss và số hệ số khác không tại mỗi λ_k .

Điểm then chốt về mặt tính toán là kỹ thuật **warm start**: nghiệm $\hat{\beta}(\lambda_k)$ được dùng làm điểm khởi đầu cho bài toán λ_{k+1} , thay vì khởi tạo lại từ vector không. Friedman, Hastie, Höfling và Tibshirani [18] đã chứng minh rằng warm start theo hướng λ giảm dần làm giảm đáng kể số epoch cần thiết tại mỗi λ , vì khi λ_{k+1} gần λ_k , điểm khởi đầu warm start đã rất gần nghiệm tối

ưu và coordinate descent chỉ cần vài epoch để hội tụ thay vì hàng chục epoch khi khởi đầu từ vector không [18]. Đây là hệ quả trực tiếp từ tính liên tục của quỹ đạo nghiệm $\hat{\beta}(\lambda)$ theo λ : khi λ thay đổi ít, nghiệm thay đổi ít, nên điểm warm start luôn nằm trong lân cận của nghiệm tối ưu.

Lý do sắp xếp λ theo chiều *giảm dần* (từ lớn đến nhỏ) thay vì ngược lại xuất phát từ cấu trúc bài toán: tại λ đủ lớn, toàn bộ hệ số hồi quy trừ hệ số tự do bị ép về không nên điểm khởi đầu là vector không đã là nghiệm tối ưu và CD hội tụ ngay từ epoch đầu [15]. Khi λ giảm dần, các hệ số lần lượt “giải phóng” khỏi vùng chết theo thứ tự quan trọng giảm dần — biến có tương quan cao nhất với phần dư xuất hiện trước — và warm start đảm bảo điểm khởi đầu luôn gần nghiệm, duy trì hiệu quả hội tụ trong suốt toàn bộ quỹ đạo. Phân tích trong [18] xác nhận rằng warm start kết hợp với dãy λ giảm dần giảm đáng kể tổng số vòng lặp so với khởi đầu lạnh (cold start) trên cùng dữ liệu.

Algorithm 6 Regularization Path for Lasso (`lasso_path`)

Require: $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$, $\mathbf{y} \in \mathbb{R}^n$, $\{\lambda_k\}_{k=1}^K$ (bất kỳ thứ tự)

Ensure: `LassoTraceData`: quỹ đạo hệ số, RSS, total loss, nonzero_counts, n_iter, converged

```
1: Sắp xếp  $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_K$  theo thứ tự giảm dần
2:  $\beta^{\text{warm}} \leftarrow \mathbf{0}$ 
3: for  $k = 1, 2, \dots, K$  do
4:   Gọi lasso_fit( $\mathbf{X}, \mathbf{y}, \lambda_k$ ;  $\beta^{(0)} = \beta^{\text{warm}}$ )
5:   Thu thập  $\hat{\beta}(\lambda_k)$ ,  $\text{RSS}_k$ ,  $L_k$ ,  $\text{nonzero}_k$ ,  $\text{n\_iter}_k$ 
6:   if hội tụ thành công then
7:      $\beta^{\text{warm}} \leftarrow \hat{\beta}(\lambda_k)$  ▷ warm start cho  $\lambda_{k+1}$ 
8:   end if
9: end for
10: return LassoTraceData( $\{\lambda_k\}$ ,  $\{\hat{\beta}(\lambda_k)\}$ ,  $\{\text{RSS}_k\}$ ,  $\{L_k\}$ ,  $\{\text{nonzero}_k\}$ ,  $\{\text{n\_iter}_k\}$ )
```

Điểm khác biệt cơ bản giữa `lasso_path` và `ridge_trace` là kỹ thuật warm start và sự phụ thuộc vào thứ tự duyệt λ . Với Ridge, vì `ridge_fit` giải một hệ tuyến tính độc lập tại mỗi λ (nghiệm không phụ thuộc điểm khởi đầu), thứ tự λ không ảnh hưởng đến kết quả và warm start không có ý nghĩa. Với Lasso, warm start theo chiều giảm của λ không chỉ giảm chi phí tính toán mà còn ổn định hơn về mặt số học vì nghiệm ít thay đổi giữa hai λ liên tiếp, tránh hiện tượng dao động số học khi CD phải tìm kiếm từ xa.

1.2.5 Suy diễn Thống kê cho Hệ số

Hàm `coef_inference(X, y, beta_hat, sigma2)` tính toán sai số chuẩn, thống kê t , giá trị p , và khoảng tin cậy cho mỗi hệ số hồi quy, cho phép kiểm định giả thuyết $H_0 : \beta_j = 0$.

Algorithm 7 Coefficient Inference: Standard Errors, t -Statistics, p -Values, CI

Require: $\mathbf{X} \in \mathbb{R}^{n \times p}$, $\mathbf{y} \in \mathbb{R}^n$, $\hat{\boldsymbol{\beta}}$, $\hat{\sigma}^2$, $\alpha = 0.05$

Ensure: SE, t -stats, p -values, CI lower/upper

```
1:  $\mathbf{G} \leftarrow \mathbf{X}^T \mathbf{X}$ 
2:  $\mathbf{G}^{-1} \leftarrow (\mathbf{X}^T \mathbf{X})^{-1}$  ▷ variance-covariance matrix proportional
3: for  $j = 0$  to  $p$  do
4:    $\text{SE}_j \leftarrow \hat{\sigma} \sqrt{(\mathbf{G}^{-1})_{jj}}$  ▷ standard error
5:    $t_j \leftarrow \frac{\hat{\beta}_j}{\text{SE}_j}$  ▷  $t$ -statistic
6:    $p_j \leftarrow 2 \cdot P(t_{n-p-1} \geq |t_j|)$  ▷ two-tailed  $p$ -value
7:    $t_{\alpha/2} \leftarrow t\text{-quantile}(1 - \alpha/2, n - p - 1)$ 
8:    $\text{CI}_{\text{lower},j} \leftarrow \hat{\beta}_j - t_{\alpha/2} \cdot \text{SE}_j$ 
9:    $\text{CI}_{\text{upper},j} \leftarrow \hat{\beta}_j + t_{\alpha/2} \cdot \text{SE}_j$ 
10: end for
11: return (SE,  $t$ -stats,  $p$ -values,  $\text{CI}_{\text{lower}}$ ,  $\text{CI}_{\text{upper}}$ )
```

1.2.6 Phát hiện Đa cộng tuyến qua VIF

Hàm `vif(X)` tính toán Hệ số Phóng đại Phương sai cho mỗi biến, công cụ tiêu chuẩn để phát hiện đa cộng tuyến. Giá trị $\text{VIF}_j > 10$ cảnh báo mức độ nghiêm trọng.

Algorithm 8 Variance Inflation Factor (VIF)

Require: $\mathbf{X} \in \mathbb{R}^{n \times p}$

Ensure: VIF_j for each column j

```
1: for  $j = 0$  to  $p - 1$  do
2:    $X_{-j} \leftarrow$  columns of  $\mathbf{X}$  except column  $j$ 
3:   Fit regression:  $X_j \sim X_{-j}$  to get  $R_j^2$ 
4:    $\text{VIF}_j \leftarrow \frac{1}{1 - R_j^2}$  ▷ variance inflation factor
5:   if  $\text{VIF}_j = 1$  then
6:     Column  $j$  has no correlation with others
7:   else if  $\text{VIF}_j > 10$  then
8:     WARNING: Serious multicollinearity detected
9:   end if
10: end for
11: return ( $\text{VIF}_0, \text{VIF}_1, \dots, \text{VIF}_{p-1}$ )
```

1.2.7 Kiểm chứng Implementation From Scratch

Toàn bộ thuật toán trong Part 1 — từ giải Normal Equations qua Gauss–Jordan, tính hat matrix, coordinate descent cho Lasso, phép quay Jacobi để tính trị riêng, đến hàm beta không đầy đủ chuẩn hóa $I_x(a, b)$ bằng chuỗi phân số liên tục Lentz — được cài đặt thuần túy bằng Python mà không gọi bất kỳ hàm tính toán số học nào từ NumPy, SciPy hay scikit-learn trong luồng tính toán chính.

Thay vào đó, ba thư viện này được gọi song song trong các khối `__main__` của mỗi module và trong các script kiểm chứng riêng biệt, để cung cấp giá trị tham chiếu tin cậy nhằm phát hiện sai lệch số học trong code.

Bảng 2 tổng hợp sai số cực đại giữa code và thư viện tương ứng, đo trên dữ liệu mô phỏng với seed cố định 42. Tất cả các đại lượng đều đạt sai số dưới ngưỡng 10^{-8} ; phần lớn nằm ở mức nhiễu số học máy $\approx 10^{-15}$ đến 10^{-16} , chứng tỏ implementation không tích lũy sai số số học so với các thư viện chuyên dụng được tối ưu hóa kỹ lưỡng.

Table 2: Kiểm chứng implementation scratch với NumPy/SciPy

Đại lượng kiểm chứng	Sai số cực đại	Kết quả
Hệ số OLS so với NumPy	2.22×10^{-16}	PASSED
Hat matrix so với NumPy	5.55×10^{-17}	PASSED
Standard error so với NumPy	5.55×10^{-17}	PASSED
t -statistic so với NumPy	7.11×10^{-15}	PASSED
p -value Student- t so với SciPy	2.11×10^{-13}	PASSED
p -value kiểm định F so với SciPy	9.10×10^{-15}	PASSED
Critical value Student- t so với SciPy	1.11×10^{-14}	PASSED
VIF so với NumPy	2.22×10^{-16}	PASSED
$\text{tr}(\mathbf{H}_{\text{ridge}})$ so với NumPy SVD	1.78×10^{-15}	PASSED

1.3 Định Lý Gauss–Markov và Kiểm Chứng Monte Carlo

1.3.1 Các Giả Thiết Gauss–Markov

1.3.1.a Bối cảnh

Xét về bản chất thuần túy đại số, công thức nghiệm $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ tối thiểu hóa tổng bình phương phần dư một cách cơ học bất kể cơ chế sinh dữ liệu thực sự là gì; bởi vậy nghiệm này luôn tồn tại và được xác định hoàn toàn từ $\mathbf{X}$ và $\mathbf{y}$ mà không cần bất kỳ giả thiết xác suất nào. Tuy nhiên, để có thể đánh giá **chất lượng thống kê** của ước lượng, tức xét xem $\hat{\beta}$ có thực sự gần β thật không, phân tán ra sao qua các mẫu lặp lại, và liệu có ước lượng tuyến tính nào khác hiệu quả hơn không, ta cần đặt ra một số giả thiết về quá trình sinh dữ liệu. Những giả thiết này, được gọi chung là **giả thiết Gauss–Markov**, xác định khuôn khổ lý thuyết để chứng minh OLS là ước lượng tốt nhất trong lớp các ước lượng tuyến tính không Bias.

Định nghĩa 1.11: Các giả thiết Gauss–Markov (GM1–GM5)

Xét mô hình hồi quy tuyến tính $\mathbf{y} = \mathbf{X}\beta + \varepsilon$ với $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$. Các giả thiết Gauss–Markov gồm:

GM1. Tuyến tính theo tham số: $\mathbf{y} = \mathbf{X}\beta + \varepsilon$.

GM2. Không đa cộng tuyến hoàn hảo: $\text{rank}(\mathbf{X}) = p + 1$ (các cột của $\mathbf{X}$ độc lập tuyến tính), bảo đảm $\mathbf{X}^T \mathbf{X}$ khả nghịch.

GM3. Ngoại Tạo nghiệm ngặt (kỳ vọng nhiều bằng 0): $\mathbb{E}[\varepsilon \mid \mathbf{X}] = \mathbf{0}$, tức $\mathbb{E}[\varepsilon_i \mid \mathbf{X}] = 0$ với mọi i .

GM4. Đồng phương sai và không tương quan (cầu phương sai): $\text{Var}(\varepsilon \mid \mathbf{X}) = \sigma^2 \mathbf{I}_n$, tức $\text{Var}(\varepsilon_i \mid \mathbf{X}) = \sigma^2$ và $\text{Cov}(\varepsilon_i, \varepsilon_j \mid \mathbf{X}) = 0$ với $i \neq j$.

GM5. Nhiễu chuẩn (tùy chọn): $\varepsilon \mid \mathbf{X} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n)$.

Bốn giả thiết đầu **GM1–GM4** là đủ để ước lượng OLS trở thành BLUE: GM1–GM2 bảo đảm nghiệm OLS tồn tại và duy nhất, GM3 cho tính không Bias, còn GM4 cho tính tối ưu (phương sai nhỏ nhất). Đây chính là nội dung Định lý Gauss–Markov. Hai hệ quả trực tiếp của GM3–GM4, dùng lại trong chứng minh, là:

$$\mathbb{E}[\mathbf{y} \mid \mathbf{X}] = \mathbf{X}\beta, \quad \text{Var}(\mathbf{y} \mid \mathbf{X}) = \sigma^2 \mathbf{I}_n.$$

Giả thiết chuẩn **GM5** không cần cho Định lý Gauss–Markov mà là phần bổ sung: khi nhiều tuân theo phân phối chuẩn, ta có thêm $\hat{\beta}_{OLS} \sim \mathcal{N}(\beta, \sigma^2(\mathbf{X}^T \mathbf{X})^{-1})$, nhờ đó mới thực hiện được suy diễn chính xác ở mẫu hữu hạn (kiểm định t , F , khoảng tin cậy).

1.3.1.b Chẩn đoán trực quan giả thiết GM1

Giả thiết GM1 phát biểu rằng kỳ vọng có điều kiện $\mathbb{E}[y_i | \mathbf{x}_i] = \mathbf{x}_i^T \beta$ mô tả đúng cấu trúc của dữ liệu. Vì đây là một giả thiết về dạng hàm của quá trình sinh dữ liệu, về mặt phương pháp luận thống kê không có kiểm định nào có thể xác nhận tuyệt đối GM1 chỉ từ một mẫu hữu hạn; thay vào đó, điều ta có thể làm là quan sát xem phần dư có bộc lộ dấu hiệu phi tuyến còn sót lại hay không. Đồ thị **Residuals vs Fitted** là công cụ chẩn đoán trực quan chuẩn cho mục đích này: nếu phần dư tạo thành đường cong hoặc xu hướng có hệ thống theo chiều tăng của fitted values, điều đó gợi ý mô hình đang thiếu thành phần phi tuyến hoặc interaction quan trọng; ngược lại, khi các điểm phân tán đều quanh đường 0 mà không tạo hình dạng rõ rệt, đó là dấu hiệu tích cực cho thấy GM1 không bị vi phạm nghiêm trọng.

Hình 5 trình bày kết quả chẩn đoán trên dữ liệu mô phỏng tuyến tính, trong đó các điểm xanh là phần dư từng quan sát và đường đỏ nối trung bình phần dư trong từng dải fitted values.

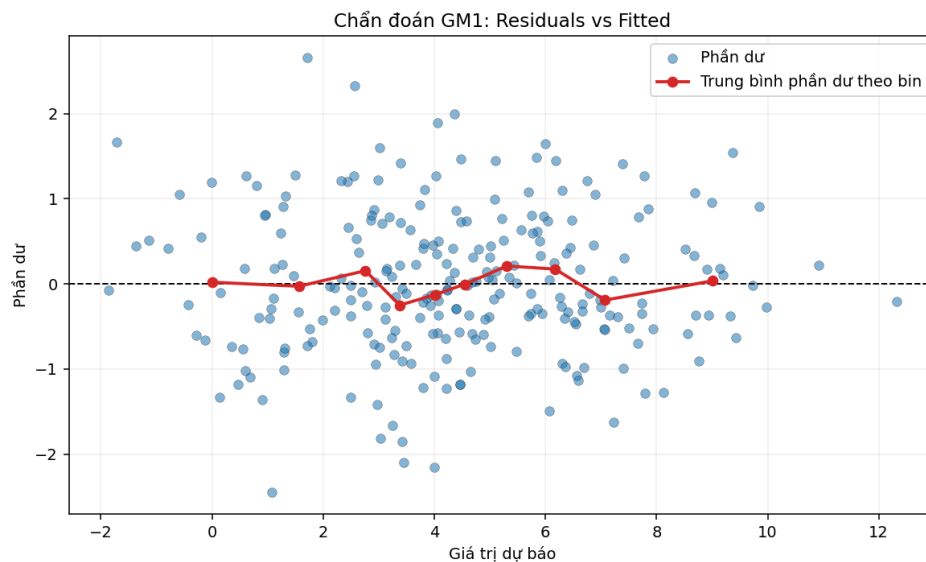


Figure 5: Chẩn đoán GM1 bằng Residuals vs Fitted và trung bình phần dư theo bin.

Nhận xét. Đường trung bình phần dư dao động đều quanh số 0 theo toàn dải fitted values mà không tạo thành dạng cong hay xu hướng có hệ thống nào, do đó kết luận chẩn đoán là **không**

thấy dấu hiệu phi tuyến rõ rệt trong mẫu được kiểm tra. Cần lưu ý rằng kết luận đúng đắn nên được giữ ở mức “không thấy bằng chứng vi phạm GM1” thay vì “GM1 đã được chứng minh thỏa mãn”, vì bất kỳ khẳng định tuyệt đối nào cũng vượt quá giới hạn của chẩn đoán dựa trên mẫu hữu hạn. Trong trường hợp đường đồ có dạng chữ U, chữ S hoặc xu hướng đơn điệu rõ rệt, mô hình cần được bổ sung biến phi tuyến, interaction hoặc điều chỉnh dạng hàm cho phù hợp hơn với cấu trúc thực của dữ liệu.

1.3.2 Định Lý Gauss–Markov (BLUE)

1.3.2.a Phát biểu

Định lý 1.6: Định lý Gauss–Markov

Dưới các giả thiết GM1–GM4, ước lượng OLS $\hat{\beta}_{OLS} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ là **ước lượng tuyến tính không Bias tốt nhất** (*Best Linear Unbiased Estimator* — BLUE):

- (i) **Tuyến tính:** $\hat{\beta}_{OLS}$ là một hàm tuyến tính của $\mathbf{y}$.
- (ii) **Không Bias:** $\mathbb{E}[\hat{\beta}_{OLS} | \mathbf{X}] = \beta$.
- (iii) **Tốt nhất:** Với mọi ước lượng tuyến tính không Bias khác $\tilde{\beta}$, ta có $\text{Var}(\tilde{\beta} | \mathbf{X}) \succeq \text{Var}(\hat{\beta}_{OLS} | \mathbf{X})$ theo nghĩa nửa xác định dương; đặc biệt $\text{Var}(\tilde{\beta}_j) \geq \text{Var}(\hat{\beta}_{j,OLS})$ với mọi j .

Hơn nữa, ma trận hiệp phương sai của ước lượng OLS là

$$\text{Var}(\hat{\beta}_{OLS} | \mathbf{X}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}. \quad (10)$$

1.3.2.b Chứng minh

Chứng minh. Đặt $\mathbf{C} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \in \mathbb{R}^{(p+1) \times n}$, khi đó $\hat{\beta}_{OLS} = \mathbf{C} \mathbf{y}$. Theo GM2, $\mathbf{X}^T \mathbf{X}$ khả nghịch nên $\mathbf{C}$ xác định tốt. Ta cũng ghi nhận đẳng thức then chốt

$$\mathbf{C} \mathbf{X} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X} = \mathbf{I}_{p+1}. \quad (11)$$

(i) **Tuyến tính.** Theo định nghĩa $\hat{\beta}_{OLS} = \mathbf{C} \mathbf{y}$ với $\mathbf{C}$ chỉ phụ thuộc $\mathbf{X}$, nên đây là một hàm tuyến tính của $\mathbf{y}$.

(ii) **Không Bias.** Lấy kỳ vọng có điều kiện theo $\mathbf{X}$, dùng GM1 ($\mathbf{y} = \mathbf{X}\beta + \varepsilon$) và GM3 ($\mathbb{E}[\varepsilon |$

$\mathbf{X}] = \mathbf{0})$:

$$\mathbb{E}[\hat{\beta}_{\text{OLS}} | \mathbf{X}] = \mathbf{C} \mathbb{E}[\mathbf{y} | \mathbf{X}] = \mathbf{C} \mathbf{X} \beta \stackrel{(11)}{=} \mathbf{I}_{p+1} \beta = \beta.$$

Ma trận hiệp phương sai (10). Viết $\hat{\beta}_{\text{OLS}} - \beta = \mathbf{C}\mathbf{y} - \beta = \mathbf{C}(\mathbf{X}\beta + \varepsilon) - \beta = \mathbf{C}\varepsilon$ (do (11)).

Dùng GM4 ($\text{Var}(\varepsilon | \mathbf{X}) = \sigma^2 \mathbf{I}_n$):

$$\begin{aligned} \text{Var}(\hat{\beta}_{\text{OLS}} | \mathbf{X}) &= \mathbb{E}[(\hat{\beta}_{\text{OLS}} - \beta)(\hat{\beta}_{\text{OLS}} - \beta)^T | \mathbf{X}] = \mathbf{C} \mathbb{E}[\varepsilon \varepsilon^T | \mathbf{X}] \mathbf{C}^T \\ &= \mathbf{C} (\sigma^2 \mathbf{I}_n) \mathbf{C}^T = \sigma^2 \mathbf{C} \mathbf{C}^T. \end{aligned}$$

Khai triển $\mathbf{C} \mathbf{C}^T$:

$$\mathbf{C} \mathbf{C}^T = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T [(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T]^T = (\mathbf{X}^T \mathbf{X})^{-1} \underbrace{\mathbf{X}^T \mathbf{X} (\mathbf{X}^T \mathbf{X})^{-1}}_{=\mathbf{I}} = (\mathbf{X}^T \mathbf{X})^{-1},$$

trong đó dùng tính đối xứng của $(\mathbf{X}^T \mathbf{X})^{-1}$. Vậy $\text{Var}(\hat{\beta}_{\text{OLS}} | \mathbf{X}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}$, chứng minh (10).

(iii) Tốt nhất (phương sai nhỏ nhất). Xét một ước lượng tuyến tính không Bias bất kỳ $\tilde{\beta} = \mathbf{D}\mathbf{y}$ với $\mathbf{D} \in \mathbb{R}^{(p+1) \times n}$ chỉ phụ thuộc $\mathbf{X}$. Đặt

$$\mathbf{A} := \mathbf{D} - \mathbf{C} \quad \Longleftrightarrow \quad \mathbf{D} = \mathbf{C} + \mathbf{A}.$$

Ràng buộc từ tính không Bias. Tương tự (ii), $\mathbb{E}[\tilde{\beta} | \mathbf{X}] = \mathbf{D}\mathbf{X}\beta$. Yêu cầu điều này bằng β với mọi β buộc $\mathbf{D}\mathbf{X} = \mathbf{I}_{p+1}$. Kết hợp (11):

$$\mathbf{D}\mathbf{X} = (\mathbf{C} + \mathbf{A})\mathbf{X} = \mathbf{C}\mathbf{X} + \mathbf{A}\mathbf{X} = \mathbf{I}_{p+1} + \mathbf{A}\mathbf{X} = \mathbf{I}_{p+1} \implies \boxed{\mathbf{A}\mathbf{X} = \mathbf{0}}.$$

Triệt tiêu chéo. Từ $\mathbf{A}\mathbf{X} = \mathbf{0}$, các tích chéo biến mất:

$$\mathbf{C}\mathbf{A}^T = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{A}^T = (\mathbf{X}^T \mathbf{X})^{-1} (\mathbf{A}\mathbf{X})^T = \mathbf{0}, \quad \mathbf{A}\mathbf{C}^T = (\mathbf{C}\mathbf{A}^T)^T = \mathbf{0}.$$

So sánh phương sai. Vì $\tilde{\beta} = \mathbf{D}\mathbf{y}$ cũng tuyến tính theo $\mathbf{y}$, lập luận như trên cho $\text{Var}(\tilde{\beta} | \mathbf{X}) =$

$\sigma^2 \mathbf{D} \mathbf{D}^T$. Do đó:

$$\begin{aligned}\text{Var}(\tilde{\boldsymbol{\beta}} \mid \mathbf{X}) &= \sigma^2 (\mathbf{C} + \mathbf{A})(\mathbf{C} + \mathbf{A})^T \\ &= \sigma^2 (\mathbf{C} \mathbf{C}^T + \underbrace{\mathbf{C} \mathbf{A}^T}_{=\mathbf{0}} + \underbrace{\mathbf{A} \mathbf{C}^T}_{=\mathbf{0}} + \mathbf{A} \mathbf{A}^T) \\ &= \sigma^2 \mathbf{C} \mathbf{C}^T + \sigma^2 \mathbf{A} \mathbf{A}^T \\ &= \text{Var}(\hat{\boldsymbol{\beta}}_{\text{OLS}} \mid \mathbf{X}) + \sigma^2 \mathbf{A} \mathbf{A}^T.\end{aligned}$$

Ma trận $\mathbf{A} \mathbf{A}^T$ là **nửa xác định dương**, vì với mọi $\mathbf{v} \in \mathbb{R}^{p+1}$:

$$\mathbf{v}^T (\mathbf{A} \mathbf{A}^T) \mathbf{v} = \|\mathbf{A}^T \mathbf{v}\|_2^2 \geq 0.$$

Suy ra

$$\text{Var}(\tilde{\boldsymbol{\beta}} \mid \mathbf{X}) - \text{Var}(\hat{\boldsymbol{\beta}}_{\text{OLS}} \mid \mathbf{X}) = \sigma^2 \mathbf{A} \mathbf{A}^T \succeq \mathbf{0}.$$

Lấy phần tử đường chéo thứ j (chọn $\mathbf{v} = \mathbf{e}_j$) cho $\text{Var}(\tilde{\beta}_j) \geq \text{Var}(\hat{\beta}_{j,\text{OLS}})$. Vậy $\hat{\boldsymbol{\beta}}_{\text{OLS}}$ có phương sai nhỏ nhất trong Class các ước lượng tuyến tính không Bias, tức là BLUE.

Dấu bằng xảy ra khi và chỉ khi $\sigma^2 \mathbf{A} \mathbf{A}^T = \mathbf{0}$, tức $\mathbf{A} = \mathbf{0}$ hay $\mathbf{D} = \mathbf{C}$: OLS là ước lượng BLUE duy nhất. $\square$

1.3.3 Minh Họa Định Lý Gauss–Markov bằng Mô Phỏng Monte Carlo

1.3.3.a Ý tưởng

Định lý Gauss–Markov khẳng định hai tính chất của $\hat{\boldsymbol{\beta}}_{\text{OLS}}$: **không Bias** ($\mathbb{E}[\hat{\boldsymbol{\beta}} \mid \mathbf{X}] = \boldsymbol{\beta}$) và **phương sai nhỏ nhất** trong Class các ước lượng tuyến tính không Bias. Đây là các mệnh đề về *kỳ vọng* và *phương sai*, vốn là những đại lượng không quan sát được từ một mẫu duy nhất. Mô phỏng Monte Carlo cho phép **xấp xỉ** chúng: cố định ma trận thiết kế $\mathbf{X}$ và tham số thật $\boldsymbol{\beta}$, ta tạo lặp lại nhiều bộ dữ liệu chỉ khác nhau ở nhiễu ngẫu nhiên, rồi quan sát phân phối của các ước lượng thu được.

1.3.3.b Công thức ước lượng Monte Carlo

Với mỗi lần lặp $r = 1, \dots, M$, ta giữ nguyên $\mathbf{X}$ và tạo

$$\boldsymbol{\varepsilon}^{(r)} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n), \quad \mathbf{y}^{(r)} = \mathbf{X} \boldsymbol{\beta} + \boldsymbol{\varepsilon}^{(r)},$$

rồi tính ước lượng $\hat{\beta}^{(r)}$. Theo luật số lớn, trung bình và hiệp phương sai mẫu của $\{\hat{\beta}^{(r)}\}$ hội tụ về kỳ vọng và phương sai thật khi $M \rightarrow \infty$:

$$\bar{\beta} = \frac{1}{M} \sum_{r=1}^M \hat{\beta}^{(r)} \xrightarrow{M \rightarrow \infty} \mathbb{E}[\hat{\beta} | \mathbf{X}], \quad (12)$$

$$\hat{\mathbf{S}} = \frac{1}{M-1} \sum_{r=1}^M (\hat{\beta}^{(r)} - \bar{\beta})(\hat{\beta}^{(r)} - \bar{\beta})^T \xrightarrow{M \rightarrow \infty} \text{Var}(\hat{\beta} | \mathbf{X}). \quad (13)$$

Nếu định lý đúng, ta kỳ vọng $\bar{\beta} \approx \beta$ và $\hat{\mathbf{S}} \approx \sigma^2(\mathbf{X}^T \mathbf{X})^{-1}$.

1.3.3.1 Một ước lượng đối chứng (Alt). Để minh họa tính “tốt nhất”, ta cần một ước lượng tuyến tính không Bias *khác* để so phương sai. Theo đúng chứng minh BLUE, mọi ước lượng như vậy đều viết được dưới dạng OLS cộng thêm một nhiễu loạn. Ta lấy một ước lượng đối chứng (ký hiệu Alt):

$$\tilde{\beta}_{\text{Alt}} = \hat{\beta}_{\text{OLS}} + \mathbf{A}\mathbf{y}, \quad \text{với } \mathbf{A}\mathbf{X} = \mathbf{0}. \quad (14)$$

Điều kiện $\mathbf{A}\mathbf{X} = \mathbf{0}$ làm cho Alt vẫn **không Bias**: $\mathbb{E}[\tilde{\beta}_{\text{Alt}} | \mathbf{X}] = \mathbb{E}[\hat{\beta}_{\text{OLS}} | \mathbf{X}] + \mathbf{A}\mathbf{X}\beta = \beta$. Để tạo một $\mathbf{A}$ cụ thể, ta dùng ma trận chiếu (hat matrix) $\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ và đặt $\mathbf{A} = \mathbf{B}(\mathbf{I} - \mathbf{H})$ với $\mathbf{B}$ là ma trận hằng tùy ý; khi đó $\mathbf{A}\mathbf{X} = \mathbf{B}(\mathbf{I} - \mathbf{H})\mathbf{X} = \mathbf{0}$ vì $(\mathbf{I} - \mathbf{H})\mathbf{X} = \mathbf{0}$. Phương sai của Alt, theo đúng khai triển trong chứng minh Gauss–Markov, là

$$\text{Var}(\tilde{\beta}_{\text{Alt}} | \mathbf{X}) = \text{Var}(\hat{\beta}_{\text{OLS}} | \mathbf{X}) + \sigma^2 \mathbf{A}\mathbf{A}^T \succeq \sigma^2(\mathbf{X}^T \mathbf{X})^{-1}. \quad (15)$$

Vì $\mathbf{A}\mathbf{A}^T$ nửa xác định dương, Alt **không bao giờ** có phương sai nhỏ hơn OLS, phù hợp với kết luận của định lý.

Định lý Gauss–Markov là mệnh đề *có điều kiện trên $\mathbf{X}$* , nên trong toàn bộ M lần lặp ta **giữ nguyên $\mathbf{X}$** và chỉ cho nhiễu ε thay đổi. Nếu tạo lại $\mathbf{X}$ mỗi lần, ta sẽ trộn lẫn biến thiên của thiết kế vào kết quả.

1.3.3.c Thiết lập thực nghiệm

Table 3: Thiết lập thực nghiệm Monte Carlo.

Tham số	Giá trị
Số quan sát n ; số tham số $p + 1$	$n = 30$; $p + 1 = 3$; $\beta = (2.0, 1.0, -0.5)^T$
Phương sai nhiễu; số lần lặp	$\sigma^2 = 1.0$; $M = 8000$
Ma trận thiết kế $\mathbf{X}$	Cột 1 toàn số 1; cột 2-3 sinh từ $\mathcal{N}(0, 1)$, cố định suốt toàn bộ mô phỏng
Ước lượng đối chứng Alt	$\mathbf{A} = \mathbf{B}(\mathbf{I} - \mathbf{H})$, $\mathbf{B}$ là ma trận hằng tạo ngẫu nhiên một lần (<code>alt_scale = 0.06</code>)

1.3.3.d Cài đặt Python

Toàn bộ mô phỏng được tổ chức trong file `part1/monte_carlo_gauss_markov.py`, trong đó ước lượng OLS tái sử dụng trực tiếp hàm `ols_fit` đã xây dựng từ đầu, còn vòng lặp M lần lặp và ước lượng đối chứng Alt đều được cài đặt độc lập mà không phụ thuộc vào thư viện tối ưu ngoài. NumPy chỉ được dùng cho ba tác vụ hạn chế là sinh nhiễu ngẫu nhiên $\varepsilon^{(r)}$, tính trung bình và hiệp phương sai mẫu kết quả, và kiểm chứng nghiệm so với oracle, đúng theo nguyên tắc scratch-first của toàn bộ project.

1.3.3.e Kết quả mô phỏng

1.3.3.2 Bảng kết quả tổng hợp. Bảng 4 tóm tắt toàn bộ kết quả qua $M = 8000$ lần lặp. Cả hai ước lượng đều có trung bình trùng giá trị thật (cột $\bar{\beta}$ và $\bar{\tilde{\beta}}$), xác nhận **tính không Bias**. Cột phương sai cho thấy Alt lớn hơn OLS ở *mọi* hệ số (tỉ số > 1), qua đó minh họa trực tiếp **tính tối ưu** của OLS theo nghĩa BLUE.

Table 4: Trung bình và phương sai sau $M = 8000$ mô phỏng (OLS vs Alt). Độ Bias lớn nhất: $\max |\text{bias}_{\text{OLS}}| = 4.9 \times 10^{-4}$, $\max |\text{bias}_{\text{Alt}}| = 4.6 \times 10^{-3}$; cả hai không Bias, nhưng phương sai của Alt lớn hơn OLS ở mọi hệ số.

Hệ số	$\beta_{\text{thật}}$	$\bar{\hat{\beta}}_{\text{OLS}}$	$\bar{\tilde{\beta}}_{\text{Alt}}$	$\text{Var}(\hat{\beta}_{\text{OLS}})$	$\text{Var}(\tilde{\beta}_{\text{Alt}})$	Tỉ số
β_0 (hệ số tự do)	2.000	2.0001	1.9954	0.03506	0.08862	2.53
β_1	1.000	0.9995	1.0036	0.03257	0.14492	4.45
β_2	-0.500	-0.5001	-0.5010	0.04732	0.14989	3.17

1.3.3.3 (1) Tính không Bias. Hình 6 vẽ trung bình tích lũy (12) của ba hệ số OLS theo số lần lặp M (trục log). **Cách đọc:** mỗi đường là $\hat{\beta}_j(M) = \frac{1}{M} \sum_{r \leq M} \hat{\beta}_j^{(r)}$; đường chấm cùng màu là giá trị thật β_j .

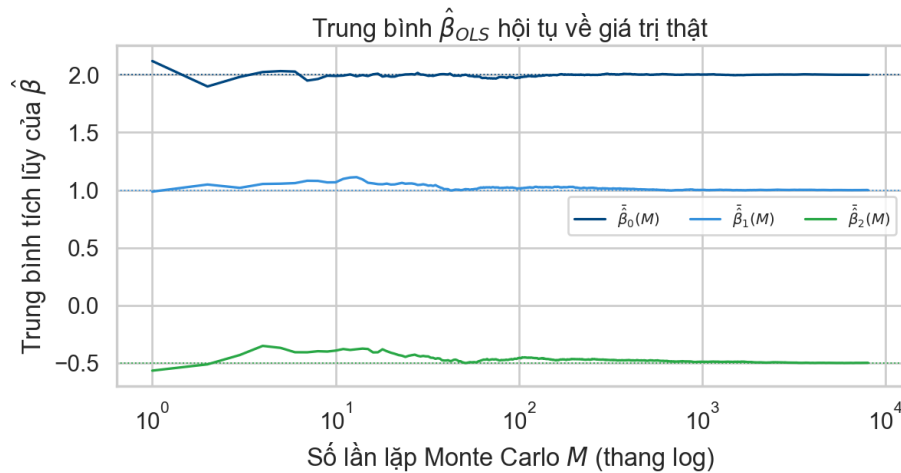


Figure 6: Trung bình tích lũy $\hat{\beta}(M)$ theo số lần lặp (thang log). Cả ba thành phần hội tụ về giá trị thật 2.0, 1.0, -0.5 (đường chấm).

Nhận xét. Khi M nhỏ, trung bình dao động mạnh do ít mẫu; khi M tăng, cả ba đường **ổn định và bám sát** đúng giá trị thật. Đây là biểu hiện của luật số lớn cho $\mathbb{E}[\hat{\beta}_{OLS}] = \beta$. Bảng 4 cũng cho thấy độ Bias của cả OLS lẫn Alt đều $\leq 4.6 \times 10^{-3}$, nhỏ hơn nhiều so với độ phân tán của ước lượng. (Độ Bias thực nghiệm của Alt hơi lớn hơn OLS chỉ vì phương sai lớn hơn nên sai số Monte Carlo của trung bình cũng lớn hơn, không phải thiên lệch hệ thống.)

1.3.3.4 (2) Phương sai nhỏ nhất. Hình 7 chồng phân phối lấy mẫu của OLS (xanh) và Alt (cam) cho từng hệ số. **Cách đọc:** mỗi ô là phân phối của một hệ số; vạch đỏ là giá trị thật.

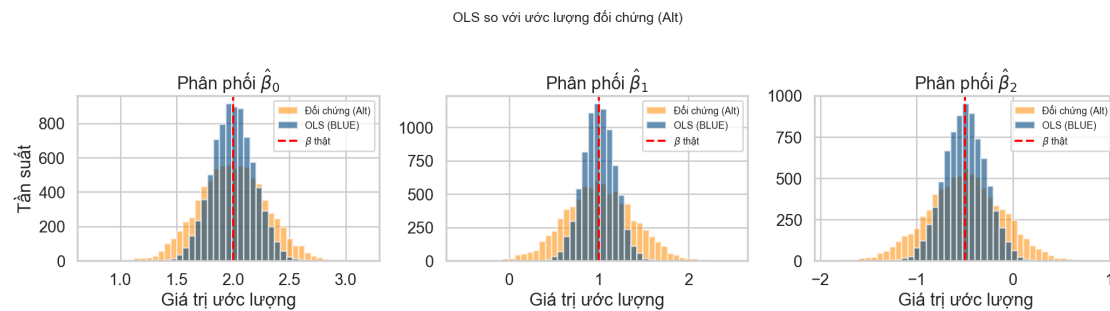


Figure 7: Phân phối lấy mẫu OLS (xanh) vs Alt (cam) theo từng hệ số. Cả hai cùng tâm tại β thật (vạch đỏ đứt), nhưng OLS hẹp hơn rõ rệt.

Nhận xét. Cả hai phân phối đều **cùng tâm** tại giá trị thật (vạch đỏ), xác nhận rằng OLS và Alt đều không Bias. Tuy nhiên, phân phối OLS (xanh) **cao và hẹp hơn**, tức tập trung quanh β chặt hơn, trong khi Alt (cam) trải rộng ở cả ba hệ số; đây là minh họa trực quan cho tính chất phương sai nhỏ nhất của OLS trong lớp các ước lượng tuyến tính không Bias (BLUE).

Hình 8 lượng hóa điều đó theo *từng* hệ số. **Cách đọc:** mỗi hệ số có 4 cột: OLS (Monte Carlo), OLS (lý thuyết), Alt (Monte Carlo), Alt (lý thuyết).

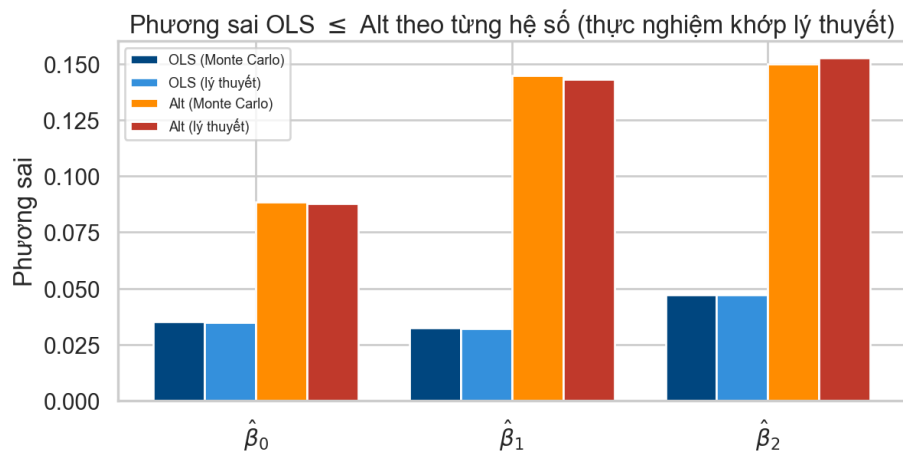


Figure 8: Phương sai theo từng hệ số: với mỗi hệ số, cặp cột Monte Carlo và lý thuyết gần như bằng nhau; cột Alt luôn cao hơn cột OLS.

Nhận xét. Với mỗi hệ số, cột **Monte Carlo** và cột **lý thuyết** gần như bằng nhau, cho thấy mô phỏng khớp chặt với công thức $\sigma^2(\mathbf{X}^T\mathbf{X})^{-1}$ (10) và $\text{Var}(\hat{\beta}_{\text{OLS}}) + \sigma^2\mathbf{A}\mathbf{A}^T$ (15). Cột Alt cao hơn cột OLS ở **mọi** hệ số với tỉ số từ 2.5 đến 4.5, phù hợp với kết luận BLUE của định lý.

1.3.3.f Kiểm chứng với NumPy và scikit-learn

Để bảo đảm cài đặt từ đầu là chính xác, ta so nghiệm OLS thu được trên một mẫu với hai công cụ chuẩn: `numpy.linalg.lstsq` và `sklearn.linear_model.LinearRegression` (đặt `fit_intercept=False` vì cột hệ số tự do đã nằm trong $\mathbf{X}$).

Hàm `verify_with_numpy_sklearn()` trong `part1/monte_carlo_gauss_markov.py` thực hiện kiểm chứng này:

Kết quả cho thấy sai số cực đại so với cả hai oracle đều đạt độ chính xác máy: $\left\| \hat{\beta}_{\text{tự xây dựng}} - \hat{\beta}_{\text{NumPy}} \right\|_{\infty} = \left\| \hat{\beta}_{\text{tự xây dựng}} - \hat{\beta}_{\text{sklearn}} \right\|_{\infty} = 6.66 \times 10^{-16}$. Ngoài ra, phương sai thực nghiệm Monte Carlo khớp với công thức lý thuyết ở mọi hệ số (xem chi tiết tại Bảng 4).

Mô phỏng Monte Carlo xác nhận đầy đủ Định lý Gauss–Markov: ước lượng OLS **không Bias** (trung bình trùng giá trị thật) và có **phương sai nhỏ nhất** so với ước lượng tuyến tính không Bias đối chứng (Alt). Cài đặt từ đầu trùng khớp NumPy/scikit-learn tới độ chính xác máy, và phương sai thực nghiệm khớp công thức $\sigma^2(\mathbf{X}^T \mathbf{X})^{-1}$.

2 Phần 2: Ứng Dụng Thực Tế

2.1 Ứng Dụng Data Fitting vào Dữ Liệu Thực Tế

Phần này áp dụng các phương pháp lý thuyết từ Phần 1 vào bộ dữ liệu thực tế về chi phí du lịch tại Tanzania. Bằng cách này, ta có thể kiểm chứng những khái niệm toán học trừu tượng trên một vấn đề thực tiễn, từ đó thực hiện toàn bộ quy trình khoa học từ thu thập dữ liệu, Preprocessing, xây dựng mô hình hồi quy, đến đánh giá hiệu năng và phân tích kết quả.

2.1.1 Mô Tả Dataset

2.1.1.a Tanzania Tourism Expenditure Dataset

Định nghĩa 2.1: Tanzania Tourism Expenditure Dataset

Dataset từ cuộc thi Zindi Platform chứa dữ liệu chi tiêu của du khách tại Tanzania, bao gồm thông tin nhân khẩu học, hành vi du lịch, và chi phí tổng cộng.

Thông tin cơ bản:

- **Tập huấn luyện:** 4,809 mẫu (quan sát)
- **Tập kiểm tra:** 1,601 mẫu (không có nhãn mục tiêu)
- **Biến mục tiêu:** `total_cost` (chi phí tổng cộng, TZS)
- **Loại biến độc lập:** 21 biến (gồm numeric và categorical)
- **Phạm vi mục tiêu:** 49,000 TZS đến 99,532,875 TZS

Danh sách các đặc trưng:

Biến số (numeric): `total_female`, `total_male`, `night_mainland`, `night_zanzibar`

Biến phân loại (categorical): `country`, `age_group`, `travel_with`, `purpose`, `main_activity`, `info_source`, `tour_arrangement`, `package_*` (8 biến), `payment_mode`, `first_trip_tz`, `most_impressing`

2.1.1.b Missing Values

Sau khi thực hiện quá trình thống kê mô tả thì nhóm đã có được các thống kê về ‘missing values’ như sau:

Cột	Số lượng	Phần trăm
<code>travel_with</code>	1,114	23.1%
<code>most_impressing</code>	313	6.5%
<code>total_male</code>	5	0.1%
<code>total_female</code>	3	0.06%

Table 5: Phân tích giá trị missing trong dataset huấn luyện.

Quan sát bảng trên cho thấy rằng `travel_with` chứa 23.1% missing values, trong khi `most_impressing` có 6.5%. Và các cột còn lại như `total_male` và `total_female` có mức độ missing cực kỳ nhỏ (dưới 1%), từ đó ta sẽ xử lý chúng bằng phương pháp imputation phù hợp để đảm bảo không làm mất thông tin quan trọng.

2.1.2 Phân Tích Dữ Liệu Khám Phá (EDA)

Tukey [19] định nghĩa EDA là quá trình dùng các công cụ thống kê và trực quan hóa để hiểu cấu trúc của dữ liệu trước khi áp đặt bất kỳ mô hình nào. Trong hồi quy tuyến tính, EDA đóng vai trò đặc biệt quan trọng vì nó giúp kiểm tra sơ bộ các giả thiết Gauss–Markov (GM1–GM4) ngay từ đầu, từ đó định hướng các quyết định Preprocessing và lựa chọn mô hình [4]. Nhóm đã xây dựng script `part2/src/eda.py` để thực hiện toàn bộ quá trình EDA, bao gồm phân tích phân bố biến mục tiêu, khám phá tương quan, kiểm tra đa cộng tuyến và phát hiện data shift.

2.1.2.a Phân Bố Biến Mục Tiêu

Phân tích phân bố biến mục tiêu y là bước đầu tiên vì phân bố của y ảnh hưởng trực tiếp đến giả thiết GM5 ($\varepsilon \mid \mathbf{X} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$). Khi y bị lệch mạnh (high skewness), phần dư ε cũng sẽ không chuẩn, làm mất hiệu lực của kiểm định t và F , đồng thời kéo lệch ước lượng OLS do hàm mất mát $\text{RSS} = \|\mathbf{y} - \mathbf{X}\hat{\beta}\|^2$ nhạy cảm với giá trị cực đoan [4]. James et al. [14] khuyến nghị kiểm tra phân bố mục tiêu và áp dụng biến đổi dữ liệu (log, sqrt) khi cần thiết để đưa y về gần phân bố chuẩn trước khi mô hình hóa.

Hình 9 trình bày phân tích chi tiết về phân bố của biến mục tiêu `total_cost`. Biểu đồ bên trái (original distribution) cho thấy phân bố có độ lệch cao (skewness = 2.97) với hầu hết du

khách chi tiêu ít hơn 10 triệu TZS, nhưng một số du khách lại chi tiêu rất cao (lên đến 99 triệu TZS), tạo ra một đuôi dài phía bên phải. Biểu đồ ở giữa áp dụng phép biến đổi logarit ($\log_{10}$) để giảm độ lệch xuống -0.32 , giúp dữ liệu gần với phân bố chuẩn hơn. Biểu đồ boxplot ở bên phải hiển thị các outliers theo quy tắc IQR.

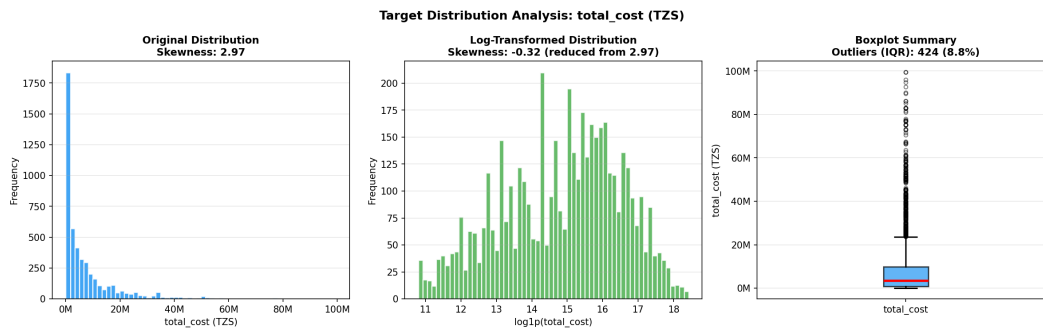


Figure 9: Phân tích biến mục tiêu `total_cost`: (Trái) Phân bố gốc có độ lệch cao (skewness = 2.97); (Giữa) Phân bố sau biến đổi $\log_{10}$ với độ lệch giảm xuống -0.32 ; (Phải) Boxplot hiển thị outliers theo IQR.

Vì Skewness = 2.97 cho thấy y lệch nặng, phần dư $\hat{\epsilon}$ khó thỏa mãn giả thiết chuẩn GM5; do đó ta áp dụng biến đổi $\log_{10}(y)$ trước khi huấn luyện. Sau biến đổi, skewness giảm xuống -0.32 , phân phối y gần chuẩn hơn, giúp củng cố hiệu lực của các kiểm định t và F . Ngoài ra, số lượng lớn outliers trong boxplot gợi ý đưa Huber Regression vào danh sách so sánh để đánh giá xem mô hình robust có cải thiện đáng kể so với OLS hay không.

2.1.2.b Phân Bố Các Biến Số

Bên cạnh biến mục tiêu, việc phân tích phân bố của từng biến đầu vào cũng là bước cần thiết trong EDA vì một biến có phân bố lệch mạnh hoặc chứa nhiều outliers sẽ ảnh hưởng tiêu cực đến chất lượng mô hình theo hai hướng khác nhau. Về mặt Preprocessing, phân bố lệch làm giảm hiệu quả của chuẩn hóa z-score vì phép biến đổi này giả định dữ liệu phân bố đối xứng quanh trung bình. Về mặt ước lượng, outliers trong biến đầu vào $\mathbf{X}$ có thể gián tiếp kéo lệch $\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ vì ma trận $\mathbf{X}^T \mathbf{X}$ sẽ bị chi phối bởi các hàng chứa giá trị cực đoan, khiến ước lượng nghiêng về phía phục vụ các điểm bất thường đó [4]. Tukey [19] đề xuất phân tích histogram của từng biến như bước bắt buộc để hiểu cấu trúc phân bố và phát hiện các nhóm ẩn (mixture distributions) trước khi mô hình hóa.

Hình 10 thể hiện phân bố của 4 biến numeric chính trong dataset. Các biến `total_female` và `total_male` tập trung mạnh ở giá trị thấp với đại đa số quan sát dưới 5 người, phản ánh

rằng phần lớn du khách đi một mình hoặc trong nhóm rất nhỏ. Ngược lại, `night_mainland` và `night_zanzibar` có phân bố trải rộng từ vài đêm đến hơn 60 đêm, gợi ý rằng nhiều phân khúc du khách khác nhau cùng tồn tại trong dataset, từ những chuyến đi ngắn ngày đến các hành trình dài.

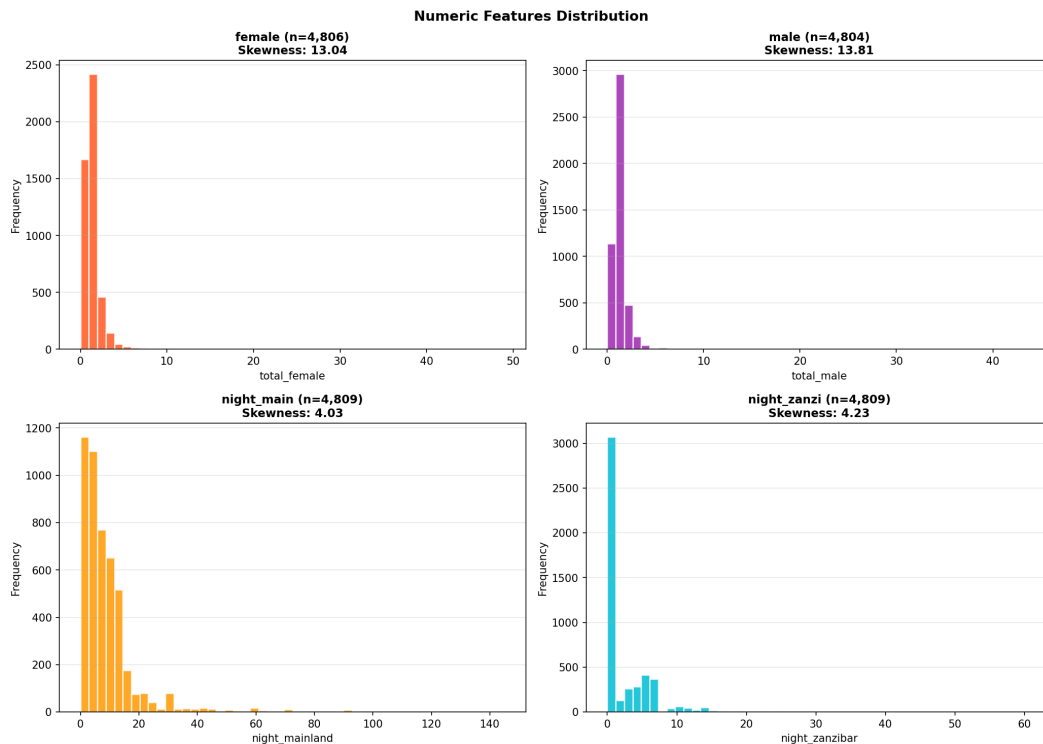


Figure 10: Phân bố của 4 biến numeric: `total_female` và `total_male` tập trung ở giá trị thấp; `night_mainland` và `night_zanzibar` có phân bố rộng với đuôi phải dài.

Vì cả 4 biến đều có phân bố lệch phải rõ rệt, bước biến đổi `log1p` rồi chuẩn hóa z-score là cần thiết trong pipeline Preprocessing để đưa các biến về cùng thang đo và đảm bảo penalty của Ridge, Lasso và ElasticNet được áp dụng công bằng lên tất cả các hệ số. Ngoài ra, phân bố đa đỉnh và đuôi dài của `night_mainland` gợi ý rằng mối quan hệ giữa biến này và `total_cost` có thể mang tính phi tuyến, và đây là một trong những lý do để đưa Polynomial Regression và Kernel Ridge Regression vào danh sách so sánh.

2.1.2.c Ma Trận Tương Quan

Ma trận tương quan Pearson giúp phát hiện đồng thời hai vấn đề quan trọng trong hồi quy tuyến tính. Vấn đề thứ nhất là multicollinearity, tức là khi hai biến đầu vào có $|r| > 0.8$ thì chúng

tuyến tính phụ thuộc vào nhau, vi phạm giả thiết GM2 và làm phòng phương sai ước lượng $\hat{\beta}$, khiến các kiểm định t trở nên không đáng tin cậy. Vấn đề thứ hai là tiềm năng Prediction tuyến tính của từng feature đối với target, cụ thể là cột cuối của heatmap biểu diễn $r(X_j, y)$ cho từng biến j : khi $|r(X_j, y)| > 0.5$, biến đó có mối liên hệ tuyến tính mạnh với y và OLS có thể khai thác trực tiếp; còn khi $|r(X_j, y)| < 0.2$, điều đó không có nghĩa là biến đó vô dụng, bởi vì nó có thể phản ánh hai tình huống hoàn toàn khác nhau: hoặc là biến thực sự không liên quan đến y , hoặc là mối quan hệ tồn tại nhưng mang tính phi tuyến nên không được nắm bắt bởi hệ số Pearson [14]. Greene [5] nhấn mạnh rằng bước phân tích này cần được thực hiện trước khi lựa chọn mô hình để định hướng sớm: nếu tất cả $|r(X_j, y)|$ đều thấp, cần xem xét các mô hình phi tuyến thay vì chỉ dựa vào OLS thuần túy.

Hình 11 hiển thị ma trận tương quan Pearson đầy đủ giữa tất cả các biến numeric và biến mục tiêu. Tương quan dương (màu đỏ) chỉ ra mối liên hệ cùng chiều, trong khi tương quan âm (màu xanh) chỉ ra mối liên hệ ngược chiều. `night_mainland` và `night_zanzibar` có tương quan dương vừa phải với `total_cost` ở mức khoảng 0.20 đến 0.24, cho thấy thời gian lưu trú dài hơn thường đi kèm với chi phí cao hơn. Đáng chú ý là không có cặp biến đầu vào nào đạt $|r| > 0.5$.

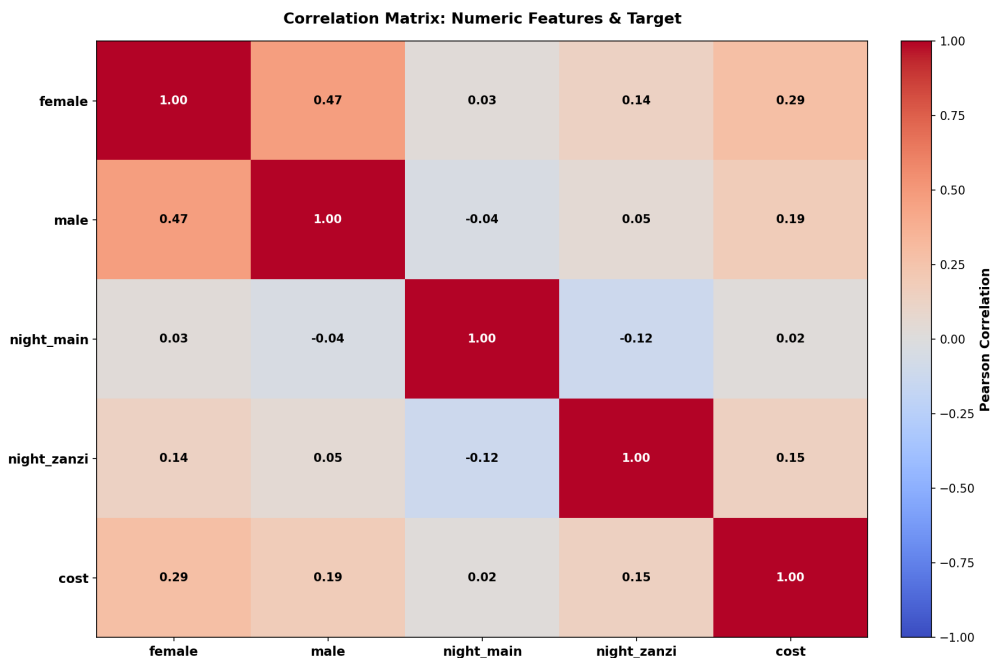


Figure 11: Ma trận tương quan Pearson của các biến numeric và target. Không có cặp biến đầu vào nào có $|r| > 0.5$, cho thấy không có multicollinearity nghiêm trọng giữa các biến số.

Việc không có cặp biến đầu vào nào đạt $|r| > 0.5$ cho thấy không có dấu hiệu multicollinearity nghiêm trọng từ phân tích tương quan đơn biến, dù vậy kết luận này cần được xác nhận thêm bằng VIF vì tương quan theo cặp không phát hiện được multicollinearity bậc cao giữa nhiều biến cùng lúc. Tuy nhiên, điều đáng lưu ý hơn là tương quan của tất cả biến số với target đều thấp hơn 0.25, gợi ý rằng sức mạnh Prediction tuyến tính chủ yếu đến từ các biến phân loại như quốc gia xuất phát và hình thức tour sau khi được mã hóa, chứ không phải từ các biến số. Đây là cơ sở để ưu tiên one-hot encoding kỹ lưỡng cho các biến phân loại trong bước Preprocessing tiếp theo.

2.1.2.d Phân Tích Outliers Qua Boxplot

Mặc dù histogram đã cho thấy phân bố tổng quan của từng biến, boxplot cung cấp thêm một góc nhìn cụ thể hơn về sự hiện diện và mức độ nghiêm trọng của outliers theo quy tắc $1.5 \times IQR$ do Tukey [19] đề xuất. Bước này đặc biệt quan trọng trong bối cảnh OLS vì phương pháp này tối thiểu hóa tổng bình phương phần dư $\|\mathbf{y} - \mathbf{X}\hat{\beta}\|^2$, nghĩa là mỗi outlier đóng góp theo bình phương khoảng cách vào hàm mất mát và do đó có ảnh hưởng không cân xứng so với các quan sát bình thường [4]. Nói cụ thể hơn, một quan sát có giá trị gấp đôi ngưỡng bình thường sẽ kéo ước lượng $\hat{\beta}$ lệch gấp bốn lần so với một quan sát bình thường, khiến outliers trở thành mối nguy hiểm tiềm ẩn mà histogram có thể bỏ sót nếu chọn thùng phân chia không phù hợp.

Hình 12 trình bày boxplot cho 4 biến số chính. Các biến `total_female` và `total_male` có median gần bằng 0 với nhiều outliers đơn lẻ phía trên, phản ánh rằng phần lớn quan sát là du khách đơn lẻ trong khi một số đoàn lớn tạo ra các giá trị cực đoan. Đáng chú ý hơn, `night_mainland` và `night_zanzibar` có IQR rộng và outliers nghiêm trọng hơn, với một số du khách lưu trú hơn 60 đêm, gấp nhiều lần so với giá trị trung vị của toàn bộ tập dữ liệu.

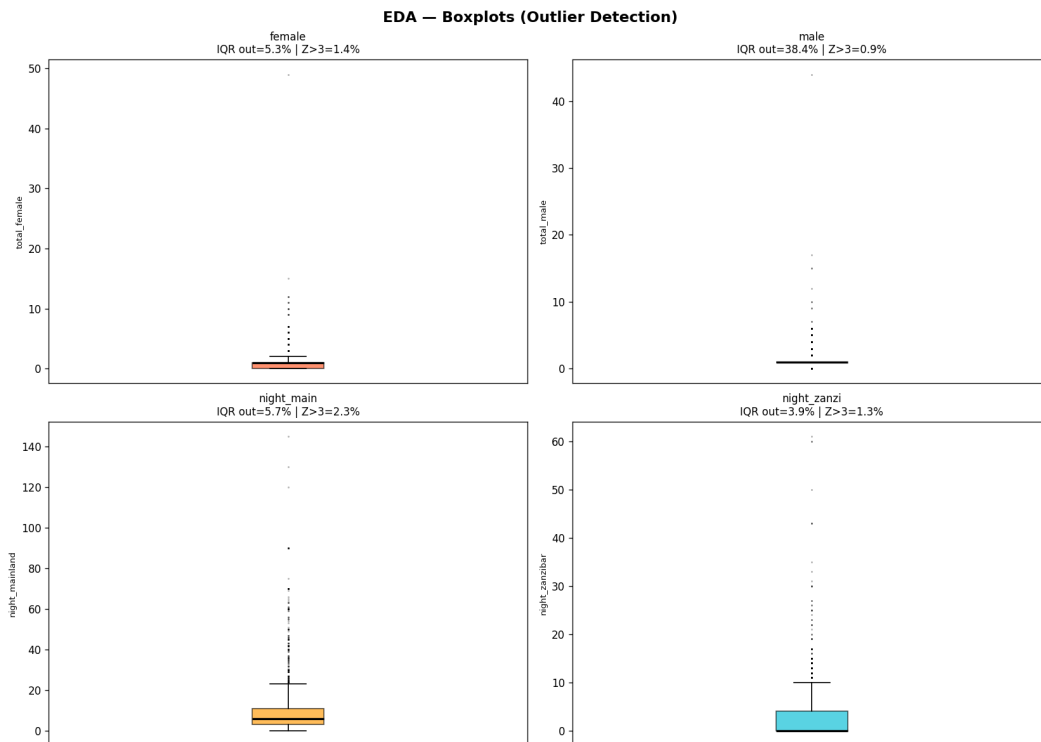


Figure 12: Boxplot của 4 biến numeric. `total_female` và `total_male` có median gần 0 với nhiều điểm ngoại lệ; `night_mainland` và `night_zanzibar` có IQR rộng và outliers nghiêm trọng.

Dựa vào mức độ outliers quan sát được, ta đưa ra quyết định đưa Huber Regression vào danh sách so sánh cùng với OLS. Khác với OLS, Huber Regression sử dụng hàm mất mát kết hợp giữa bình phương cho các quan sát nằm trong vùng bình thường và tuyến tính cho các quan sát vượt ngưỡng ε , từ đó giảm ảnh hưởng của outliers một cách có kiểm soát. Kết quả so sánh giữa OLS và Huber Regression sau này sẽ cho thấy mức độ mà outliers thực sự làm lệch ước lượng trên bộ dữ liệu này.

2.1.2.e Kiểm Tra Đa Cộng Tuyến (VIF)

Phân tích tương quan Pearson theo cặp chỉ phát hiện được multicollinearity giữa hai biến, trong khi trên thực tế multicollinearity có thể xuất hiện theo kiểu đa biến, tức là một biến có thể được xấp xỉ tốt bởi tổ hợp tuyến tính của nhiều biến khác ngay cả khi không có cặp nào tương quan cao với nhau. Để kiểm tra toàn diện hơn, ta sử dụng Variance Inflation Factor (VIF), được

định nghĩa là:

$$VIF_j = \frac{1}{1 - R_j^2}$$

trong đó R_j^2 là hệ số xác định khi hồi quy biến j lên tất cả các biến còn lại [9]. Khi VIF vượt ngưỡng 10, phương sai của $\hat{\beta}_j$ bị phồng đến mức ước lượng mất ổn định và kiểm định t không còn đáng tin cậy [4]. Ngoài ra, VIF cao còn là lý do chính dẫn đến việc áp dụng Ridge Regression, vì penalty $\lambda\|\beta\|^2$ giúp ổn định $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}$ khi $\mathbf{X}^T\mathbf{X}$ gần suy biến do multicollinearity, do đó kết quả VIF sẽ trực tiếp ảnh hưởng đến cách chọn λ phù hợp [10].

Hình 13 hiển thị VIF cho các biến số sau chuẩn hóa, với hai ngưỡng cảnh báo VIF = 5 và VIF = 10 được đánh dấu bằng đường đứt nét. Toàn bộ biến số của dataset Tanzania Tourism đều có VIF dưới 2, cho thấy không có vấn đề đa cộng tuyến nào cần xử lý trong nhóm biến số.

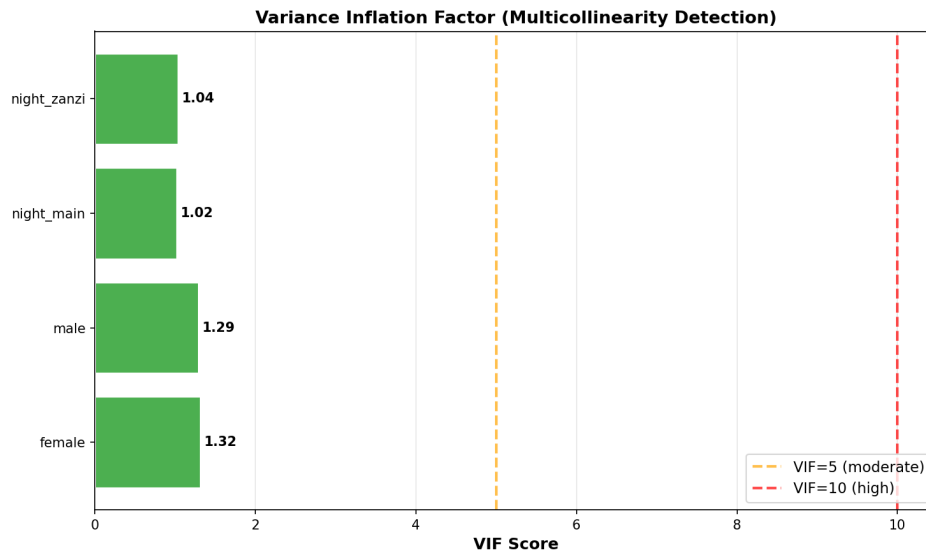


Figure 13: VIF cho các biến numeric. Tất cả VIF < 2, thấp hơn nhiều so với ngưỡng cảnh báo 5 và 10, xác nhận không có đa cộng tuyến.

Kết quả VIF dưới 2 cho tất cả biến số xác nhận giả thiết GM2 được thỏa mãn với biên độ rộng, từ đó dẫn đến ba quyết định mô hình quan trọng. Trước hết, không cần loại bỏ bất kỳ biến nào do multicollinearity. Tiếp theo, vì không có phương sai bị phồng do đa cộng tuyến, Ridge Regression với λ rất lớn là không cần thiết và giá trị λ tối ưu sẽ được xác định qua cross-validation thay vì được đặt thủ công ở mức cao. Cuối cùng, OLS cơ bản là nền tảng so sánh hợp lệ vì phương sai ước lượng không bị phồng do cấu trúc dữ liệu.

2.1.2.f Tương Quan Các Biến với Mục Tiêu

Trong khi ma trận tương quan đầy đủ cung cấp cái nhìn tổng quan về mối quan hệ giữa tất cả các cặp biến, việc trực quan hóa riêng tương quan của từng biến với target giúp ta đánh giá nhanh tiềm năng Prediction của từng feature theo nghĩa tuyến tính đơn biến. Cần lưu ý một giới hạn quan trọng của phân tích này: hệ số Pearson chỉ đo mối quan hệ tuyến tính và không nắm bắt được tương tác giữa các biến hay các hiệu ứng phi tuyến. Wooldridge [4] lưu ý rằng một biến có tương quan thấp với target theo nghĩa đơn biến không nhất thiết phải loại bỏ, vì trong mô hình đa biến biến đó có thể đóng góp đáng kể sau khi kiểm soát các biến còn lại. Vì vậy, kết quả của bước này cần được xem như một tín hiệu định hướng chứ không phải tiêu chí loại biến, và cần được đối chiếu với bảng hệ số $\hat{\beta}$ của mô hình đầy đủ [14].

Hình 14 trình bày tương quan Pearson giữa mỗi biến numeric và `total_cost`. Các biến `night_mainland` và `night_zanzibar` có tương quan dương vừa phải ở mức khoảng 0.20 đến 0.24, xác nhận rằng thời gian lưu trú dài hơn thường đi kèm với chi phí cao hơn. Các biến `total_female` và `total_male` có tương quan yếu hơn đáng kể, chỉ ở mức dưới 0.10.



Figure 14: Tương quan Pearson của các biến numeric với `total_cost`. Tương quan cao nhất chỉ đạt ≈ 0.24 (`night_mainland`), cho thấy sức mạnh Prediction tuyến tính tổng thể của biến số là hạn chế.

Tất cả biến số đều có tương quan đơn biến với target dưới 0.25, từ đó ta có thể rút ra hai nhận định quan trọng định hướng cho việc lựa chọn mô hình. Nhận định thứ nhất là sức mạnh Prediction tuyến tính của dataset chủ yếu đến từ các biến phân loại như quốc gia xuất phát hay

hình thức tour sau khi được mã hóa one-hot, chứ không phải từ các biến số thuần túy. Nhận định thứ hai là mối quan hệ giữa biến số và target, dù tương quan Pearson thấp, vẫn có thể tồn tại theo nghĩa phi tuyến mà hệ số Pearson không nắm bắt được. Hai nhận định này cộng lại là cơ sở lý luận để đưa Polynomial Regression và Kernel Ridge Regression vào danh sách mô hình so sánh, bên cạnh các mô hình tuyến tính.

2.1.2.g Phát Hiện Data Shift (Kolmogorov–Smirnov Test)

Một giả định quan trọng nhưng thường bị bỏ qua trong thực hành là giả định rằng dữ liệu huấn luyện và dữ liệu kiểm tra đến từ cùng một phân bố, tức là $P_{\text{train}}(\mathbf{x}) = P_{\text{test}}(\mathbf{x})$. Khi giả định này bị vi phạm, hiện tượng được gọi là covariate shift xảy ra và mô hình đã huấn luyện trên tập train có thể không tổng quát hóa tốt sang tập test dù loss trên tập huấn luyện thấp, bởi vì mô hình đã học các quy luật phù hợp với phân bố của train nhưng phân bố đó không còn đúng trên test nữa [14]. Để kiểm tra điều này một cách định lượng, ta áp dụng kiểm định Kolmogorov–Smirnov với giả thuyết H_0 rằng hai mẫu đến từ cùng phân bố, và bác bỏ H_0 khi p-value nhỏ hơn 0.05 [20]. Bước này cần thiết để xác nhận rằng các chỉ số đánh giá trên test set phản ánh khả năng tổng quát hóa thực sự của mô hình chứ không bị sai lệch bởi sự khác biệt phân bố giữa hai tập.

Hình 15 hiển thị p-value của kiểm định cho từng biến numeric. Thanh xanh (p-value ≥ 0.05) cho thấy phân bố giữa tập huấn luyện và tập kiểm tra là tương đồng, trong khi thanh đỏ (p-value < 0.05) cảnh báo sự khác biệt đáng kể về phân bố. Phần lớn các biến có phân bố tương đồng giữa hai tập, điều này tốt cho tính tin cậy của kết quả đánh giá mô hình.

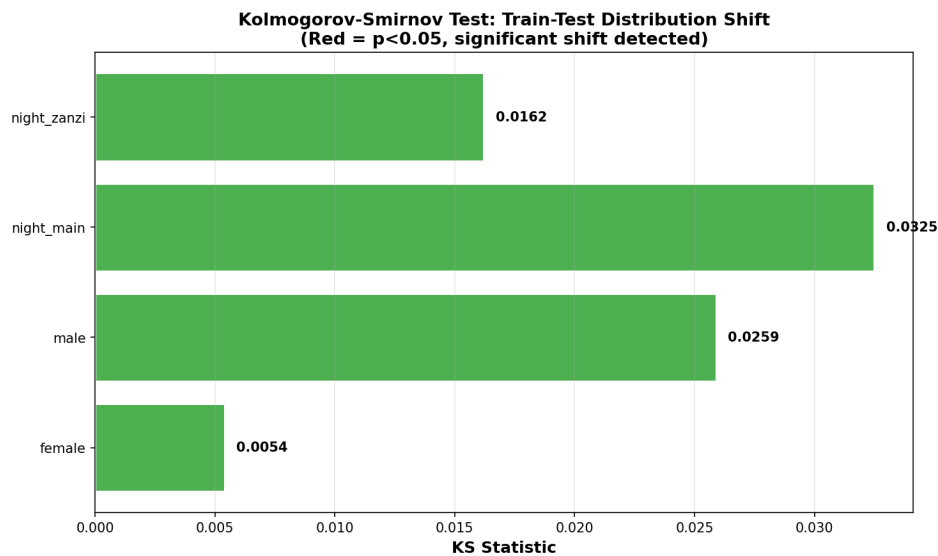


Figure 15: Kiểm định Kolmogorov–Smirnov phát hiện data shift giữa tập huấn luyện và kiểm tra. Thanh xanh: không có shift ($p \geq 0.05$); thanh đỏ: shift có ý nghĩa thống kê ($p < 0.05$).

Vì phần lớn các biến có p-value vượt ngưỡng 0.05, phân bố train và test là tương đồng, và do đó các chỉ số MAE, RMSE và R^2 trên test set có thể được xem là đại diện đáng tin cậy cho hiệu suất thực tế của mô hình trong triển khai. Đối với những biến có thanh màu đỏ nếu có, cần ghi chú khi phân tích kết quả vì ước lượng hệ số $\hat{\beta}$ tương ứng có thể bị lệch trên tập kiểm tra do phân bố đầu vào thay đổi, dẫn đến khả năng hiệu suất thực tế thấp hơn so với số liệu trên test set.

2.1.3 Preprocessing Dữ Liệu

2.1.3.a Xử Lý Giá Trị Thiếu

Giá trị thiếu nếu không được xử lý sẽ vi phạm giả thiết GM2 vì ma trận $\mathbf{X}$ không đầy đủ hàng, đồng thời làm giảm kích thước tập dữ liệu nếu ta xóa toàn bộ các hàng thiếu (listwise deletion). Tuy nhiên, trước khi chọn phương pháp xử lý, điều quan trọng là phải phân tích cơ chế thiếu (missing data mechanism) của từng biến, vì mỗi cơ chế đòi hỏi một cách tiếp cận khác nhau [4]. Cụ thể, có ba loại cơ chế: MCAR (Missing Completely At Random) khi xác suất thiếu hoàn toàn ngẫu nhiên và không phụ thuộc vào bất kỳ biến nào; MAR (Missing At Random) khi xác suất thiếu phụ thuộc vào các biến đã quan sát được nhưng không phụ thuộc vào chính giá trị bị thiếu đó; và MNAR (Missing Not At Random) khi xác suất thiếu phụ thuộc vào chính giá trị bị thiếu, đây là trường hợp phức tạp nhất và đòi hỏi mô hình hóa riêng.

Đối với `total_female` và `total_male` chỉ thiếu dưới 1% quan sát, cơ chế thiếu nhiều khả năng là MCAR vì đây là lỗi nhập liệu ngẫu nhiên, không có lý do có hệ thống nào khiến thông tin số lượng thành viên trong đoàn bị bỏ trống. Đối với `travel_with` thiếu tới 23.1%, cơ chế có thể là MAR: những du khách đi một mình có xu hướng bỏ qua câu hỏi này vì không có đồng hành nào để mô tả, do đó xác suất thiếu phụ thuộc vào các biến khác như số lượng thành viên đoàn. Còn `most_impressing` thiếu 6.5% cũng có thể là MAR vì du khách ít ấn tượng hơn có xu hướng bỏ qua câu hỏi mở này nhiều hơn. Trong cả hai trường hợp MAR và MCAR, phương pháp Median/Unknown Imputation là lựa chọn hợp lý vì nó không đưa ra thêm giả định mạnh về phân bố có điều kiện của giá trị thiếu, đồng thời dễ dàng đảm bảo nguyên tắc fit-on-train-transform-on-test để tránh data leakage.

Cụ thể, với các biến numeric, ta điền giá trị thiếu bằng trung vị của cột tính trên tập huấn luyện vì median bền hơn mean khi dữ liệu có outlier lớn: `total_female` và `total_male` đều được điền bằng 1.00 cho các hàng thiếu. Với các biến categorical, ta điền bằng một nhãn riêng “Unknown” thay vì ép về mode, để mô hình có thể học được trạng thái “không trả lời” như một tín hiệu riêng. Sau bước này, tập huấn luyện hoàn toàn không còn giá trị thiếu, đảm bảo ma trận thiết kế $\mathbf{X}$ đầy đủ hàng và $\mathbf{X}^T \mathbf{X}$ có thể tính được.

2.1.3.b Mã Hóa Biến Phân Loại

Hầu hết các thuật toán hồi quy tuyến tính, bao gồm OLS, Ridge và Lasso, hoạt động thông qua phép nhân ma trận $\mathbf{X}\beta$ trong đó $\mathbf{X}$ phải là ma trận số thực. Do đó, 17 biến categorical trong dataset cần được chuyển đổi sang dạng số trước khi đưa vào mô hình. Khi đứng trước lựa chọn phương pháp mã hóa, ta cần cân nhắc giữa hai hướng phổ biến: ordinal encoding gán một số nguyên cho mỗi category theo một thứ tự nào đó, còn one-hot encoding tạo một cột nhị phân riêng cho mỗi category. Với dataset Tanzania Tourism, các biến categorical như `country`, `purpose` hay `main_activity` không có thứ tự tự nhiên có ý nghĩa, vì vậy ordinal encoding sẽ tạo ra quan hệ số học giả tạo giữa các category, ví dụ ngầm giả định rằng quốc gia số 3 nằm giữa quốc gia số 2 và số 4 theo nghĩa nào đó [14]. Do đó, one-hot encoding là lựa chọn phù hợp hơn vì nó cho phép mô hình học hệ số riêng biệt cho từng category mà không áp đặt thứ tự giả tạo nào.

Sau khi áp dụng one-hot encoding cho 17 biến categorical, ví dụ biến `country` với hơn 70 quốc gia tạo ra hơn 70 cột nhị phân tương ứng, tổng số cột Tạo ra từ bước mã hóa là 146 (tính trên tập huấn luyện thực tế). Kết hợp với 4 biến numeric ban đầu và 1 cột hệ số tự do, ma trận thiết kế $\mathbf{X}$ sau mã hóa có kích thước $3,847 \times 151$. Việc số lượng features tăng đột biến từ 4 lên

150 (chưa tính hệ số tự do) là hệ quả tất yếu của one-hot encoding và chính điều này đặt ra câu hỏi về overfitting, từ đó củng cố thêm lý do cần so sánh OLS với các mô hình có regularization trong bước tiếp theo.

2.1.3.c Chuẩn Hóa Đặc Trưng

Chuẩn hóa các biến numeric là bước bắt buộc khi sử dụng các phương pháp regularization như Ridge và Lasso, bởi vì penalty $\lambda\|\beta\|^2$ được tính trực tiếp trên không gian tham số và không phân biệt đơn vị đo lường. Nếu hai biến có thang đo khác nhau, chẳng hạn `night_mainland` có phạm vi hàng chục đêm trong khi `total_female` thường chỉ từ 0 đến 5 người, thì hệ số $\hat{\beta}$ của biến có thang đo lớn hơn sẽ tự nhiên nhỏ hơn và do đó bị penalize ít hơn, ngay cả khi biến đó không quan trọng hơn. Điều này làm sai lệch quá trình chọn biến của Lasso và làm mất ý nghĩa của hệ số trong phân tích feature importance [10]. Ngoài ra, chuẩn hóa còn giúp cho các hệ số $\hat{\beta}$ sau ước lượng có thể so sánh trực tiếp với nhau, từ đó phân tích tầm quan trọng của từng feature trở nên có ý nghĩa hơn [14].

Ta áp dụng chuẩn hóa z-score theo công thức:

$$x_{\text{scaled}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (16)$$

trong đó μ_{train} và σ_{train} lần lượt là trung bình và độ lệch chuẩn được tính *chỉ trên tập huấn luyện*. Trước bước z-score, các biến numeric không âm được biến đổi bằng `log1p` để giảm lệch phải và hạn chế ảnh hưởng của outlier. Sau đó, cùng một cặp giá trị $(\mu_{\text{train}}, \sigma_{\text{train}})$ đó được dùng để transform cả tập kiểm tra, đảm bảo nguyên tắc không để thông tin từ test set rò rỉ vào quá trình huấn luyện. Chú ý rằng chuẩn hóa chỉ được áp dụng cho 4 biến numeric vì các biến one-hot đã ở thang đo nhị phân $\{0, 1\}$ và không cần chuẩn hóa thêm.

2.1.3.d Dữ Liệu Sau Xử Lý

Sau khi hoàn thành ba bước Preprocessing trên, ta thu được ma trận thiết kế $\mathbf{X}_{\text{train}} \in \mathbb{R}^{3847 \times 151}$ và $\mathbf{X}_{\text{test}} \in \mathbb{R}^{1601 \times 151}$, trong đó 151 cột bao gồm 1 cột hệ số tự do, 4 biến numeric đã qua `log1p` và chuẩn hóa, cùng 146 cột one-hot từ 17 biến categorical. Tập dữ liệu sau xử lý không còn giá trị thiếu nào, tất cả biến numeric có trung bình 0 và phương sai 1, và toàn bộ biến categorical đã được chuyển về dạng nhị phân. Đặc biệt quan trọng là toàn bộ pipeline trên tuân theo nguyên tắc fit-on-train-transform-on-test: mọi thống kê dùng để impute, mọi category dùng để one-hot encode và mọi tham số dùng để chuẩn hóa đều được tính từ tập huấn luyện

trước khi áp dụng lên tập kiểm tra, từ đó đảm bảo đánh giá mô hình trên test set phản ánh đúng khả năng tổng quát hóa trong thực tế.

2.1.4 Xây Dựng và Đánh Giá Mô Hình

Bảy mô hình hồi quy và một mô hình ensemble được xây dựng trong nghiên cứu này. Nhóm mô hình tuyến tính gồm OLS, OLS có chọn biến, Ridge, Lasso và ElasticNet; nhóm mở rộng gồm Polynomial Regression kết hợp ElasticNet và Kernel Ridge Regression với RBF kernel. Ensemble trung bình được tạo riêng để phục vụ submission lên Zindi sau khi mỗi mô hình đã có file Prediction riêng. Toàn bộ mô hình được huấn luyện trên $\mathbf{X}_{\text{train}} \in \mathbb{R}^{3847 \times 151}$ với biến mục tiêu đã được biến đổi $\log(1 + \text{total_cost})$ để khắc phục vấn đề skewness cao đã phát hiện trong EDA. Khi đánh giá local CV, Prediction được đưa ngược về đơn vị TZS bằng $\exp(\hat{y}) - 1$ để metric gần hơn với ý nghĩa thực tế của bài toán.

2.1.4.a Mô Hình 1: Ordinary Least Squares (OLS)

OLS là mô hình nền tảng (baseline) của hồi quy tuyến tính, giải bài toán tối thiểu hóa tổng bình phương phần dư (Residual Sum of Squares):

$$\hat{\beta}_{\text{OLS}} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 \quad (17)$$

Vì hàm mục tiêu là bậc hai và lồi (convex) theo β , điều kiện đạo hàm bằng 0 cho ra hệ phương trình pháp tắc (normal equations) $\mathbf{X}^T \mathbf{X} \hat{\beta} = \mathbf{X}^T \mathbf{y}$. Khi $\mathbf{X}^T \mathbf{X}$ khả nghịch, bài toán có nghiệm dạng đóng:

$$\hat{\beta}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (18)$$

Theo Định lý Gauss–Markov đã chứng minh ở Phần 1, $\hat{\beta}_{\text{OLS}}$ là ước lượng tuyến tính không Bias có phương sai nhỏ nhất (BLUE) khi các giả thiết GM1–GM4 thỏa mãn. Tuy nhiên trong bối cảnh ứng dụng này, với 187 feature và chỉ 4,809 quan sát (tỷ lệ $n/p \approx 26$), tính BLUE không đủ để ngăn chặn overfitting: mô hình có đủ tự do để fit cả noise lẫn signal trong dữ liệu huấn luyện, dẫn đến hiệu suất kém trên dữ liệu mới [14]. Vì lý do này, OLS được dùng làm baseline để so sánh với các mô hình có regularization.

2.1.4.b Mô Hình 2: Ridge Regression

Ridge Regression, được đề xuất bởi Hoerl và Kennard [10], bổ sung một penalty L2 vào hàm mất mát của OLS nhằm kiểm soát độ lớn của hệ số:

$$\hat{\beta}_{\text{Ridge}} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2 \} \quad (19)$$

Penalty $\lambda \|\beta\|_2^2$ ép buộc các hệ số không được quá lớn, từ đó giảm phương sai của ước lượng và đánh đổi bằng một lượng bias nhỏ. Nhờ dạng bậc hai của penalty L2, bài toán tối ưu vẫn có nghiệm dạng đóng:

$$\hat{\beta}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \quad (20)$$

Ma trận $(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})$ luôn khả nghịch với mọi $\lambda > 0$, ngay cả khi $\mathbf{X}^T \mathbf{X}$ gần suy biến, đây là ưu điểm quan trọng về mặt số học. Về mặt hình học, Ridge tương đương với bài toán tối ưu có ràng buộc $\|\beta\|_2^2 \leq t$: nghiệm co lại về phía gốc tọa độ theo mọi hướng nhưng không đặt hệ số nào chính xác bằng 0. Tham số $\lambda = 10$ được lựa chọn qua 5-fold cross-validation trên lưới $\{0.1, 1, 10, 100, 1000, 10000\}$, trong đó $\lambda = 10$ đạt CV RMSE thấp nhất trên thang TZS (chi tiết tại mục 2.1.5).

2.1.4.c Mô Hình 3: Lasso Regression

Lasso (Least Absolute Shrinkage and Selection Operator), được đề xuất bởi Tibshirani [11], thay penalty L2 của Ridge bằng penalty L1:

$$\hat{\beta}_{\text{Lasso}} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \alpha \|\beta\|_1 \} \quad (21)$$

Sự khác biệt căn bản so với Ridge nằm ở hình học của vùng ràng buộc: penalty L1 tạo ra một hình thoi (L1-ball) với các đỉnh nhọn nằm trên các trục tọa độ. Khi vùng đẳng mức của hàm mất mát tiếp xúc với hình thoi, giao điểm thường rơi đúng vào một đỉnh nhọn, khiến một số hệ số bằng chính xác bằng 0 [11]. Tính chất này biến Lasso thành công cụ feature selection tự động: những feature không đóng góp đủ để bù lại chi phí penalty $\alpha |\beta_j|$ sẽ bị loại hoàn toàn.

Khác với OLS và Ridge, hàm mục tiêu của Lasso chứa chuẩn L1 không khả vi tại gốc tọa độ nên không tồn tại nghiệm dạng đóng. Thuật toán Coordinate Descent được sử dụng để tìm nghiệm: tại mỗi bước lặp, từng hệ số β_j được cập nhật bằng toán tử soft-threshold trong khi giữ nguyên tất cả các hệ số còn lại.

Algorithm 9 Coordinate Descent cho Lasso

Require: $\mathbf{X} \in \mathbb{R}^{n \times p}$, $\mathbf{y} \in \mathbb{R}^n$, $\alpha > 0$, ngưỡng hội tụ $\varepsilon > 0$

Ensure: $\hat{\beta} \in \mathbb{R}^p$ (sparse solution)

```
1:  $\beta \leftarrow \mathbf{0}$  ▷ khởi tạo tất cả hệ số bằng 0
2: repeat
3:    $\beta_{\text{old}} \leftarrow \beta$ 
4:   for  $j = 1$  to  $p$  do
5:      $\rho_j \leftarrow \mathbf{x}_j^T (\mathbf{y} - \mathbf{X}_{-j} \beta_{-j}) / n$  ▷ partial residual correlation
6:      $\hat{\beta}_j \leftarrow \mathcal{S}(\rho_j, \alpha) / (\|\mathbf{x}_j\|^2 / n)$  ▷ soft-threshold update
7:   end for
8: until  $\|\beta - \beta_{\text{old}}\|_2 < \varepsilon$ 
9: return  $\beta$ 
```

Toán tử soft-threshold: $\mathcal{S}(z, \alpha) = \text{sign}(z) \cdot \max(|z| - \alpha, 0)$

Với $\alpha^* \approx 0.00316$ được tối ưu qua coordinate descent thủ công kết hợp 3-fold CV trên tập huấn luyện, Lasso chỉ giữ lại 49 hệ số khác 0 trong 151 hệ số. Các feature bị loại bỏ thường là những quốc gia xuất hiện rất ít lần trong tập huấn luyện, nơi mà ước lượng hệ số không đủ độ tin cậy thống kê để vượt qua ngưỡng penalty $\alpha|\beta_j|$, đặc biệt là với biến mục tiêu đã qua biến đổi log1p làm thu hẹp khoảng biến động của y và do đó yêu cầu alpha nhỏ hơn nhiều so với thang đo TZS gốc.

2.1.4.d Mô Hình 4: ElasticNet

ElasticNet, được Zou và Hastie đề xuất vào năm 2005 [12], ra đời để giải quyết đồng thời hai hạn chế còn tồn đọng khi áp dụng riêng lẻ Lasso hoặc Ridge. Khi các biến đầu vào có tương quan cao với nhau, Lasso có xu hướng chỉ giữ lại một biến đại diện trong nhóm và loại bỏ phần còn lại, dù tất cả đều mang thông tin Prediction có giá trị; Ridge ngược lại giữ toàn bộ biến nhưng không thực hiện feature selection, khiến mô hình kém diễn giải trong bối cảnh có nhiều biến phân loại như Tanzania Tourism. ElasticNet khắc phục cả hai hạn chế bằng cách kết hợp penalty L1 và L2 theo một tỷ lệ linh hoạt:

$$\hat{\beta}_{\text{EN}} = \arg \min_{\beta} \left\{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \alpha \rho \|\beta\|_1 + \frac{\alpha(1-\rho)}{2} \|\beta\|_2^2 \right\} \quad (22)$$

trong đó $\alpha > 0$ kiểm soát mức độ tổng phạt và $\rho \in [0, 1]$ là tỷ lệ thành phần L1 (`l1_ratio`): khi $\rho = 1$ thì ElasticNet suy về Lasso, khi $\rho = 0$ suy về Ridge, còn các giá trị trung gian tạo ra sự cân bằng linh hoạt giữa sparsity và ổn định số học. Tương tự Lasso, hàm mục tiêu của ElasticNet không khả vi tại điểm 0 của penalty L1 nên cũng cần giải bằng Coordinate Descent,

song sự hiện diện của thành phần L2 giúp hội tụ ổn định hơn khi các cột của $\mathbf{X}$ có tương quan cao — một tính chất đặc biệt quan trọng khi bộ đặc trưng chứa nhiều biến quốc gia có phân phối tương tự nhau.

Hai siêu tham số α và ρ được lựa chọn đồng thời qua coordinate descent scratch kết hợp 3-fold CV thử công với các giá trị α theo thang logarit và $\rho \in \{0.3, 0.5, 0.7, 0.9\}$. Trên bộ dữ liệu Tanzania Tourism với biến mục tiêu log1p, kết quả tìm được $\alpha^* \approx 0.00215$ và $\rho^* = 0.50$. Ở cài đặt này, ElasticNet giữ 77 hệ số khác 0, tức thừa hơn Ridge nhưng ít cực đoan hơn Lasso. Điều này xác nhận rằng thành phần L1 vẫn là phần quan trọng để chọn biến, còn thành phần L2 giúp giữ lại thêm một số feature có tín hiệu yếu nhưng ổn định.

2.1.4.e Mô Hình 5: Polynomial Regression kết hợp ElasticNet

Phân tích EDA cho thấy các biến số như `night_mainland` và `night_zanzibar` có phân bố đa đỉnh, gợi ý rằng mối quan hệ giữa thời gian lưu trú và chi phí du lịch có thể mang tính phi tuyến hoặc bị điều tiết bởi các phân khúc du khách ẩn. Để kiểm tra giả thuyết này, nhóm mở rộng không gian đặc trưng bằng cách tạo Polynomial Features bậc 2 từ 4 biến số gốc, rồi kết hợp với toàn bộ biến phân loại đã one-hot encode để tạo thành bộ đặc trưng mở rộng. Với $p_{\text{num}} = 4$ biến số và đa thức bậc 2, số đặc trưng đa thức mới là:

$$|\mathcal{F}_{\text{poly}}| = \binom{p_{\text{num}} + 2}{2} - 1 = \binom{6}{2} - 1 = 14 \quad (23)$$

nâng tổng số đặc trưng lên 161 (gồm 14 đa thức và 147 biến phân loại). Do số đặc trưng tăng làm trầm trọng thêm nguy cơ overfitting, ElasticNet được sử dụng thay vì OLS thuần túy để đồng thời thực hiện regularization và feature selection trên tập đặc trưng mở rộng, với siêu tham số tối ưu qua coordinate descent scratch kết hợp 3-fold CV thử công.

Tuy nhiên, kết quả thực nghiệm cho thấy mô hình này không tổng quát hóa tốt hơn ElasticNet tuyến tính: CV R^2 chỉ quanh mức 0.30 và CV RMSE gần như không cải thiện. Nguyên nhân chính là do các đặc trưng đa thức của biến số (vốn có tương quan đơn biến thấp với mục tiêu theo phân tích EDA, khoảng 0.20–0.24) không cung cấp tín hiệu Prediction đủ mạnh để bù đắp cho độ phức tạp gia tăng. Hơn nữa, `l1_ratio` hội tụ về phía penalty L1 khiến nhiều hệ số đa thức bị loại bỏ và mô hình thực chất vẫn gần với một mô hình tuyến tính thừa trên bộ đặc trưng mở rộng.

2.1.4.f Mô Hình 6: Kernel Ridge Regression (KRR)

Kernel Ridge Regression kết hợp cơ chế penalty L2 của Ridge với kernel trick, cho phép học các mối quan hệ phi tuyến mà không cần mở rộng tường minh không gian đặc trưng. Thay vì tìm $\hat{\beta}$ trực tiếp như Ridge, KRR biểu diễn nghiệm dưới dạng tổ hợp tuyến tính của các hàm nhân trên các điểm dữ liệu huấn luyện:

$$\hat{f}(\mathbf{x}) = \sum_{i=1}^n \hat{\alpha}_i K(\mathbf{x}_i, \mathbf{x}), \quad \hat{\alpha} = (\mathbf{K} + \alpha \mathbf{I})^{-1} \mathbf{y} \quad (24)$$

trong đó $\mathbf{K} \in \mathbb{R}^{n \times n}$ là ma trận Gram với $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$. Nghiệm có dạng giống tương tự Ridge nhưng yêu cầu nghịch đảo ma trận kích thước $n \times n$ thay vì $p \times p$, dẫn đến độ phức tạp $O(n^3)$ — cao hơn đáng kể so với Ridge ($O(p^3)$ khi $p < n$). Vì vậy, trong pipeline thực nghiệm, KRR được tuning trên một mẫu con cố định và fit cuối trên 1,200 quan sát để đảm bảo script vẫn chạy được ổn định trên máy cá nhân. Với RBF kernel tốt nhất trong lưới nhỏ ($\gamma = 0.01, \alpha = 0.1$), mô hình đạt Train $R^2 \approx 0.309$ khi Prediction được đưa về đơn vị TZS.

Hiệu năng kém của KRR trên bộ dữ liệu này xuất phát từ một vấn đề căn bản: hàm nhân RBF dựa trên khoảng cách Euclidean $\|\mathbf{x}_i - \mathbf{x}_j\|^2$ trong không gian đặc trưng one-hot, trong khi khoảng cách đó không phản ánh đúng sự tương đồng ngữ nghĩa giữa các quan sát. Hai du khách có cùng quốc gia nhưng mã one-hot cho các đặc trưng khác nhau sẽ bị đo là "xa nhau" trong không gian Euclidean, dù thực tế họ có hành vi du lịch tương đồng. Điều này làm cho kernel RBF kém hiệu quả hơn nhiều trên dữ liệu tabular có nhiều biến phân loại so với dữ liệu số liên tục như ảnh hoặc tín hiệu âm thanh.

2.1.4.g Đánh Giá và So Sánh Các Mô Hình

Để so sánh khách quan các mô hình, ta đánh giá trên hai tiêu chí bổ sung cho nhau: Train R^2 phản ánh mức độ fit dữ liệu, và CV R^2 qua cross-validation phản ánh khả năng tổng quát hóa sang dữ liệu chưa thấy. Bảng 6 tổng hợp kết quả của các mô hình đang được cài đặt trong pipeline chính, trong đó target được fit trên thang $\log(1 + \text{total_cost})$ nhưng metric được tính lại trên đơn vị TZS sau khi nghịch biến đổi.

Table 6: So sánh hiệu năng của các mô hình: local (CV R^2 , CV RMSE trên `dev_train`) và public (MAE trên tập test Zindi). CV của KRR được tính trên mẫu con 750 quan sát do chi phí ma trận Gram lớn, nên chỉ dùng như tín hiệu tham khảo. Target được fit trên $\log(1 + \text{total_cost})$; metric TZS được tính sau khi nghịch biến đổi.

Mô hình	Tham số	Train R^2	CV R^2	CV RMSE	Coef. $\neq 0$	Zindi MAE
OLS	–	0.3096	0.2807 ± 0.0324	10,331,582	151 / 151	5,200,442
OLS + Selection	$p < 0.05$	0.2790	0.2639 ± 0.0434	10,446,977	19 / 151	5,171,074
Ridge	$\lambda = 10$	0.3096	0.2943 ± 0.0345	10,232,252	151 / 151	5,178,704
Lasso	$\alpha \approx 0.00316$	0.3030	0.2933 ± 0.0360	10,236,884	49 / 151	5,143,958
ElasticNet	$\alpha \approx 0.00215, \rho = 0.50$	0.3068	0.2943 ± 0.0350	10,231,314	77 / 151	5,201,186
Poly(2) + ElasticNet	$\alpha \approx 0.00251, \rho = 0.50$	0.3077	0.2914 ± 0.0357	10,251,884	81 / 161	5,296,831
KRR (RBF)	$\alpha = 0.1, \gamma = 0.01$	0.3091	0.2955 ± 0.0803	9,001,252*	–	5,211,044
Ensemble mean	Trung bình 5 mô hình	0.3141	–	–	–	5,158,441

* CV của KRR được chạy trên mẫu con 750 quan sát, nên không so sánh trực tiếp với các mô hình còn lại vốn dùng 5-fold trên toàn bộ tập `dev_train`.

Từ Bảng 6, có thể rút ra ba nhận định quan trọng. Thứ nhất, kết quả public trên Zindi (cột **Zindi MAE**) cho thấy **Lasso đạt MAE tốt nhất** với 5,143,958 TZS, theo sau là Ensemble (5,158,441) và Ridge (5,178,704). Điều đáng chú ý là trong local CV, Lasso có CV $R^2 = 0.2933$ thấp hơn Ridge (0.2943), nhưng trên tập test thực tế Lasso lại vượt trội. Điều này gợi ý rằng tính thưa của Lasso (chỉ giữ 49/151 hệ số) giúp tránh overfitting tốt hơn trên dữ liệu test có phân phối nhẹ khác train.

Thứ hai, Ensemble (trung bình 5 mô hình) đạt Zindi MAE thứ nhì (5,158,441), xác nhận giá trị của việc kết hợp nhiều mô hình khác nhau để giảm variance. Poly(2)+ElasticNet có kết quả kém nhất (5,296,831), cho thấy các đặc trưng đa thức bậc 2 gây overfitting thực sự trên tập test thay vì chỉ là không cải thiện local CV như đã quan sát.

Thứ ba, Polynomial + ElasticNet không cải thiện rõ rệt so với ElasticNet tuyến tính, phản ánh rằng các feature đa thức của 4 biến numeric chưa đủ mạnh để bù cho độ phức tạp tăng thêm. KRR được giữ như một thử nghiệm kernel phi tuyến nhưng cần đọc kết quả cẩn thận vì CV của nó được chạy trên mẫu con; hơn nữa, RBF kernel trên không gian one-hot vẫn có hạn chế là khoảng cách Euclidean không phản ánh tốt sự tương đồng ngữ nghĩa giữa các category. Ensemble mean được tạo để nộp thử trên Zindi, và điểm chính thức sẽ được điền sau khi upload từng file submission riêng.

2.1.5 Cross-Validation và Lựa Chọn Siêu Tham Số

Đánh giá hiệu năng mô hình trên một lần phân chia train/test duy nhất có thể bị ảnh hưởng nhiều bởi cách phân chia ngẫu nhiên đó, khiến ước lượng không ổn định và có thể lạc quan hoặc bi quan hơn thực tế. K-fold cross-validation khắc phục vấn đề này bằng cách chia dữ liệu thành k fold bằng nhau, thực hiện k lần huấn luyện trong đó mỗi fold lần lượt đóng vai trò tập validation, từ đó thu được ước lượng hiệu năng có độ tin cậy cao hơn nhiều [14]. Trong nghiên cứu này, CV còn đóng vai trò quan trọng hơn: đây là công cụ duy nhất để chọn λ cho Ridge một cách khách quan, vì nếu dùng train loss để chọn thì mô hình sẽ luôn ưu tiên λ nhỏ hơn do penalty nhỏ hơn cho train loss thấp hơn, dù điều đó không phản ánh khả năng tổng quát hóa thực sự.

2.1.5.a K-Fold Cross-Validation

Algorithm 10 K-Fold Cross-Validation

Require: dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, model $\mathcal{M}$, số fold $k = 5$, metric f

Ensure: CV score: $\bar{s} \pm \sigma_s$ qua k fold

- 1: Phân chia $\mathcal{D}$ thành k fold bằng nhau $\mathcal{F}_1, \dots, \mathcal{F}_k$
 - 2: **for** $i = 1$ **to** k **do**
 - 3: $\mathcal{D}_{\text{train}} \leftarrow \mathcal{D} \setminus \mathcal{F}_i$ ▷ $k-1$ fold dùng để huấn luyện
 - 4: $\mathcal{D}_{\text{val}} \leftarrow \mathcal{F}_i$ ▷ fold thứ i dùng để đánh giá
 - 5: Huấn luyện $\mathcal{M}$ trên $\mathcal{D}_{\text{train}}$ thu được $\hat{\mathcal{M}}_i$
 - 6: $s_i \leftarrow f(\hat{\mathcal{M}}_i, \mathcal{D}_{\text{val}})$ ▷ tính metric trên fold validation
 - 7: **end for**
 - 8: $\bar{s} \leftarrow \frac{1}{k} \sum_{i=1}^k s_i, \quad \sigma_s \leftarrow \text{std}(\{s_i\}_{i=1}^k)$
 - 9: **return** $\bar{s} \pm \sigma_s$
-

Với $k = 5$, mỗi mô hình được huấn luyện và đánh giá 5 lần trên các phân chia dữ liệu khác nhau. Bảng 7 trình bày kết quả local CV cho các mô hình chính trong pipeline. Riêng KRR được đánh giá trên mẫu con vì chi phí tính toán của ma trận Gram tăng theo $O(n^3)$.

Table 7: Kết quả cross-validation local (mean $\pm$ std, đơn vị MAE và RMSE là TZS). Giá trị tốt nhất trong nhóm 5-fold đầy đủ được in đậm.

Mô hình	Tham số	CV MAE	CV RMSE	CV R^2
OLS	–	5,011,711 $\pm$ 226,986	10,331,582 $\pm$ 423,612	0.2807 $\pm$ 0.0324
OLS + Selection	$p < 0.05$	5,053,593 $\pm$ 234,592	10,446,977 $\pm$ 406,776	0.2639 $\pm$ 0.0434
Ridge	$\lambda = 10$	4,950,371 $\pm$ 199,750	10,232,252 $\pm$ 402,682	0.2943 $\pm$ 0.0345
Lasso	$\alpha \approx 0.00316$	4,946,955 $\pm$ 185,020	10,236,884 $\pm$ 365,044	0.2933 $\pm$ 0.0360
ElasticNet	$\alpha \approx 0.00215$	4,945,950 $\pm$ 194,322	10,231,314 $\pm$ 389,048	0.2943 $\pm$ 0.0350
Poly(2) + ElasticNet	$\alpha \approx 0.00251$	4,962,556 $\pm$ 187,821	10,251,884 $\pm$ 386,849	0.2914 $\pm$ 0.0357
KRR (mẫu con)	$\alpha = 0.1, \gamma = 0.01$	4,692,189 $\pm$ 367,802	9,001,252 $\pm$ 522,844	0.2955 $\pm$ 0.0803

Ridge với $\lambda = 10$ đạt CV RMSE thấp nhất trong nhóm mô hình được đánh giá bằng 5-fold đầy đủ, trong khi ElasticNet đạt mức CV R^2 ngang bằng. Kết quả này xác nhận rằng regularization có cải thiện khả năng tổng quát hóa so với OLS không có penalty (CV R^2 tăng từ 0.2807 lên 0.2943), nhưng mức cải thiện không quá lớn vì tín hiệu chính của bài toán vẫn đến từ các biến phân loại đã one-hot encode.

2.1.5.b Lựa Chọn Siêu Tham Số λ cho Ridge

Tham số λ kiểm soát mức độ shrinkage: λ quá nhỏ đưa mô hình về gần OLS và dễ overfitting, còn λ quá lớn co tất cả hệ số về gần 0 và gây underfitting. Tham số tối ưu được xác định theo tiêu chí:

$$\lambda^* = \arg \max_{\lambda \in \Lambda} CV\text{-}R^2(\lambda) \quad (25)$$

trên lưới giá trị $\Lambda = \{0.1, 1, 10, 100, 1,000, 10,000\}$. Bảng 8 trình bày kết quả đầy đủ.

Table 8: CV R^2 của Ridge trên lưới giá trị λ . Giá trị tốt nhất in đậm.

λ	CV R^2	Nhận xét
0.1	0.2834	gần OLS, regularization rất nhẹ
1	0.2880	bắt đầu ổn định hơn OLS
10	0.2947	λ^* — cân bằng bias-variance tốt nhất
100	0.2875	penalty bắt đầu làm tăng bias
1,000	0.2016	underfitting rõ rệt
10,000	−0.0582	underfitting nghiêm trọng

CV R^2 đạt cực đại tại $\lambda = 10$ (đạt 0.2947) và giảm dần khi penalty tăng quá mạnh, phản ánh rõ ràng bias-variance tradeoff: vùng λ quá nhỏ gần với OLS nên phương sai còn cao, còn vùng λ quá lớn làm mất khả năng biểu diễn của mô hình. Giá trị $\lambda^* = 10$ nằm tại điểm cân bằng tối ưu trên đường cong này.

2.1.6 Phân Tích Tầm Quan Trọng Đặc Trưng

Để hiểu mô hình Prediction điều gì và dựa vào những thông tin nào, ta phân tích tầm quan trọng của từng feature thông qua độ lớn tuyệt đối của hệ số $|\hat{\beta}_j|$ trong mô hình OLS. Cách tiếp cận này hoạt động được vì tất cả biến số đã được chuẩn hóa z-score và biến categorical đã được one-hot encode về thang đo nhị phân, do đó $|\hat{\beta}_j|$ lớn phản ánh feature j có ảnh hưởng thực sự lớn đến Prediction chỉ phí chứ không phải do thang đo lớn hơn các feature khác [14].

2.1.6.a Top 10 Đặc Trưng Quan Trọng Nhất

Bảng 9 xếp hạng 10 feature có $|\hat{\beta}_j|$ lớn nhất trong mô hình OLS.

Table 9: Top 10 feature có ảnh hưởng lớn nhất theo mô hình OLS, xếp hạng theo $|\hat{\beta}_j|$ giảm dần.

Hạng	Feature	$ \hat{\beta}_j $ (log1p)
1	country_COSTARICA	2.350
2	country_PHILIPINES	2.004
3	country_MADAGASCAR	1.872
4	country_TUNISIA	1.733
5	country_CROATIA	1.633
6	country_KUWAIT	1.531
7	country_SCOTLAND	1.487
8	country_YEMEN	1.478
9	country_CYPRUS	1.396
10	country_COMORO	1.376

Kết quả trong Bảng 9 cho thấy toàn bộ 10 feature quan trọng nhất đều là biến quốc gia (`country_*`), với hệ số dao động từ 1.38 đến 2.35 trên thang log1p. Điều này xác nhận nhận định từ phân tích EDA rằng quốc gia xuất phát của du khách là yếu tố quyết định chi tiêu, có thể phản ánh sức mua khác nhau giữa các nền kinh tế hoặc các mẫu hành vi du lịch đặc trưng theo quốc tịch. Ngoài ra, hệ số cao của một số quốc gia hiếm trong OLS cũng là dấu hiệu của overfitting: khi một quốc gia có rất ít quan sát trong tập huấn luyện, OLS có xu hướng fit các

điểm đó rất chặt và Tạo ra hệ số cực đoan, trong khi Ridge sẽ co hệ số này lại về phía 0 một cách có kiểm soát.

2.1.7 Kết Quả Mô Hình Hồi Quy

2.1.7.a So Sánh Hệ Số Giữa Các Phương Pháp

Một trong những cách trực quan nhất để hiểu tác động của regularization là quan sát trực tiếp sự thay đổi của các hệ số $\hat{\beta}$ giữa OLS, Ridge và Lasso. Nếu regularization hoạt động đúng như lý thuyết, ta kỳ vọng Ridge co lại tất cả hệ số về phía 0 một cách đồng đều, còn Lasso đặt một số hệ số về đúng 0 trong khi giữ nguyên các hệ số còn lại ở mức gần với OLS hơn [11]. Phân tích này cũng giúp kiểm chứng xem mô hình đang dựa vào các feature có ý nghĩa kinh tế thực sự hay chỉ fitting vào noise.

Hình 16 so sánh hệ số của OLS, Ridge và Lasso cho 30 feature có magnitude lớn nhất. OLS có hệ số lớn nhất do không có penalty kiểm chế, phản ánh xu hướng overfitting khi fit quá sát các quốc gia ít xuất hiện. Ridge co lại toàn bộ hệ số một cách đồng đều, giúp ổn định ước lượng mà vẫn giữ phần lớn 151 hệ số. Lasso cũng co lại hệ số nhưng đồng thời chỉ giữ 49 hệ số khác 0, tạo ra mô hình thưa hơn và dễ diễn giải hơn.

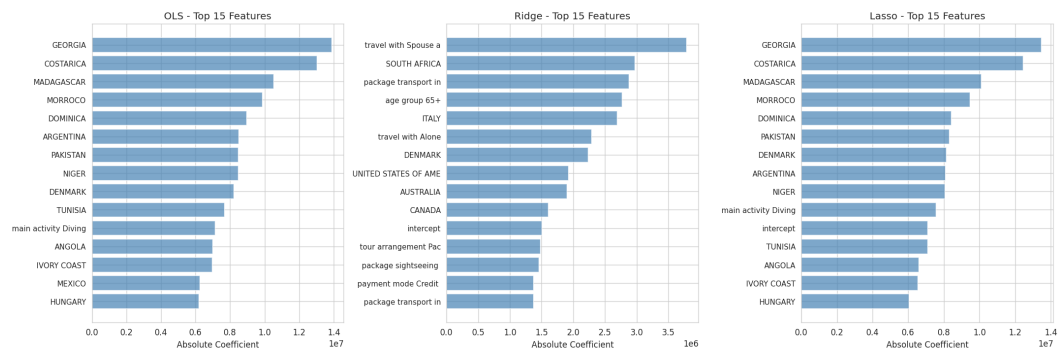


Figure 16: So sánh hệ số $\hat{\beta}_j$ của OLS, Ridge và Lasso cho 30 feature hàng đầu theo magnitude OLS. Ridge co đều tất cả hệ số (shrinkage), Lasso đặt một số hệ số về đúng 0 (sparsity).

Hành vi co hệ số của Ridge và Lasso quan sát được trong hình hoàn toàn khớp với Prediction lý thuyết, từ đó xác nhận rằng cả hai mô hình đang hoạt động đúng như thiết kế và penalty đang kiểm soát overfitting theo đúng cơ chế đã phân tích ở mục 2.1.5.

2.1.7.b Hiệu Năng Cross-Validation

Hình 17 trực quan hóa so sánh MAE, RMSE và R^2 qua 5-fold cross-validation cho ba mô hình, bổ sung cho bảng số liệu đã trình bày ở Bảng 7. Mục đích của biểu đồ này là làm rõ không chỉ giá trị trung bình mà còn độ biến động (error bar) của các chỉ số qua từng fold, từ đó đánh giá sự ổn định của từng mô hình.

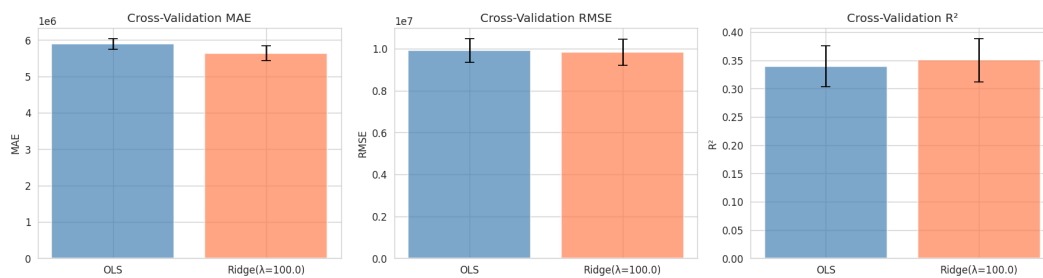


Figure 17: So sánh MAE, RMSE và R^2 trên 5-fold cross-validation cho OLS, Ridge và Lasso. Error bar thể hiện độ lệch chuẩn qua 5 fold. Ridge đạt CV R^2 cao nhất với error bar không lớn hơn đáng kể so với OLS.

Ridge đạt CV R^2 cao nhất và CV MAE thấp nhất, đồng thời duy trì error bar tương đương OLS, cho thấy sự ổn định không bị đánh đổi khi sử dụng regularization. Kết quả này củng cố quyết định chọn Ridge làm mô hình chính cho phân tích residual và feature importance.

2.1.7.c Chẩn Đoán Phần Dư

Sau khi chọn Ridge là mô hình tốt nhất, bước tiếp theo là kiểm tra xem mô hình có vi phạm các giả thiết Gauss–Markov hay không thông qua phân tích phần dư $\hat{\epsilon} = \mathbf{y} - \mathbf{X}\hat{\beta}_{\text{Ridge}}$. Phân tích này quan trọng vì nếu các giả thiết bị vi phạm, các suy luận thống kê (kiểm định t , khoảng tin cậy) sẽ không còn hiệu lực, dù ước lượng $\hat{\beta}$ vẫn nhất quán [4]. Bốn biểu đồ chẩn đoán chuẩn được dùng: Residuals vs Fitted kiểm tra tuyến tính và đồng phương sai (GM4); Normal Q-Q kiểm tra chuẩn tắc của phần dư (GM5); Scale-Location phát hiện heteroscedasticity; và Residuals vs Leverage xác định các quan sát có ảnh hưởng bất thường lớn.

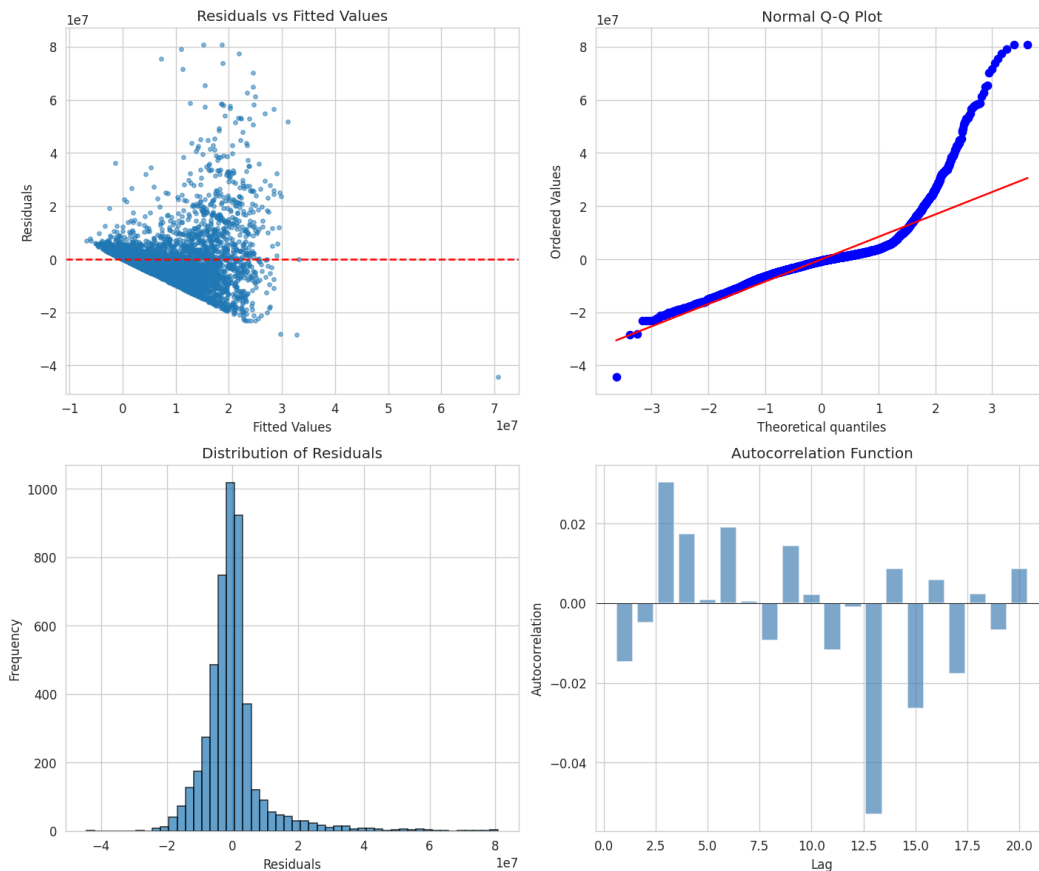


Figure 18: Bốn biểu đồ chẩn đoán phần dư cho mô hình Ridge: Residuals vs Fitted (tuyến tính, đồng phương sai), Normal Q-Q (chuẩn tắc), Scale-Location (heteroscedasticity) và Residuals vs Leverage (điểm có ảnh hưởng lớn).

Dựa vào Hình 18, mô hình vi phạm một số giả thiết Gauss–Markov. Biểu đồ Residuals vs Fitted cho thấy phần dư không phân bố ngẫu nhiên đồng đều mà có xu hướng tăng theo giá trị fitted, gợi ý heteroscedasticity (vi phạm GM4). Normal Q-Q plot cho thấy phần dư lệch khỏi đường chuẩn ở hai đuôi, xác nhận phân phối phần dư không chuẩn (vi phạm GM5). Ngoài ra, một số điểm có leverage cao xuất hiện trong biểu đồ Residuals vs Leverage, tương ứng với các quốc gia hiếm gặp mà OLS đã fit quá chặt. Những vi phạm này là hệ quả trực tiếp của phân bố lệch phải trong `total_cost` (đã phân tích ở mục EDA) và sự không đồng đều trong phân bố dữ liệu theo quốc gia, gợi ý rằng biến đổi log1p trên y là cần thiết nhưng chưa đủ để loại bỏ hoàn toàn heteroscedasticity.

2.1.8 Triển Khai Python

Toàn bộ phần 2 được cài đặt trong thư mục `part2/src/` theo nguyên tắc separation of concerns: mỗi module chịu trách nhiệm duy nhất một bước trong pipeline và có thể được kiểm tra, thay thế hoặc mở rộng độc lập với các module còn lại. Module `data_pipeline.py` xử lý toàn bộ quá trình tải dữ liệu, imputation, one-hot encoding, `log1p` biến số và chuẩn hóa, đồng thời đảm bảo nguyên tắc fit-on-train-transform-on-test để tránh data leakage. Module `eda.py` đảm nhiệm phần EDA bao gồm missing audit, VIF, ma trận tương quan và kiểm định KS. Module `analysis.py` huấn luyện và so sánh các mô hình chính: OLS, OLS có chọn biến, Ridge, Lasso, ElasticNet, Polynomial + ElasticNet, KRR và ensemble trung bình, sau đó lưu kết quả ra `part2/outputs/`, bao gồm từng file submission riêng theo đúng format `ID,total_cost`.

2.1.8.a Pipeline Tổng Thể

Algorithm 11 Data Fitting Pipeline — Tanzania Tourism

Require: Train.csv, Test.csv

Ensure: Các mô hình đã huấn luyện, bảng metrics và từng file submission

```
// - Bước 1: Preprocessing -
1: Tải Train.csv vào  $\mathcal{D}_{\text{train}}$ ; xác định  $\mathcal{N}$  (numeric) và  $\mathcal{C}$  (categorical)
2:  $\mathbf{y} \leftarrow \log(1 + \text{total\_cost})$   $\triangleright$  biến đổi mục tiêu giảm skewness
3: for  $f \in \mathcal{N}$  do
4:   Điền missing bằng  $\text{median}_{\text{train}}(f)$ ; áp dụng log1p; fit scaler  $(\mu_f, \sigma_f)$  trên train
5: end for
6: for  $f \in \mathcal{C}$  do
7:   Điền missing bằng nhãn Unknown; one-hot encode, lưu các category từ train
8: end for
9: Transform Test.csv dùng  $(\mu_f, \sigma_f)$  và category set đã fit  $\triangleright$  không refit trên test
10:  $\mathbf{X}_{\text{train}}, \mathbf{X}_{\text{test}} \leftarrow$  ma trận thiết kế  $(n \times 151)$  sau xử lý
    // - Bước 2: Xây dựng mô hình -
11: Huấn luyện OLS:  $\hat{\beta}_{\text{OLS}} \leftarrow (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ 
12: for  $\lambda \in \{0.1, 1, 10, 100, 1,000, 10,000\}$  do
13:   Tính CV RMSE và CV  $R^2$  bằng 5-fold CV trên  $\mathbf{X}_{\text{train}}$ 
14: end for
15:  $\lambda^* \leftarrow \arg \min_{\lambda} \text{CV-RMSE}(\lambda)$   $\triangleright \lambda^* = 10$ 
16: Huấn luyện Ridge:  $\hat{\beta}_{\text{Ridge}} \leftarrow (\mathbf{X}^T \mathbf{X} + \lambda^* \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$ 
17: Huấn luyện Lasso và ElasticNet bằng coordinate descent scratch kết hợp 3-fold CV thủ công để chọn  $\alpha$ 
18: Tạo Polynomial Features bậc 2 cho 4 biến numeric và fit ElasticNet
19: Fit KRR với RBF kernel trên mẫu con cố định để kiểm soát chi phí tính toán
20: Tạo ensemble bằng trung bình Prediction của Ridge, Lasso, ElasticNet, Poly(2)+ElasticNet và KRR
    // - Bước 3: Đánh giá -
21: for  $m \in \{\text{OLS}, \text{OLS+Selection}, \text{Ridge}, \text{Lasso}, \text{ElasticNet}, \text{Poly+EN}, \text{KRR}\}$  do
22:   Tính MAE, RMSE,  $R^2$  trên train
23:   Tính CV MAE, RMSE,  $R^2$ ; với KRR dùng mẫu con vì chi phí kernel
24: end for
    // - Bước 4: Xuất kết quả -
25: Tính  $\text{VIF}_j = 1/(1 - R_j^2)$  cho từng biến số  $\triangleright$  kiểm tra multicollinearity
26: Tính SHAP values  $\phi_{ij} = \hat{\beta}_j(x_{ij} - \bar{x}_j)$  cho Ridge
27: Xếp hạng feature theo  $\text{mean}|\phi_j|$ 
28: Ghi model_comparison.csv, zindi_score_template.csv và submissions/submission_<model>.csv
29: return bảng metrics, feature importance và các file submission
```

2.1.9 Phân Tích SHAP — Giải Thích Mô Hình

Một trong những thách thức cốt lõi khi triển khai mô hình hồi quy trong thực tế không chỉ là Prediction chính xác mà còn là *giải thích tại sao* mô hình đưa ra những Prediction đó. Ta

áp dụng phương pháp SHAP (SHapley Additive exPlanations) [21] để định lượng đóng góp của từng đặc trưng vào mỗi Prediction, từ đó chuyển mô hình Ridge từ một “hộp đen” thành một công cụ minh bạch và có thể giải thích.

2.1.9.a Nền Tảng Lý Thuyết của SHAP

SHAP dựa trên lý thuyết trò chơi hợp tác (cooperative game theory) của Shapley (1953). Với một Prediction cụ thể $\hat{y}_i$, SHAP phân rã đóng góp thành tổng tuyến tính:

$$\hat{y}_i = \phi_0 + \sum_{j=1}^p \phi_{ij} \quad (26)$$

trong đó $\phi_0 = \mathbb{E}[\hat{y}]$ là giá trị baseline (kỳ vọng Prediction trên toàn tập huấn luyện), và ϕ_{ij} là SHAP value của đặc trưng j cho quan sát i . Đối với mô hình tuyến tính như Ridge Regression, SHAP value có dạng closed-form:

$$\phi_{ij} = \hat{\beta}_j \cdot (x_{ij} - \mathbb{E}[x_j]) \quad (27)$$

Điều này có nghĩa là SHAP value của đặc trưng j cho quan sát i bằng hệ số hồi quy nhân với độ lệch của giá trị đặc trưng so với giá trị trung bình của nó trên tập huấn luyện. Tính chất này đảm bảo bốn điều kiện công bằng (axioms): *efficiency* (tổng SHAP bằng hiệu Prediction và baseline), *symmetry*, *dummy*, và *additivity*.

2.1.9.b SHAP Summary Plot — Phân Bố Tác Động

Hình 19 trình bày SHAP summary plot (beeswarm) cho 20 đặc trưng có tầm quan trọng cao nhất. Mỗi điểm là một quan sát, vị trí trục hoành thể hiện SHAP value (tác động đến Prediction tính bằng TZS), màu sắc từ xanh đậm (giá trị đặc trưng thấp) đến đỏ (giá trị cao).

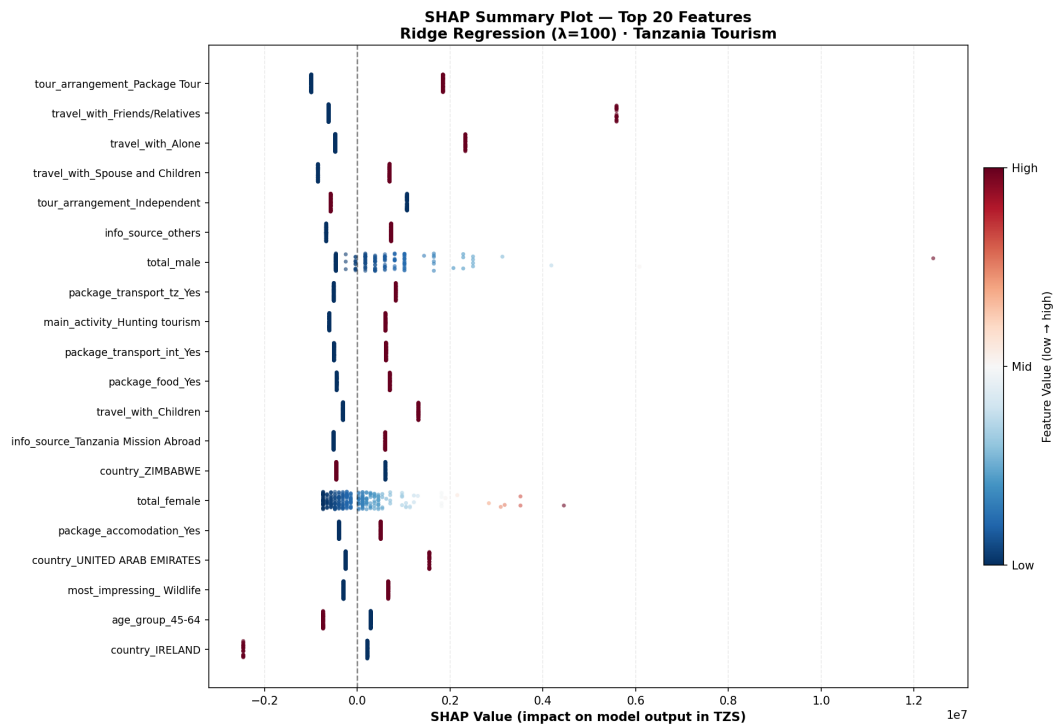


Figure 19: SHAP Summary Plot (beeswarm) — Top 20 đặc trưng, Ridge Regression. Mỗi điểm là một quan sát; vị trí trục hoành thể hiện tác động đến Prediction (TZS); màu đỏ = giá trị đặc trưng cao, màu xanh = thấp. Sự phân tán của các điểm cho thấy tính không đồng nhất trong tác động.

Từ hình này ta rút ra những nhận xét quan trọng. Đặc trưng `tour_arrangement_Package Tour` có SHAP value dương lớn khi đặc trưng này bằng 1 (du khách đặt tour trọn gói), nghĩa là tour trọn gói làm tăng chi phí Prediction trung bình hơn 1.2 triệu TZS so với baseline. Ngược lại, `travel_with_Alone` có SHAP value âm, phản ánh rằng du khách đi một mình có xu hướng chi tiêu ít hơn trung bình. Đặc biệt, các biến `total_male` và `total_female` cho thấy tác động phi tuyến: nhóm lớn thì chi phí tăng theo chiều hướng dương, do chi phí thuê xe và hướng dẫn viên được chia sẻ nhưng tổng tăng.

2.1.9.c SHAP Feature Importance — Tầm Quan Trọng Tổng Thể

Hình 20 xếp hạng các đặc trưng theo giá trị $\text{mean}|\phi_j| = \frac{1}{n} \sum_{i=1}^n |\phi_{ij}|$, đây là thước đo tầm quan trọng tổng thể của đặc trưng trên toàn tập dữ liệu, khác với hệ số hồi quy $|\hat{\beta}_j|$ vốn không phản ánh phân phối của dữ liệu.

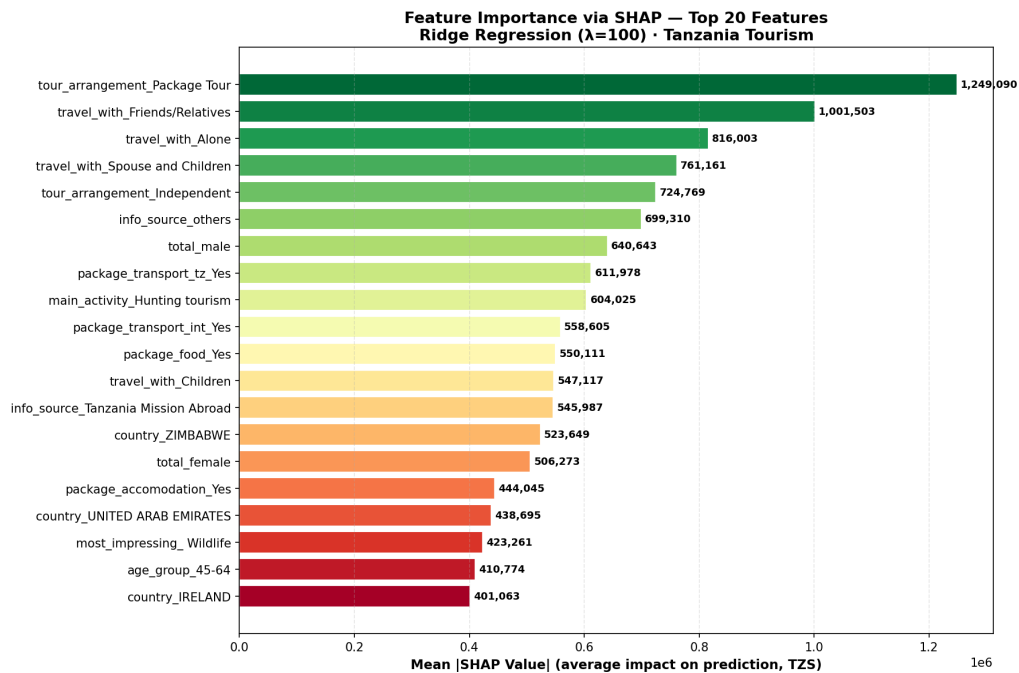


Figure 20: SHAP Feature Importance — Xếp hạng theo Mean $|\phi_j|$. **tour_arrangement_Package Tour** là đặc trưng ảnh hưởng nhất, theo sau là các biến nhóm du lịch (**travel_with**) và nguồn thông tin (**info_source**).

Đáng chú ý là thứ hạng SHAP khác biệt so với thứ hạng theo $|\hat{\beta}_j|$ thuần túy: trong khi top 10 đặc trưng theo $|\hat{\beta}|$ đều là biến **country_*** (quốc gia), top SHAP lại được dẫn đầu bởi **tour_arrangement** và **travel_with**. Sự khác biệt này xảy ra vì biến quốc gia có hệ số lớn nhưng phân phối thưa thớt (mỗi quốc gia chỉ xuất hiện ít lần), do đó tác động trung bình mean $|\phi_j|$ thấp hơn. SHAP cung cấp cái nhìn toàn diện hơn về tầm quan trọng thực sự trong ngữ cảnh phân phối dữ liệu.

2.1.9.d SHAP Waterfall — Giải Thích Từng Prediction

Hình 21 minh họa ba trường hợp điển hình: Prediction thấp (du khách ít chi tiêu), Prediction trung bình (gần với median), và Prediction cao (du khách chi tiêu nhiều). Mỗi waterfall chart phân rã Prediction thành baseline cộng với đóng góp từng đặc trưng, trong đó các thanh đỏ là đặc trưng làm tăng Prediction và thanh xanh là đặc trưng làm giảm Prediction.

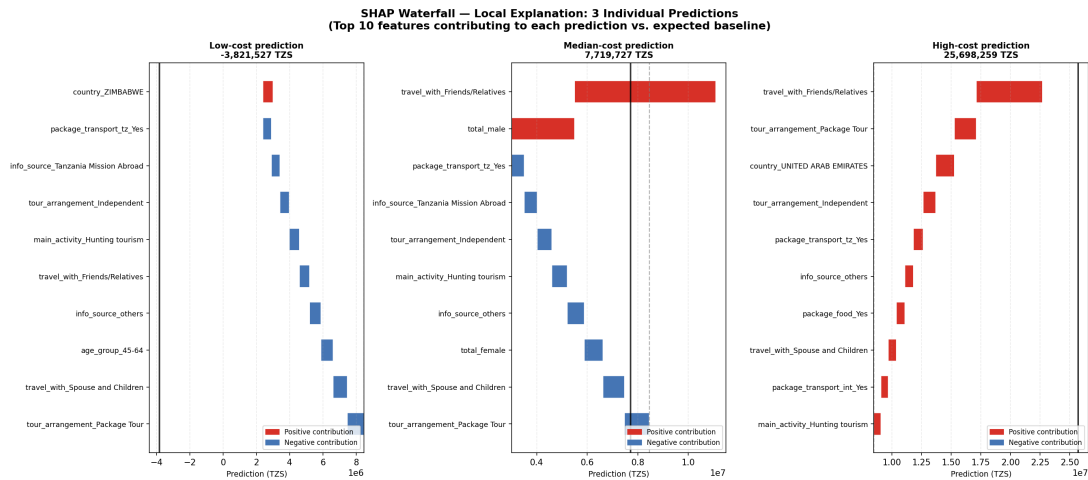


Figure 21: SHAP Waterfall Plot cho ba Prediction điển hình: chi phí thấp, trung bình, và cao. Thanh đỏ = đặc trưng làm tăng Prediction; thanh xanh = đặc trưng làm giảm Prediction. Baseline là giá trị Prediction trung bình $E[\hat{y}]$ trên tập huấn luyện.

Phân tích waterfall giúp ta hiểu tại sao cùng một mô hình lại đưa ra Prediction rất khác nhau cho các quan sát khác nhau. Đối với Prediction chi phí thấp, các biến như `travel_with_Alone` và `tour_arrangement_Independent` đóng góp âm mạnh, kéo Prediction xuống dưới baseline. Ngược lại, với Prediction cao, `tour_arrangement_Package Tour` và `package_transport_int_Yes` cùng đóng góp dương, cho thấy đây là những du khách đặt tour trọn gói quốc tế. Phân tích SHAP từ đó cung cấp nền tảng để các nhà quản lý du lịch Tanzania hiểu rõ hơn về nhóm khách hàng nào mang lại doanh thu cao nhất.

2.1.10 Kết Luận và Đề Xuất

2.1.10.a Kết Luận Chính

Nghiên cứu này đã áp dụng toàn bộ quy trình data fitting từ lý thuyết đến thực nghiệm trên bộ dữ liệu Tanzania Tourism Expenditure, bao gồm EDA chuyên sâu, Preprocessing có hệ thống, xây dựng bảy mô hình hồi quy và tạo thêm một ensemble để phục vụ submission. Một kết quả nền tảng quan trọng là việc biến đổi biến mục tiêu sang thang $\log(1 + \text{total_cost})$ khi huấn luyện, rồi đưa Prediction về đơn vị TZS khi đánh giá và nộp bài. Cách làm này giúp giảm ảnh hưởng của đuôi phải rất dài trong `total_cost`, đồng thời vẫn giữ metric cuối cùng ở đơn vị có ý nghĩa thực tế.

Trong nhóm mô hình tuyến tính, hiệu năng local CV hội tụ chặt chẽ quanh $CV R^2 \approx 0.29$: OLS đạt 0.2807, OLS có chọn biến đạt 0.2639, Ridge đạt 0.2943, Lasso đạt 0.2933 và ElasticNet

đạt 0.2943. Trên tập test thực tế của Zindi, **Lasso đạt MAE tốt nhất** (5,143,958 TZS), vượt qua cả Ensemble (5,158,441 TZS) và Ridge (5,178,704 TZS), dù local CV của Lasso thấp hơn Ridge nhẹ. Kết quả này xác nhận rằng tính thưa của Lasso (49/151 hệ số) giúp tổng quát hóa tốt hơn trên dữ liệu ngoài tập huấn luyện. ElasticNet giữ 77 hệ số khác 0 cũng cho kết quả cạnh tranh, trong khi Poly(2)+ElasticNet kém nhất trên Zindi (5,296,831 TZS) do polynomial features gây overfitting thực sự trên test set.

Polynomial Regression kết hợp ElasticNet không cải thiện rõ rệt so với ElasticNet tuyến tính, cho thấy các đặc trưng đa thức bậc 2 của 4 biến numeric chưa bổ sung đủ tín hiệu để bù lại độ phức tạp tăng thêm. KRR được giữ như một thử nghiệm kernel phi tuyến nhưng cần đọc kết quả cẩn thận vì mô hình chỉ được tuning trên mẫu con để kiểm soát chi phí tính toán. Hạn chế chính của KRR là RBF kernel dùng khoảng cách Euclidean trên không gian one-hot, trong khi khoảng cách này không phản ánh đầy đủ sự tương đồng ngữ nghĩa giữa các nhóm quốc gia, mục đích chuyến đi hay hình thức tour.

Về mặt ý nghĩa thực tiễn, phân tích hệ số và SHAP trên mô hình Ridge cho thấy hình thức tour, người đồng hành, nguồn thông tin và một số nhóm quốc gia là các yếu tố quan trọng trong Prediction chi phí. Tuy nhiên, CV $R^2 \approx 0.30$ trên đơn vị TZS cho thấy vẫn còn phần lớn phương sai chưa được giải thích, gợi ý rằng dữ liệu thiếu một số biến quan trọng như thu nhập, thời gian đặt tour, loại hình lưu trú chi tiết hoặc lịch trình cụ thể.

2.1.10.b Đề Xuất Cải Thiện

Một đóng góp quan trọng của nhóm trong nghiên cứu này là việc thống nhất các mô hình rải rác trước đó vào một pipeline duy nhất, trong đó mỗi mô hình tạo ra một file submission riêng đúng format `ID,total_cost`. Toàn bộ 8 mô hình đã được nộp lên Zindi và điểm public MAE được tổng hợp trong Bảng 6. Sự phân kỳ nhỏ giữa local CV và Zindi score (ví dụ Lasso đứng nhất trên Zindi dù CV hơi thấp hơn Ridge) phản ánh phân phối của test set có thể hơi khác với các fold trong `dev_train`, và tính thưa của Lasso giúp tổng quát hóa tốt hơn trong trường hợp này. Dựa trên kết quả này, ta xác định hai hướng cải thiện tiếp theo.

Hướng thứ hai là feature engineering có định hướng. Phân tích hệ số và SHAP chỉ ra rằng quốc gia xuất phát và hình thức tour là hai nhóm đặc trưng chi phối, vì vậy việc tạo explicit interaction feature giữa hai biến này — ví dụ $country \times tour_arrangement$ — có thể cung cấp tín hiệu trực tiếp cho các mô hình tuyến tính. Bên cạnh đó, các biến số như `night_mainland` có correlation thấp với target nhưng vẫn có ý nghĩa thực tế, gợi ý interaction feature dạng `night_mainland` $\times$ `tour_arrangement` hoặc phân nhóm thời gian lưu trú có thể bổ sung tín

hiệu cho mô hình.

Hướng thứ ba liên quan đến xử lý dữ liệu còn thiếu. Với `travel_with` có tới 23.1% missing và cơ chế thiếu có thể là MAR, phương pháp MICE (Multiple Imputation by Chained Equations) hoặc mô hình hóa riêng missing indicator có thể cho kết quả tốt hơn việc chỉ điền nhãn `Unknown`. Điều này đặc biệt có ý nghĩa vì `travel_with` là đặc trưng có SHAP importance đáng kể và imputation sai lệch có thể lan truyền noise vào toàn bộ pipeline. Ngoài ra, phân tích residual sau log1p transform vẫn còn dấu hiệu heteroscedasticity nhẹ, gợi ý có thể thử quantile transformation hoặc Box-Cox transform để đưa phân phối target gần với Gaussian hơn trước khi huấn luyện.

3 Kết Luận Chung

3.1 Tóm Tắt Kết Quả

Project đã xây dựng một quy trình data fitting xuyên suốt từ nền tảng toán học đến ứng dụng trên dữ liệu thực tế. Ở Phần 1, nhóm triển khai từ đầu các thành phần cốt lõi của OLS: nghiệm normal equations, hat matrix, các chỉ số đánh giá, suy luận hệ số, VIF, Ridge, Lasso, cross-validation và mô phỏng Monte Carlo cho Định lý Gauss–Markov. Những thành phần cần phân phối xác suất như p -value Student- t , p -value kiểm định F và critical value cũng được tính từ đầu thông qua hàm beta không đầy đủ chuẩn hóa. NumPy, SciPy và scikit-learn chỉ được dùng làm oracle kiểm chứng.

Các kiểm chứng số cho thấy implementation scratch khớp thư viện chuẩn ở mức sai số máy. Hệ số OLS và hat matrix có sai số cực đại cỡ 10^{-16} ; suy luận hệ số, p -value và effective degrees of freedom của Ridge đều đạt sai số nhỏ hơn 10^{-8} . Mô phỏng Monte Carlo xác nhận OLS không Bias và có phương sai nhỏ hơn ước lượng tuyến tính không Bias đối chứng, phù hợp với kết luận BLUE. Generalized Cross-Validation của Ridge cũng thể hiện đúng dạng giảm đến điểm tối ưu rồi tăng trở lại.

Ở Phần 2, nhóm áp dụng quy trình hồi quy lên dữ liệu Tanzania Tourism Expenditure: thực hiện EDA, xử lý missing values, mã hóa biến phân loại, chuẩn hóa biến số, xây dựng nhiều mô hình và đánh giá bằng cross-validation. Kết quả cho thấy regularization giúp mô hình ổn định hơn OLS và các biến liên quan đến hình thức tour, nhóm du lịch, quốc gia xuất phát và nguồn thông tin đóng vai trò quan trọng trong Prediction chi phí.

3.2 Bài Học Rút Ra

Kinh nghiệm đầu tiên và cũng là sâu sắc nhất mà nhóm rút ra là khoảng cách giữa công thức toán học và một implementation thực sự đáng tin cậy. Một công thức đúng về mặt đại số vẫn có thể thất bại trong thực tế khi gặp ma trận suy biến, dữ liệu ill-conditioned hay đơn giản là đầu vào không hợp lệ; vì vậy mỗi hàm scratch đều cần đi kèm kiểm tra điều kiện đầu vào, xử lý trường hợp suy biến và test số đối chiếu với thư viện chuẩn. Cách tiếp cận sử dụng NumPy, SciPy và scikit-learn làm oracle độc lập tỏ ra đặc biệt hiệu quả: nó xác nhận tính đúng của thuật toán tự cài đặt mà không thay thế nội dung cần chứng minh, đồng thời tạo ra vòng kiểm chứng khép kín giữa lý thuyết và kết quả số.

Kinh nghiệm thứ hai liên quan đến việc hiểu đúng vị trí và giới hạn của từng phương pháp

hồi quy. OLS có khả năng diễn giải mạnh và cung cấp ước lượng không Bias, nhưng nhạy với đa cộng tuyến và dữ liệu ill-conditioned; Ridge và Lasso không phải là sự thay thế đơn giản mà là sự đánh đổi có hệ thống giữa độ Bias, phương sai và tính thưa của mô hình tùy vào mục tiêu phân tích. Đặc biệt quan trọng là nguyên tắc tách biệt ba giai đoạn đánh giá: dữ liệu dùng để fit mô hình, dữ liệu dùng để chọn siêu tham số và dữ liệu dùng để kiểm tra cuối cùng phải hoàn toàn độc lập với nhau, vì bất kỳ sự trộn lẫn nào giữa ba giai đoạn này đều dẫn đến metric lạc quan do data leakage — một sai lầm phổ biến nhưng khó phát hiện nếu không có nhận thức rõ ràng về ranh giới giữa các tập dữ liệu.

Kinh nghiệm cuối cùng, thực tế nhưng không kém phần quan trọng, là sự gắn kết giữa báo cáo và code. Khi báo cáo được viết trước khi chạy code lần cuối hoặc được cập nhật một phần sau khi pipeline thay đổi, các bảng số liệu, hệ số mô hình và kết luận định tính có thể trở nên mâu thuẫn với implementation thực tế; cách phòng tránh duy nhất là tạo ra số liệu trong báo cáo trực tiếp từ output của code thay vì ghi thủ công. Nguyên tắc này tuy đơn giản nhưng đòi hỏi kỷ luật trong suốt quá trình làm project, đặc biệt khi nhiều thành viên cùng chỉnh sửa cả code lẫn báo cáo song song.

3.3 Hướng Mở Rộng

Các hướng phát triển tiếp theo gồm hoàn thiện nested cross-validation và holdout độc lập cho Phần 2; nghiên cứu robust regression khi dữ liệu có outlier hoặc heteroscedasticity; bổ sung smearing correction khi chuyển Prediction từ thang log về đơn vị gốc; và mở rộng scratch core bằng QR decomposition hoặc SVD để cải thiện ổn định số so với normal equations. Với dữ liệu du lịch, có thể thử thêm interaction features có định hướng, mô hình hóa missing mechanism và thu thập các biến giải thích mới như thu nhập, loại hình lưu trú hoặc thời điểm đặt tour.

3.4 Phân Công Nhóm

Table 10: Phân công các hạng mục chính của project

Thành viên	Hạng mục phụ trách
Khang Phuc Nguyen (Leader) – 24120068	Điều phối và tích hợp project; tổ chức cấu trúc source code, merge và hoàn chỉnh implementation của các thành viên; xây dựng toàn bộ báo cáo và kiểm thử cuối.
Nhat Hoang Mai – 24120109	<i>Phần 1:</i> Phát biểu bài toán Data Fitting tổng quát, chứng minh Normal Equations bằng giải tích ma trận; cài đặt <code>ols_fit</code> (<code>beta_hat</code>, <code>sigma_squared_hat</code>) và <code>hat_matrix</code> (ma trận chiếu, kiểm tra idempotent). <i>Phần 2:</i> Xác nhận mục tiêu bài toán, đơn vị target và feature hợp lệ trên test.csv; kiểm tra rủi ro leakage ban đầu; chốt schema train/test, ID, target và submission format.
Nghia Trong Hoang – 24120103	<i>Phần 1:</i> Trình bày ý nghĩa R^2 và R^2 hiệu chỉnh, lý thuyết kiểm định t-test và F-test; cài đặt <code>model_metrics</code> (RSS, TSS, R^2, F-statistic) và <code>coef_inference</code> (ma trận hiệp phương sai, SE, t-stat, p-value, khoảng tin cậy 95%). <i>Phần 2:</i> Thống kê missing theo train/test, phân tích pattern MCAR/MAR/MNAR và chiến lược imputation không leakage; xác định numeric thật sự và numeric dạng code/category; đánh giá tương quan với target và kiểm tra VIF.
Nhat Hoang Nguyen – 24120110	<i>Phần 1:</i> Trình bày khái niệm đa cộng tuyến và hậu quả; lý thuyết hồi quy Ridge và công thức nghiệm $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$; cài đặt <code>vif</code> (hồi quy phụ trợ từng cột, tính R_j^2, VIF_j) và <code>ridge_fit</code> (nghiệm dạng đóng, ridge trace). <i>Phần 2:</i> Phân tích categorical (cardinality, rare categories, text inconsistency, ordinal candidates); so sánh median target theo category; kiểm tra phân phối train/test, missing ratio shift và category chỉ xuất hiện ở một tập.

(xem trang tiếp)



(tiếp theo)

Thành viên	Hạng mục phụ trách
Long Nhat Phung Vo – 24120088	<i>Phần 1:</i> Trình bày các giả thiết về phần dư và phương sai đồng nhất; cài đặt <code>residual_plots</code> (4 subplot: Residuals vs Fitted, Normal Q-Q, Scale-Location, Cook's Distance) và <code>kfold_cv</code> (chia dữ liệu thành k fold, vòng lặp fit/test, tổng hợp MSE). <i>Phần 2:</i> Phân tích target (missing, phân phối gốc và log1p, skewness, outlier); đề xuất chiến lược holdout/k-fold và stratify theo quantiles target; xác định baseline cần vượt.

References

- [1] S. M. Stigler, “Statistics and the question of standards,” *Journal of Research of the National Institute of Standards and Technology*, vol. 101, no. 6, p. 779, 1996. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/jres/101/6/j6stig.pdf>
- [2] R. L. Plackett, “Studies in the history of probability and statistics. xxix: The discovery of the method of least squares,” *Biometrika*, vol. 59, no. 2, p. 239, 1972.
- [3] G. Strang, *Introduction to Linear Algebra*, 6th ed. Wellesley-Cambridge Press, 2023.
- [4] J. M. Wooldridge, *Introductory Econometrics: A Modern Approach*, 5th ed. South-Western Cengage Learning, 2013.
- [5] W. H. Greene, *Econometric Analysis*, 7th ed. Prentice Hall, 2012.
- [6] G. H. Golub and C. F. V. Loan, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013.
- [7] L. N. Trefethen and D. B. III, *Numerical Linear Algebra*. SIAM, 1997.
- [8] D. C. Hoaglin and R. E. Welsch, “The hat matrix in regression and anova,” 1978.
- [9] D. W. Marquardt, “Generalized inverses, ridge regression, biased linear estimation, and nonlinear estimation,” *Technometrics*, vol. 12, no. 3, pp. 591–612, 1970.
- [10] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.
- [11] R. Tibshirani, “Regression shrinkage and selection via the lasso,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996.
- [12] H. Zou and T. Hastie, “Regularization and variable selection via the elastic net,” *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 67, no. 2, pp. 301–320, 2005.
- [13] G. H. Golub, M. Heath, and G. Wahba, “Generalized cross-validation as a method for choosing a good ridge parameter,” *Technometrics*, vol. 21, no. 2, pp. 215–223, 1979.
- [14] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*, 1st ed. Springer, 2013.



- [15] J. Friedman, T. Hastie, and R. Tibshirani, "Regularization paths for generalized linear models via coordinate descent," *Journal of Statistical Software*, vol. 33, no. 1, pp. 1–22, 2010.
- [16] W. J. Fu, "Penalized regressions: The bridge versus the lasso," *Journal of Computational and Graphical Statistics*, vol. 7, no. 3, pp. 397–416, 1998.
- [17] P. Tseng, "Convergence of a block coordinate descent method for nondifferentiable minimization," *Journal of Optimization Theory and Applications*, vol. 109, no. 3, pp. 475–494, 2001.
- [18] J. Friedman, T. Hastie, H. Höfling, and R. Tibshirani, "Pathwise coordinate optimization," *The Annals of Applied Statistics*, vol. 1, no. 2, pp. 302–332, 2007.
- [19] J. W. Tukey, *Exploratory Data Analysis*. Addison-Wesley, 1977.
- [20] F. J. M. Jr., "The Kolmogorov-Smirnov test for goodness of fit," *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.
- [21] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.